



Razvan BIZOI

Sénior Data Scientist



razvan@bizoi.fr
06 85 20 70 68
44, rue Paul Claudel
67200 STRASBOURG

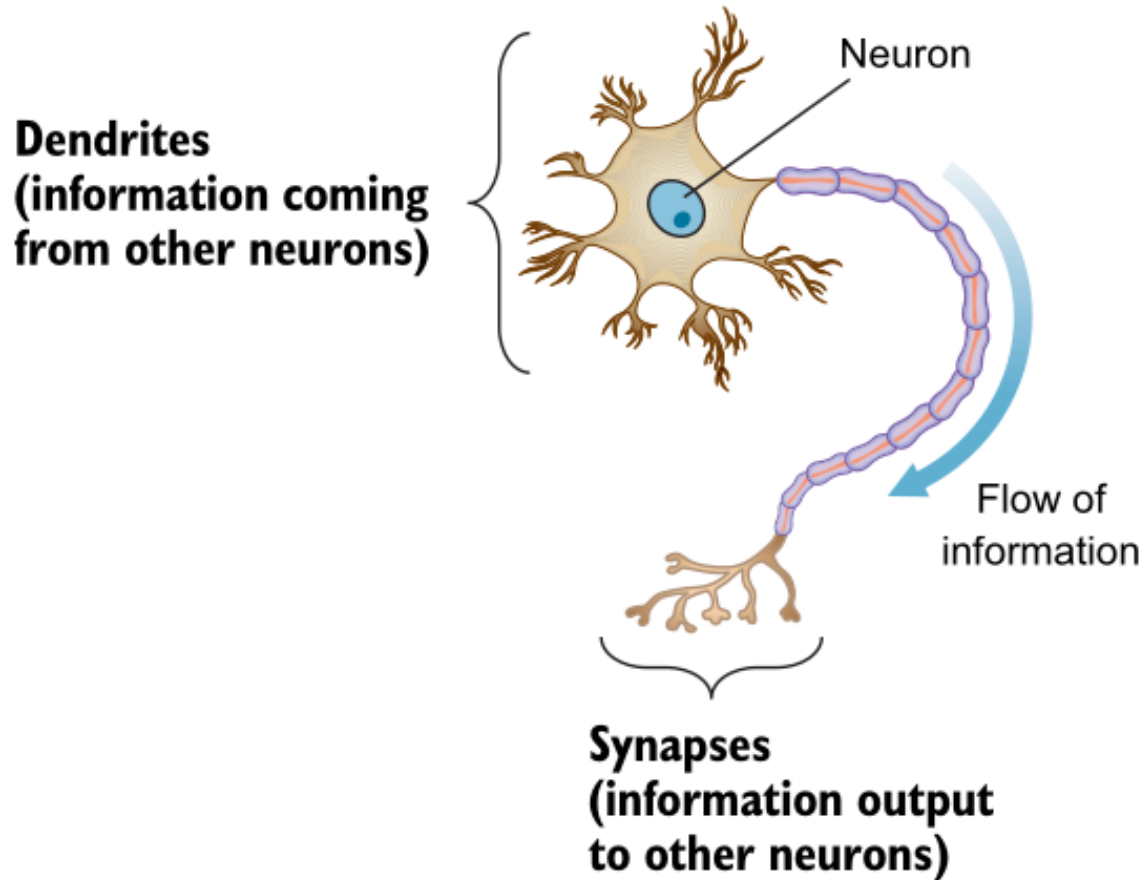
Deep neural networks

- ❑ Introduction
- ❑ Convolutional neural networks CNN
- ❑ Model Architectures
- ❑ Object Detection
- ❑ Segmentation
- ❑ Instance Segmentation
- ❑ Autoencoders

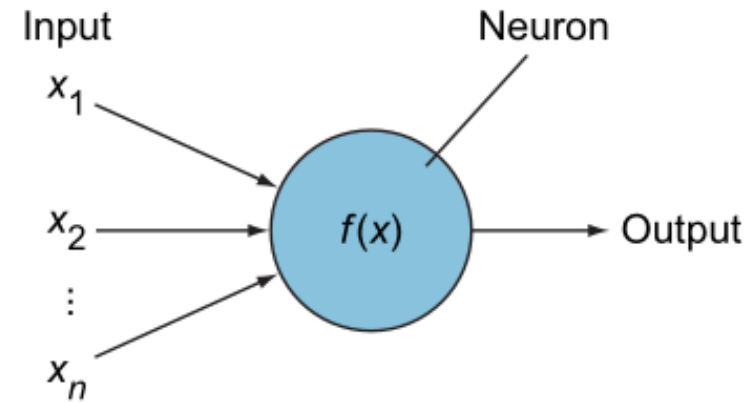
Biological Neuron

Computer vision

Biological neuron



Artificial neuron

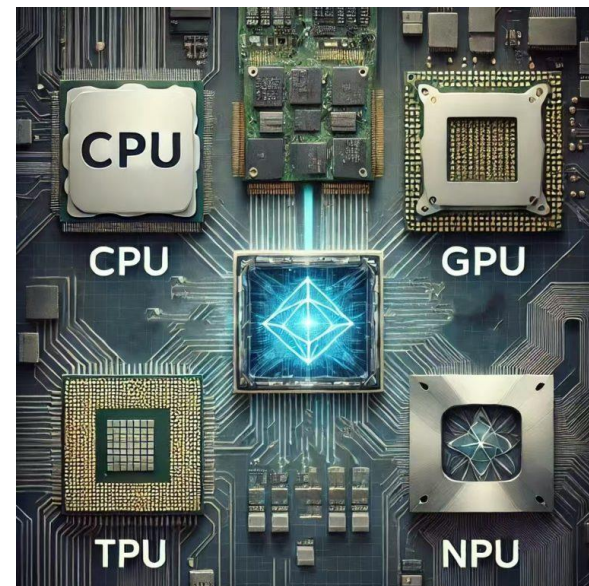


AI Processors — CPU, GPU, TPU, NPU

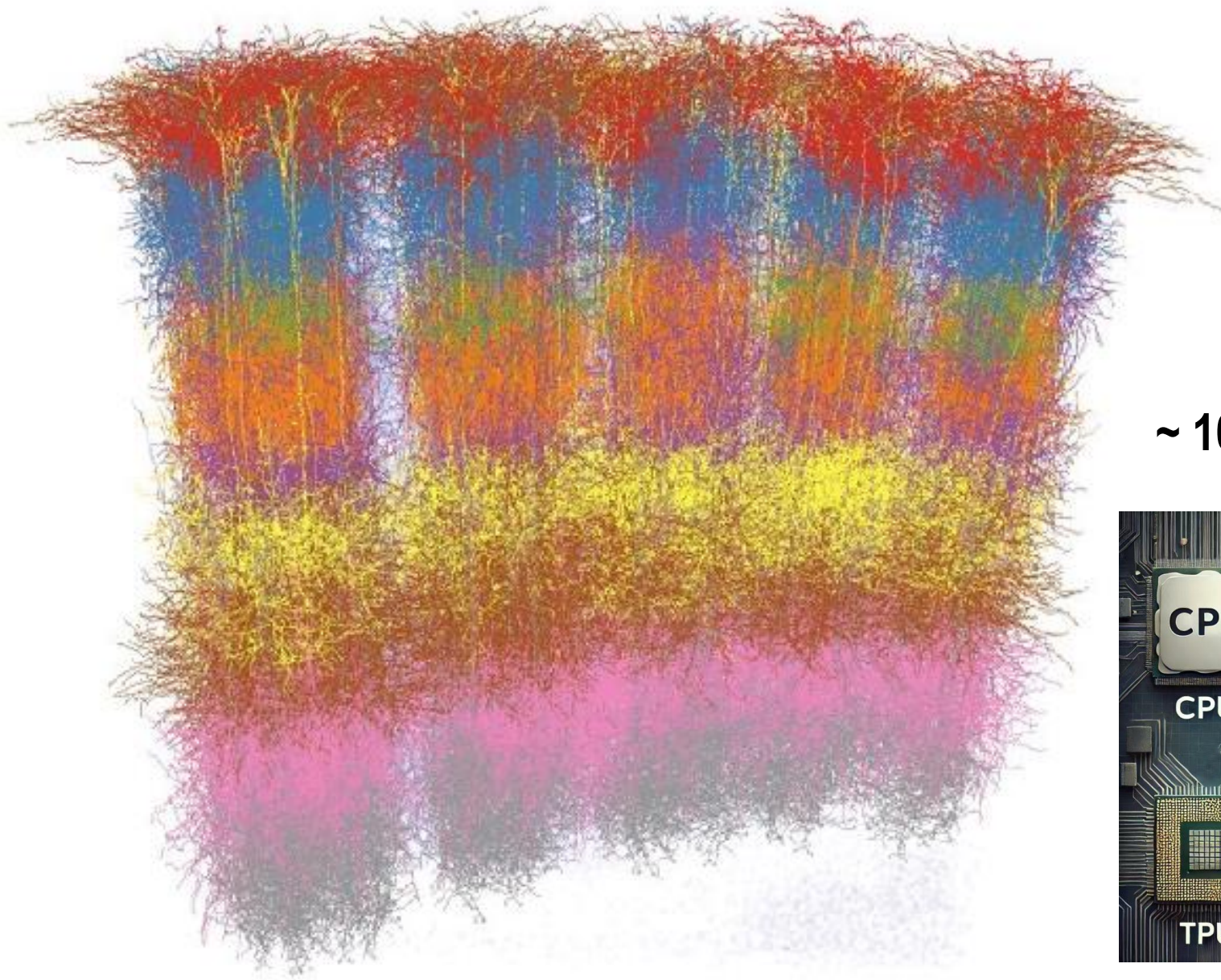
- CPU (Central Processing Unit) ~2.9 FP32 TFLOPS
- GPU (Graphics Processing Unit) ~312 TFLOPS
- TPU (Tensor Processing Unit) ~42.5 EFLOPS (2025)
- NPU (Neural Processing Unit) ~45 TOPS

TFLOPS measure floating-point throughput; TOPS (Tera Operations Per Second) often indicates integer or mixed-precision operations used in AI workloads

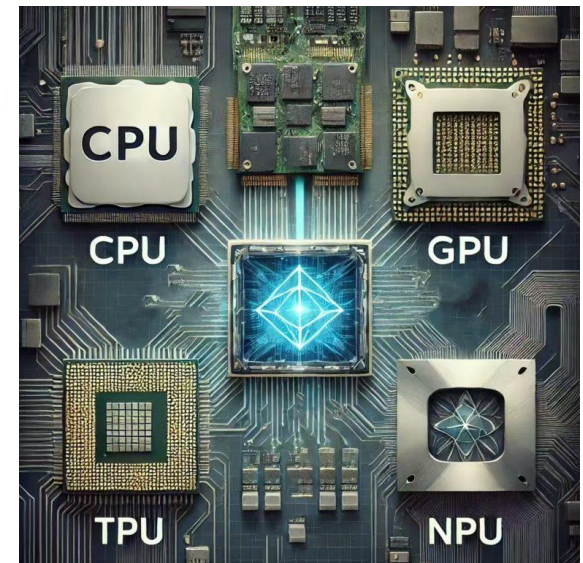
yotta	[Y]	10^{24}	= 1 000 000 000 000 000 000 000 000 000
zetta	[Z]	10^{21}	= 1 000 000 000 000 000 000 000 000 000
exa	[E]	10^{18}	= 1 000 000 000 000 000 000 000 000
peta	[P]	10^{15}	= 1 000 000 000 000 000 000
tera	[T]	10^{12}	= 1 000 000 000 000
giga	[G]	10^9	= 1 000 000 000



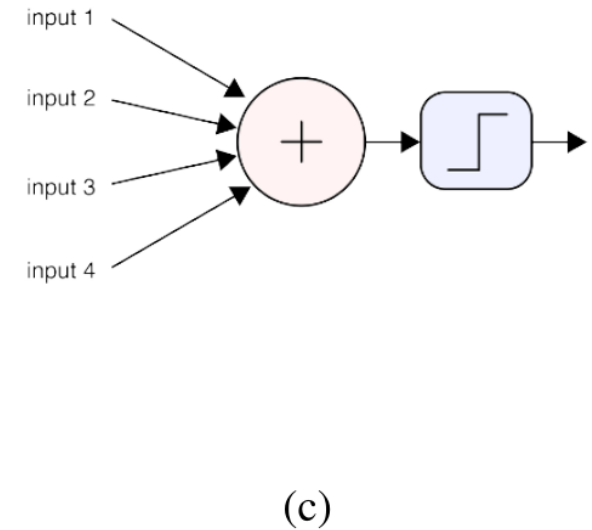
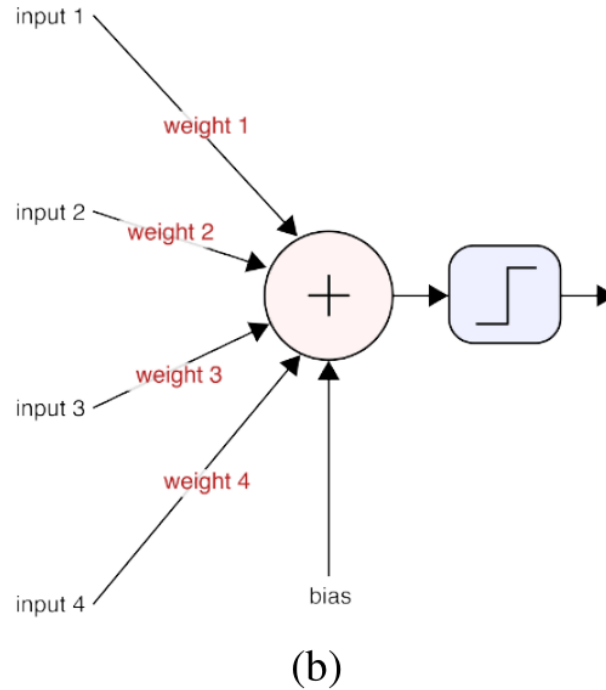
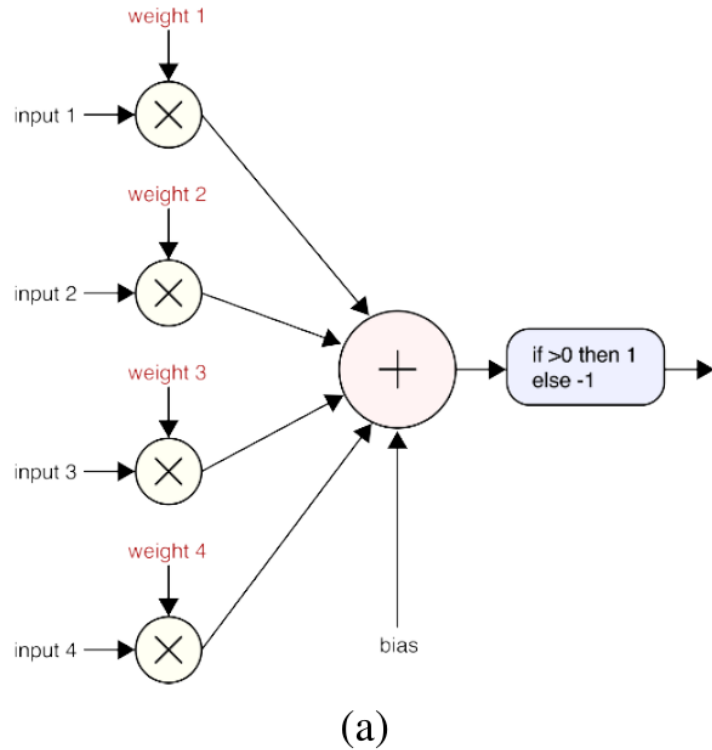
Biological Neurons



~ 100 000 x GPU

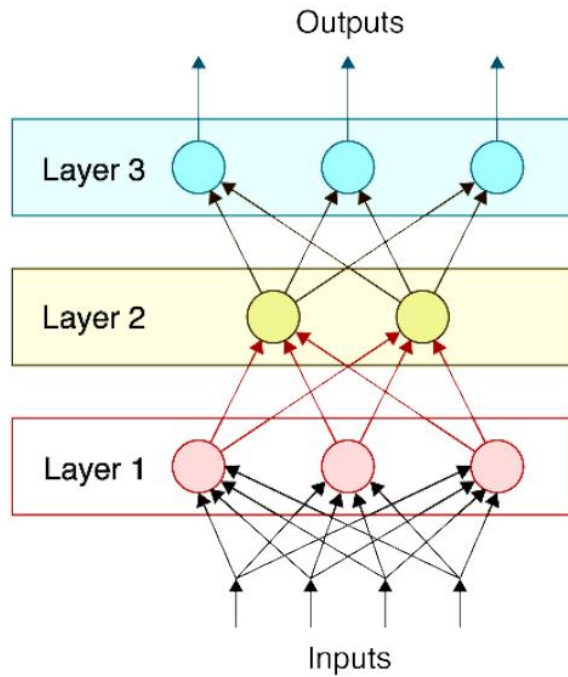


A perceptron unit

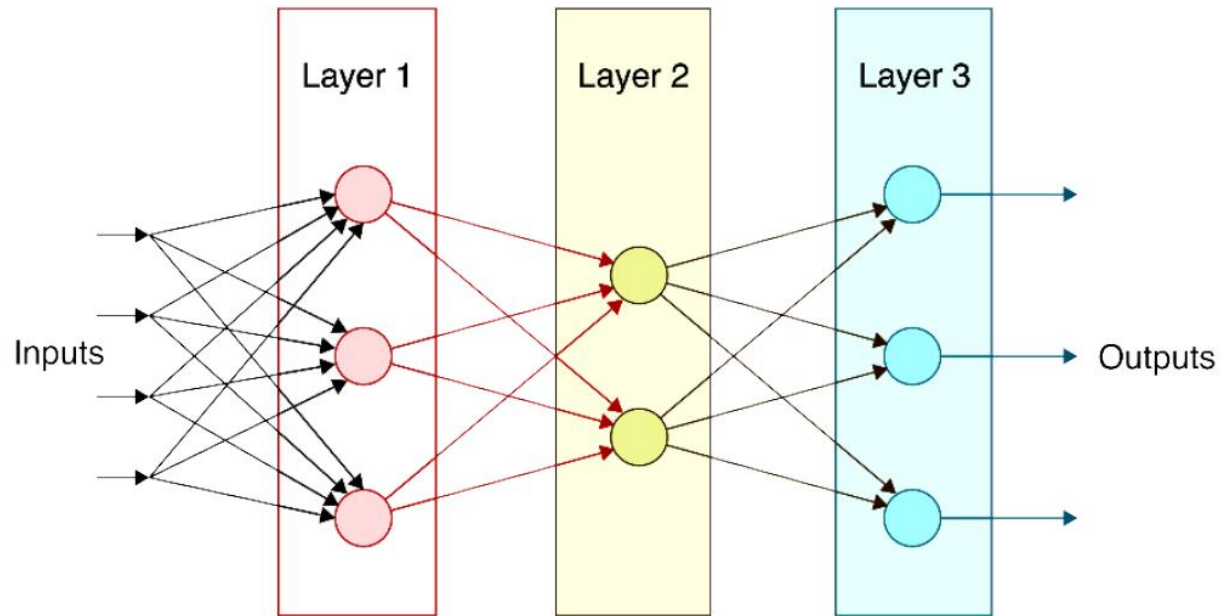


A perceptron unit (a) explicitly showing the weights being multiplied by the inputs, (b) with the weights written on the input connections, and (c) the most common form, with the weights and bias omitted. A non-linear activation function follows the weighted summation. © Glassner (2018)

Deep neural networks



(a)

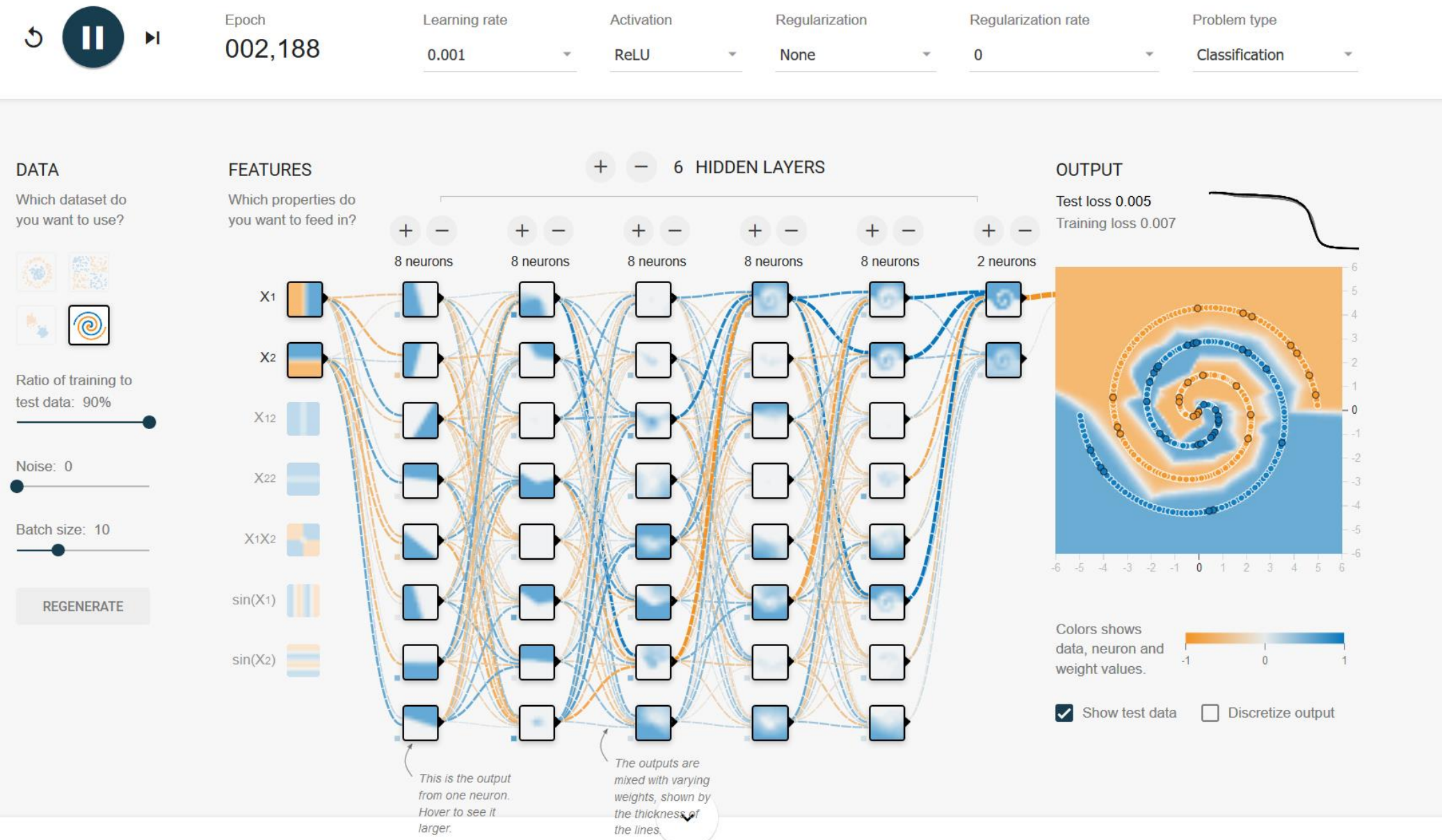


(b)

Two different ways to draw neural networks: (a) inputs at bottom, outputs at top, (b) inputs at left, outputs at right. © Glassner (2018)

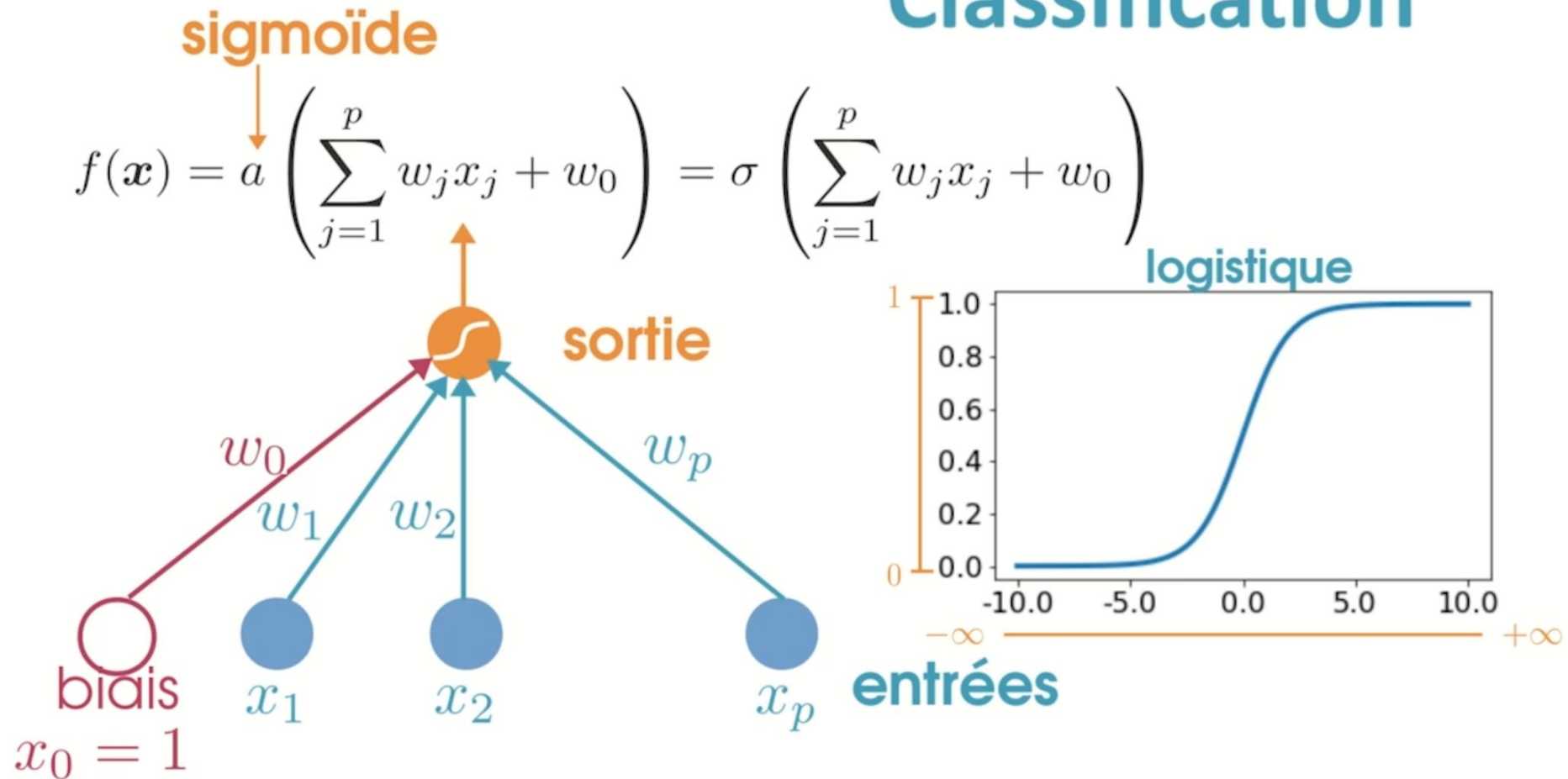
MLP

<https://playground.tensorflow.org/>

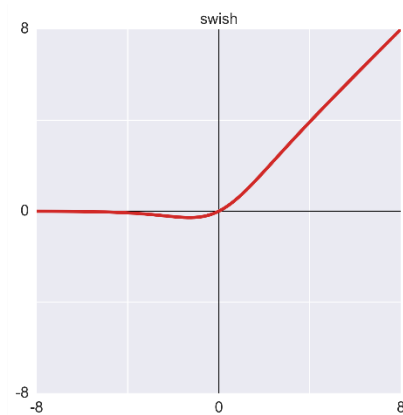
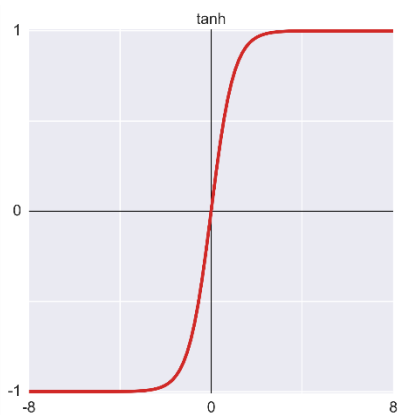
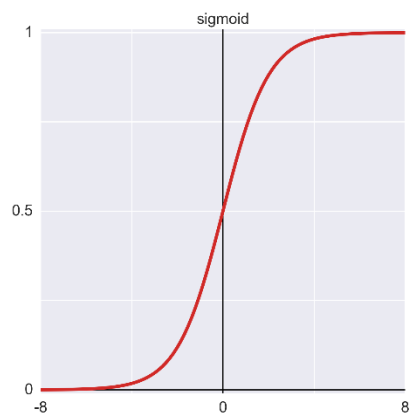
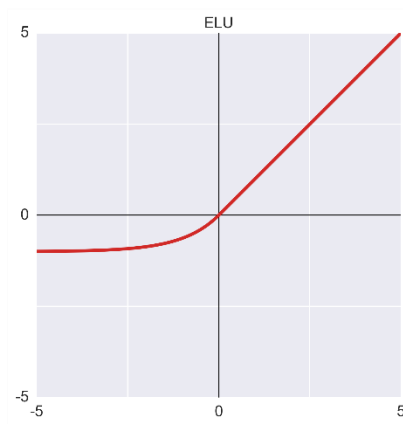
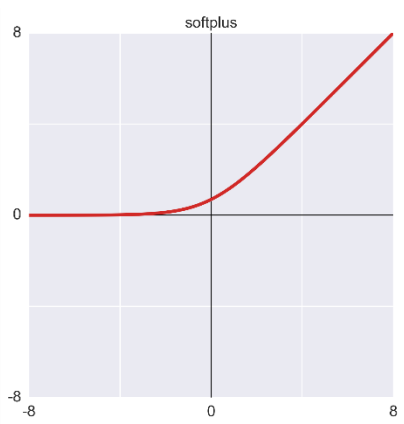
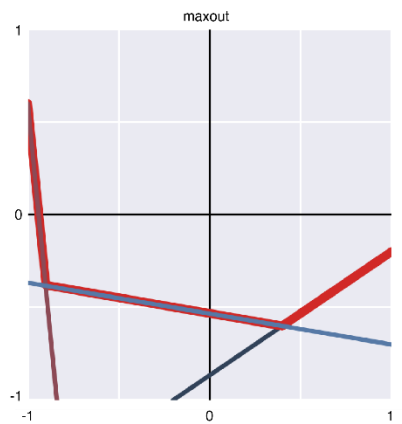
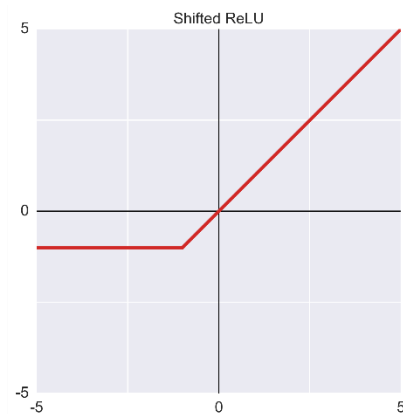
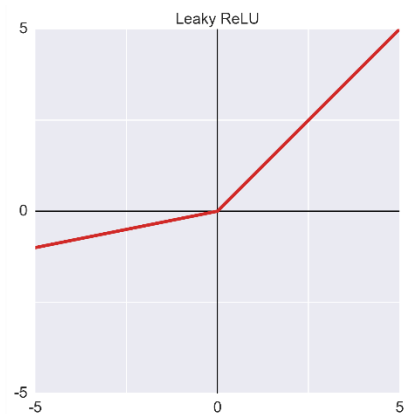
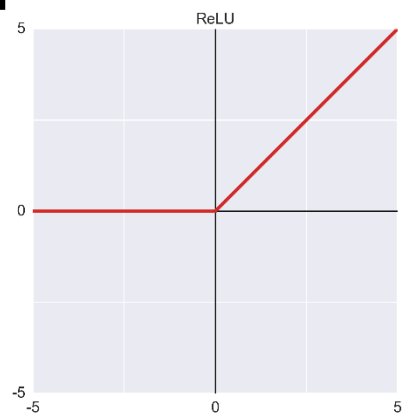


A perceptron unit

Classification



Some popular non-linear activation functions



Backpropagation

Descente de gradient

Classification

$$\{(\mathbf{x}^{(1)}, y^{(1)}), \dots (\mathbf{x}^{(i)}, y^{(i)}), \dots (\mathbf{x}^{(n)}, y^{(n)})\}$$

$$w_j \leftarrow w_j - \eta \frac{\partial \text{Erreur}}{\partial w_j}$$

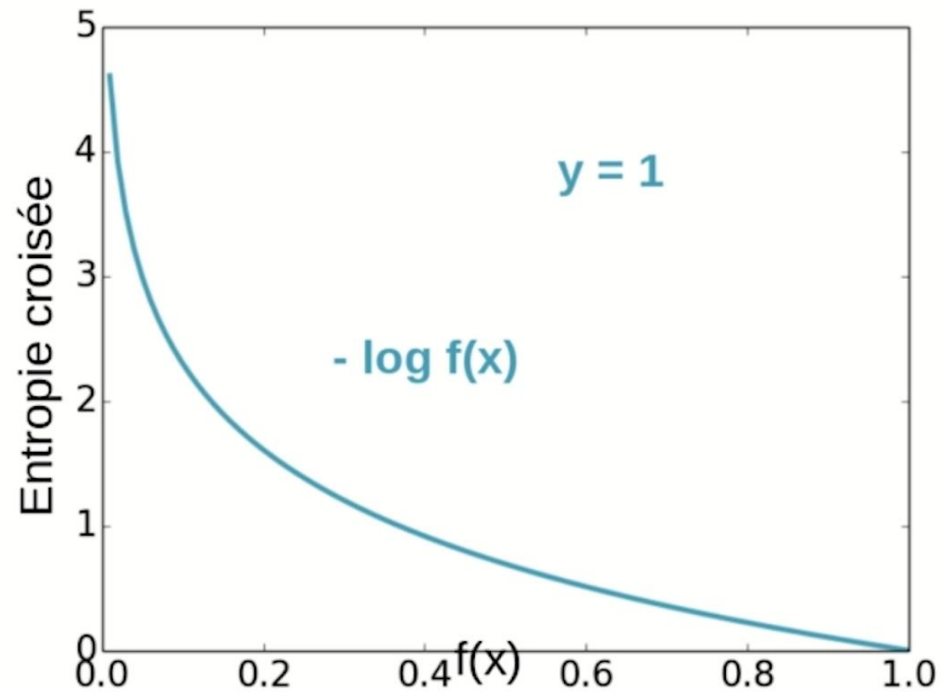
entropie croisée

$$\text{Erreur} = -y^{(i)} \log f(\mathbf{x}^{(i)}) - (1 - y^{(i)}) \log(1 - f(\mathbf{x}^{(i)}))$$

Cross-Entropy

Entropie croisée

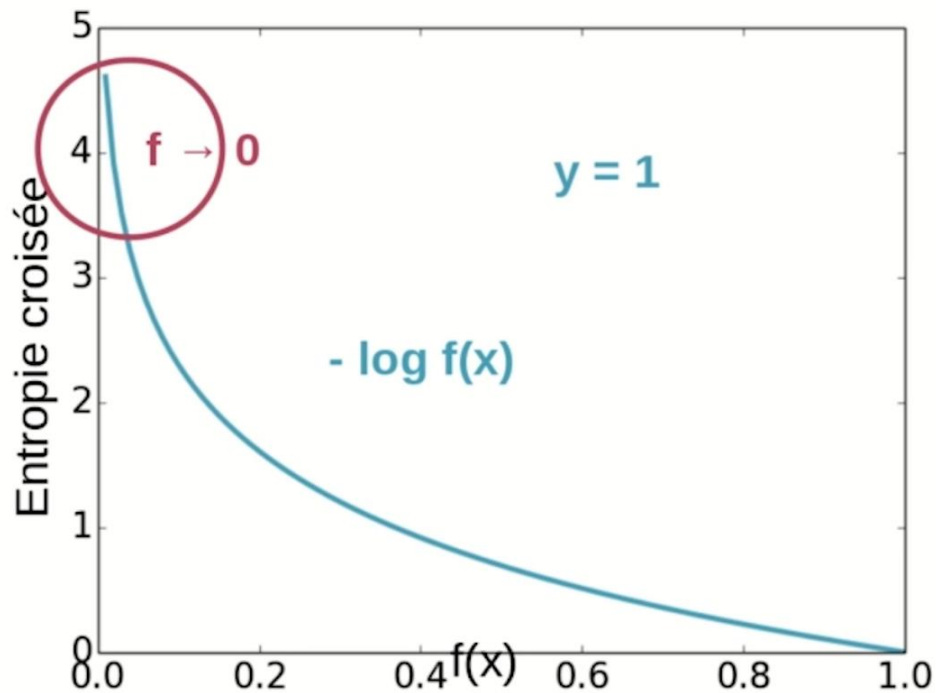
$$\text{Erreur} = -y^{(i)} \log f(\mathbf{x}^{(i)}) - (1 - y^{(i)}) \log(1 - f(\mathbf{x}^{(i)}))$$



Cross-Entropy

Entropie croisée

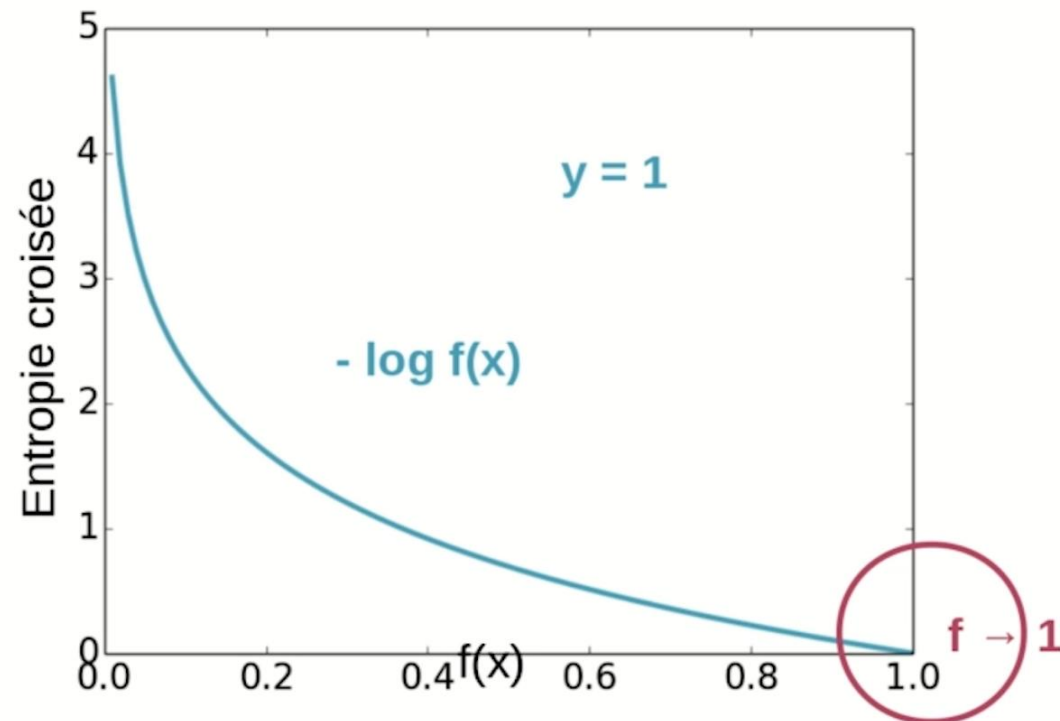
$$\text{Erreur} = -y^{(i)} \log f(\mathbf{x}^{(i)}) - (1 - y^{(i)}) \log(1 - f(\mathbf{x}^{(i)}))$$



Cross-Entropy

Entropie croisée

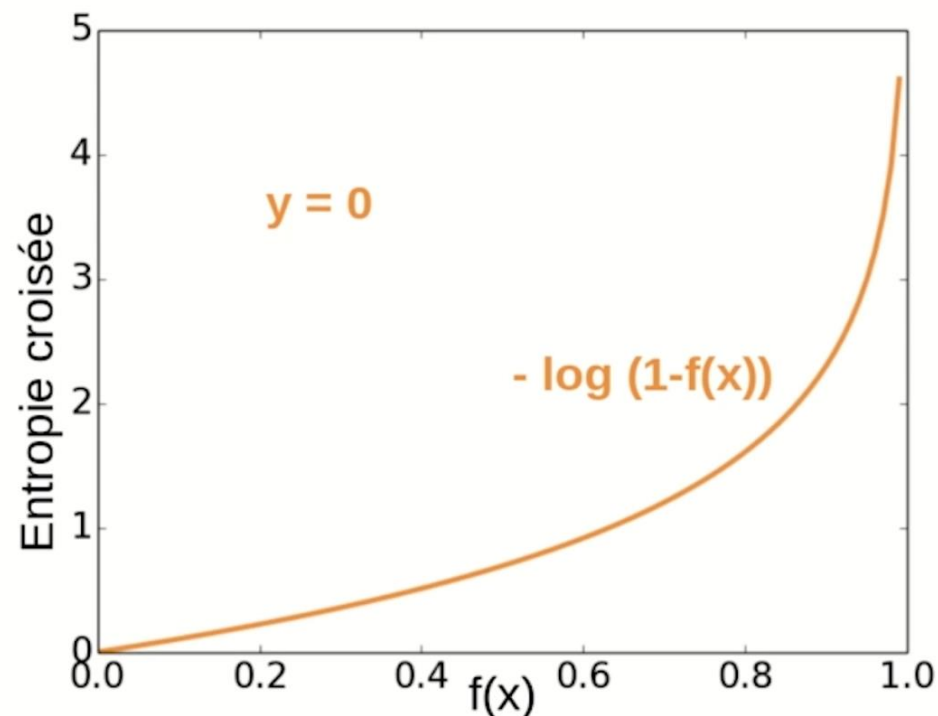
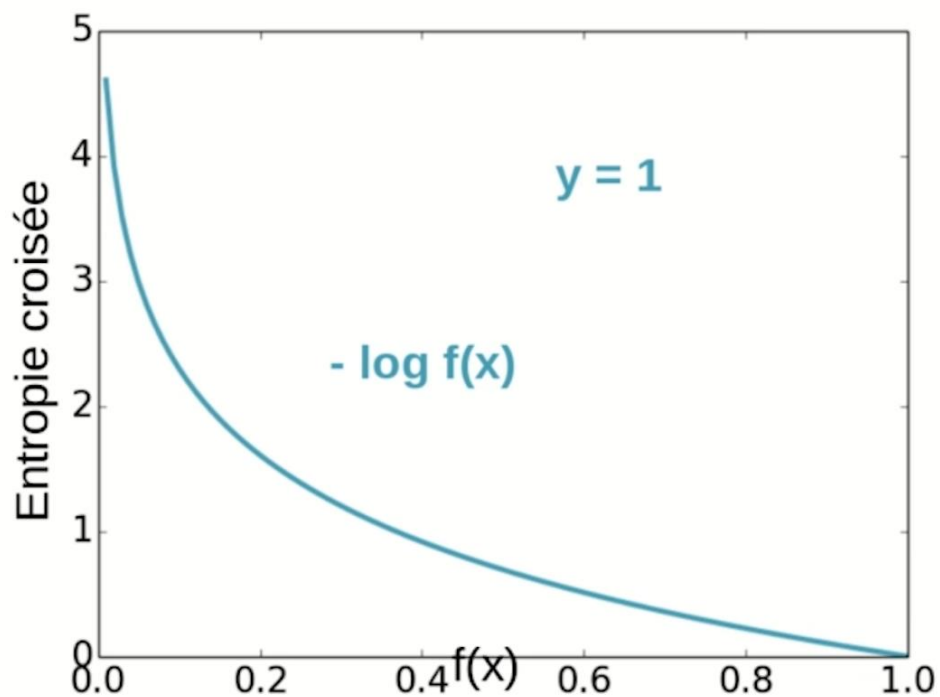
$$\text{Erreur} = -y^{(i)} \log f(\mathbf{x}^{(i)}) - (1 - y^{(i)}) \log(1 - f(\mathbf{x}^{(i)}))$$



Cross-Entropy

Entropie croisée

$$\text{Erreur} = \cancel{-y^{(i)} \log f(\mathbf{x}^{(i)})} - (1 - y^{(i)}) \log(1 - f(\mathbf{x}^{(i)}))$$



Images MNIST



28×28
= 784 pixels

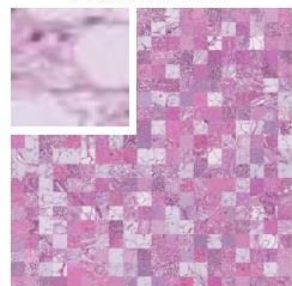


0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	55	87	157	156	187	215	81	0	0	0	0
0	0	0	0	0	0	0	0	0	5	25	54	25	96	140	215	215	251	254	254	254	254	254	163	0	0	0	0
0	0	0	0	0	0	0	0	0	157	254	254	254	0	254	254	254	254	254	254	254	254	241	100	0	0	0	0
0	0	0	0	0	0	0	0	0	156	117	184	214	214	156	117	49	19	19	141	254	241	158	23	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	16	173	254	246	70	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	90	247	254	219	58	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	43	203	254	250	121	15	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	43	145	254	254	229	111	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	4	137	245	254	254	238	40	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	130	254	254	254	254	254	235	106	23	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	55	196	196	114	194	223	254	254	216	23	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	11	75	245	253	55	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	150	254	134	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	14	200	254	134	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	11	246	253	59	0	0	0	0	0	0	0	
0	0	0	17	11	18	0	0	0	0	0	0	0	0	0	67	254	254	170	0	0	0	0	0	0	0	0	
0	0	0	128	254	159	0	0	0	0	0	0	20	140	248	254	254	9	0	0	0	0	0	0	0	0	0	
0	0	0	154	254	229	80	79	79	161	176	218	254	254	254	104	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	29	203	254	254	254	215	254	254	254	250	95	11	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	5	80	155	155	156	111	58	58	36	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

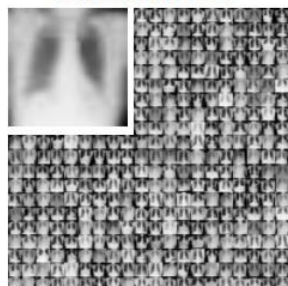
The computer sees this image as a 28×28 matrix of pixel values ranging from 0 to 255.

MedMNIST

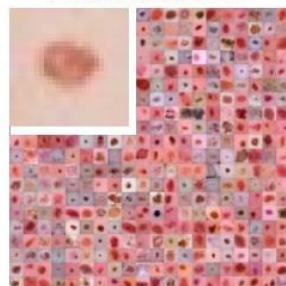
PathMNIST



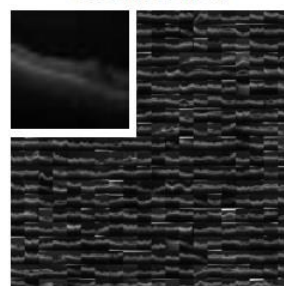
ChestMNIST



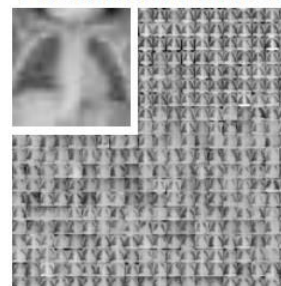
DermaMNIST



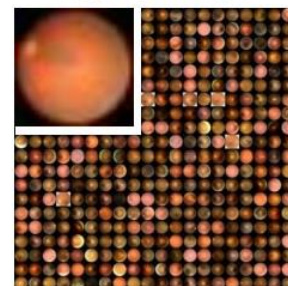
OCTMNIST



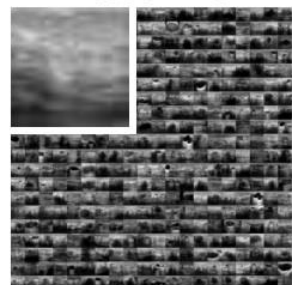
PneumoniaMNIST



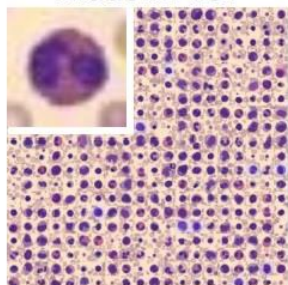
RetinaMNIST



BreastMNIST



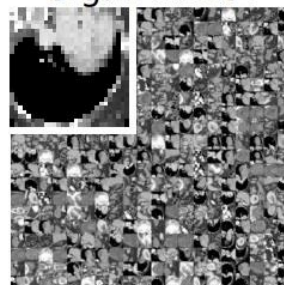
BloodMNIST



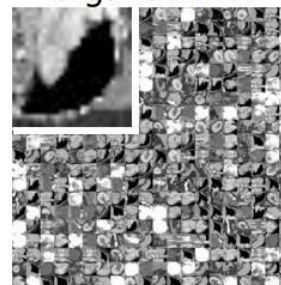
TissueMNIST



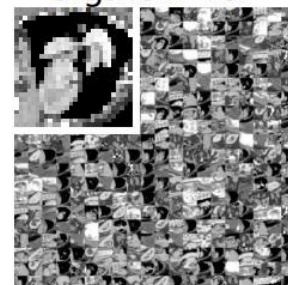
OrganAMNIST



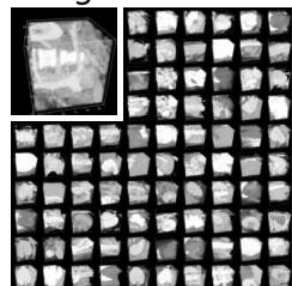
OrganCMNIST



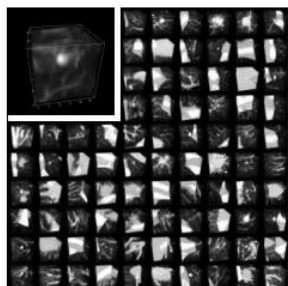
OrganSMNIST



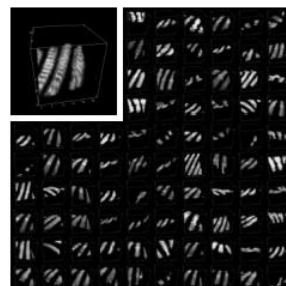
OrganMNIST3D



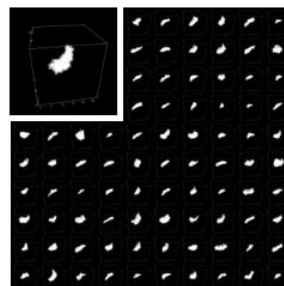
NoduleMNIST3D



FractureMNIST3D



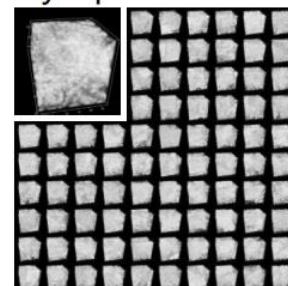
AdrenalMNIST3D



VesselMNIST3D

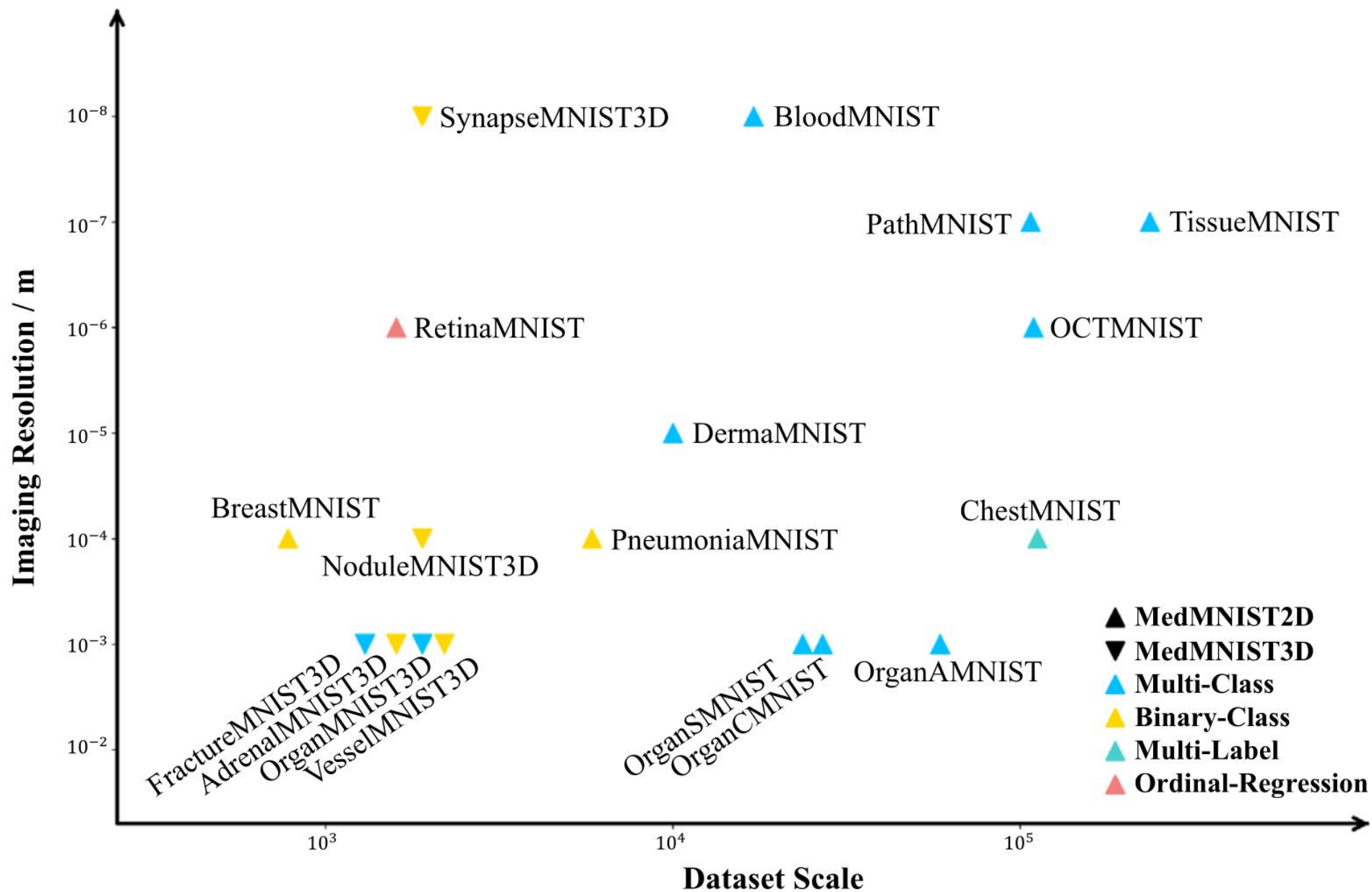


SynapseMNIST3D



<https://medmnist.com>

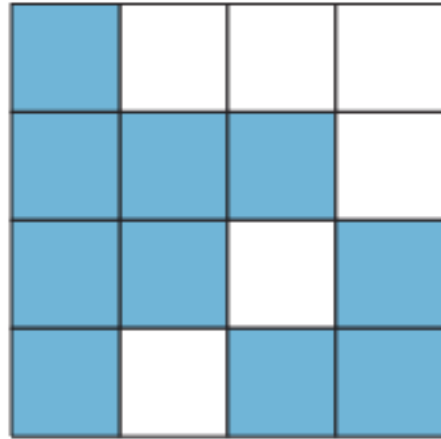
MedMNIST



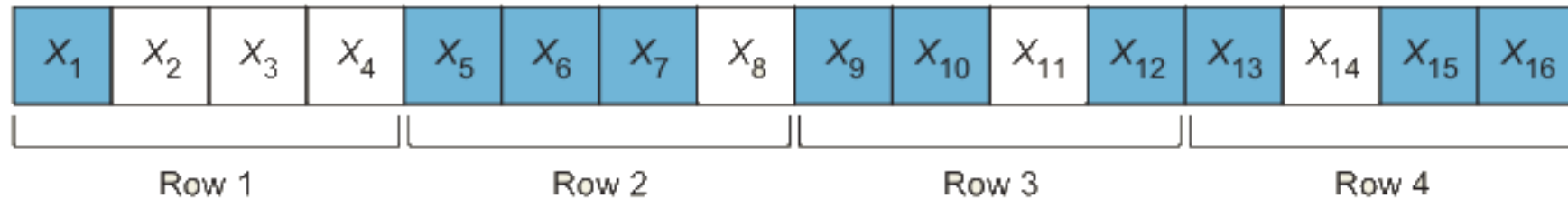
MedMNIST

Name	Source	Data Modality	Task (# Classes/ Labels)	# Samples	# Training/Validation/Test
<i>MedMNIST2D</i>					
PathMNIST	Kather <i>et al.</i> ^{16,17}	Colon Pathology	MC (9)	107,180	89,996/10,004/7,180
ChestMNIST	Wang <i>et al.</i> ¹⁸	Chest X-Ray	ML (14) BC (2)	112,120	78,468/11,219/22,433
DermaMNIST	Tschandl <i>et al.</i> ^{19,20} , Codella <i>et al.</i> ²¹	Dermatoscope	MC (7)	10,015	7,007/1,003/2,005
OCTMNIST	Kermany <i>et al.</i> ^{22,23}	Retinal OCT	MC (4)	109,309	97,477/10,832/1,000
PneumoniaMNIST	Kermany <i>et al.</i> ^{22,23}	Chest X-Ray	BC (2)	5,856	4,708/524/624
RetinaMNIST	DeepDRiD Team ²⁴	Fundus Camera	OR (5)	1,600	1,080/120/400
BreastMNIST	Al-Dhabyani <i>et al.</i> ²⁵	Breast Ultrasound	BC (2)	780	546/78/156
BloodMNIST	Acevedo <i>et al.</i> ^{26,27}	Blood Cell Microscope	MC (8)	17,092	11,959/1,712/3,421
TissueMNIST	Ljosa <i>et al.</i> ²⁹	Kidney Cortex Microscope	MC (8)	236,386	165,466/23,640/47,280
OrganAMNIST	Bilic <i>et al.</i> ³⁰ , Xu <i>et al.</i> ³¹	Abdominal CT	MC (11)	58,850	34,581/6,491/17,778
OrganCMNIST	Bilic <i>et al.</i> ³⁰ , Xu <i>et al.</i> ³¹	Abdominal CT	MC (11)	23,660	13,000/2,392/8,268
OrganSMNIST	Bilic <i>et al.</i> ³⁰ , Xu <i>et al.</i> ³¹	Abdominal CT	MC (11)	25,221	13,940/2,452/8,829
<i>MedMNIST3D</i>					
OrganMNIST3D	Bilic <i>et al.</i> ³⁰ , Xu <i>et al.</i> ³¹	Abdominal CT	MC (11)	1,743	972/161/610
NoduleMNIST3D	Armato <i>et al.</i> ³²	Chest CT	BC (2)	1,633	1,158/165/310
AdrenalMNIST3D	New	Shape from Abdominal CT	BC (2)	1,584	1,188/98/298
FractureMNIST3D	Jin <i>et al.</i> ³³	Chest CT	MC (3)	1,370	1,027/103/240
VesselMNIST3D	Yang <i>et al.</i> ³⁴	Shape from Brain MRA	BC (2)	1,909	1,335/192/382
SynapseMNIST3D	New	Electron Microscope	BC (2)	1,759	1,230/177/352

Image classification using MLP



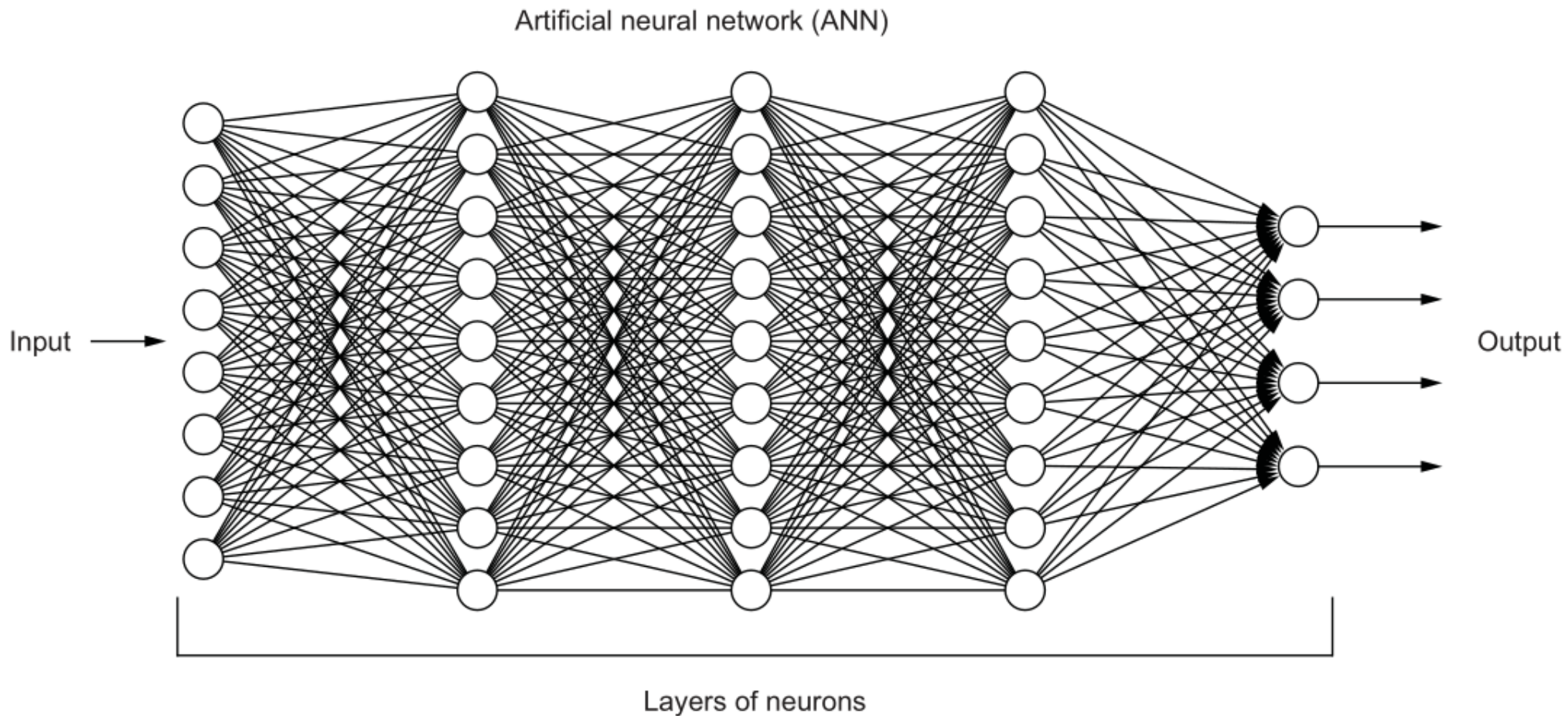
The input (x) is a flattened vector with the dimensions (1, 16):



So, if we have pixel values of 0 for black and 255 for white, the input vector will be as follows:

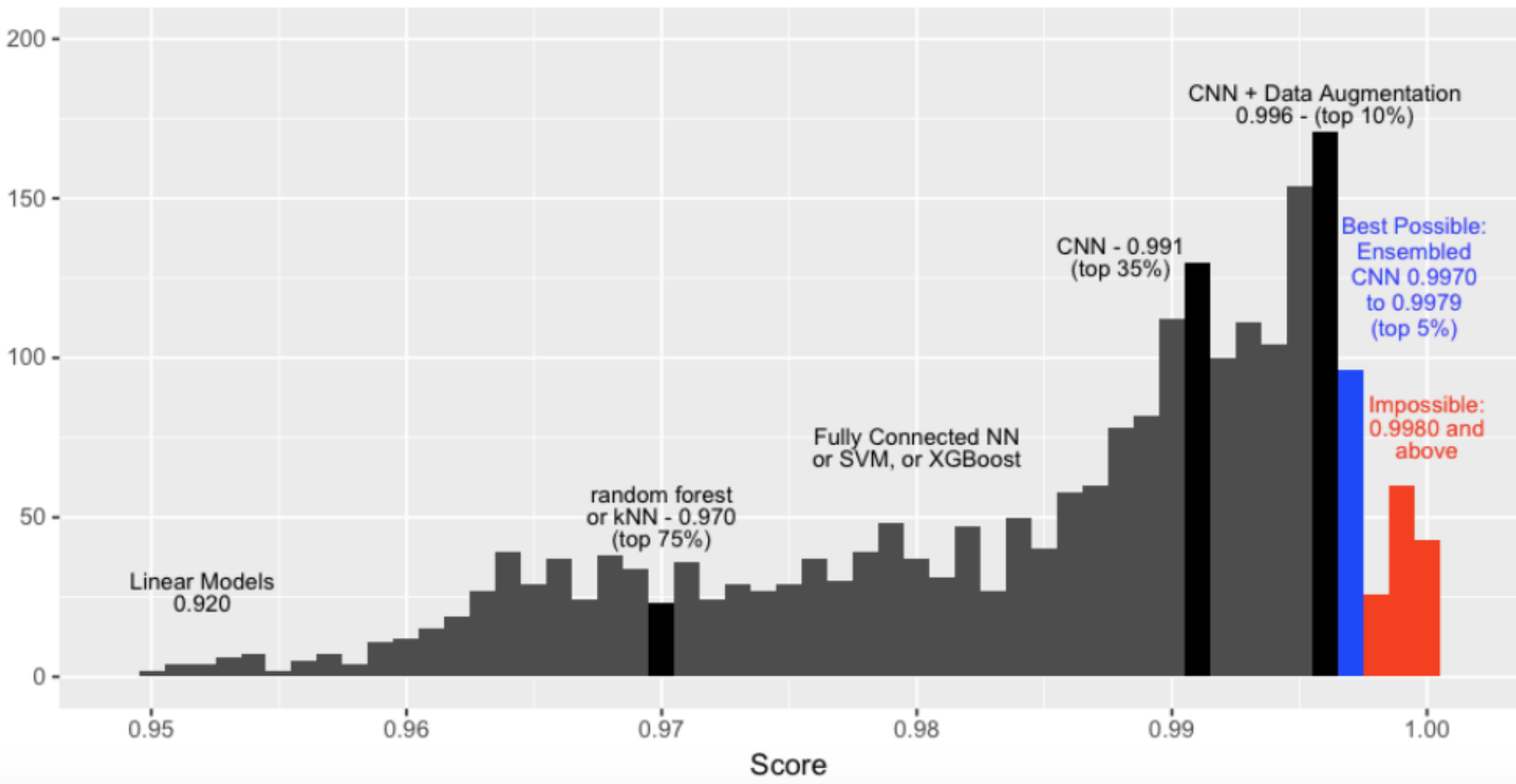
Input = [0, 255, 255, 255, 0, 0, 0, 255, 0, 0, 255, 0, 0, 255, 0, 0]

Image classification using MLP



MLPs vs. CNNs

Histogram of Kaggle MNIST public leaderboard scores, July 15 2018

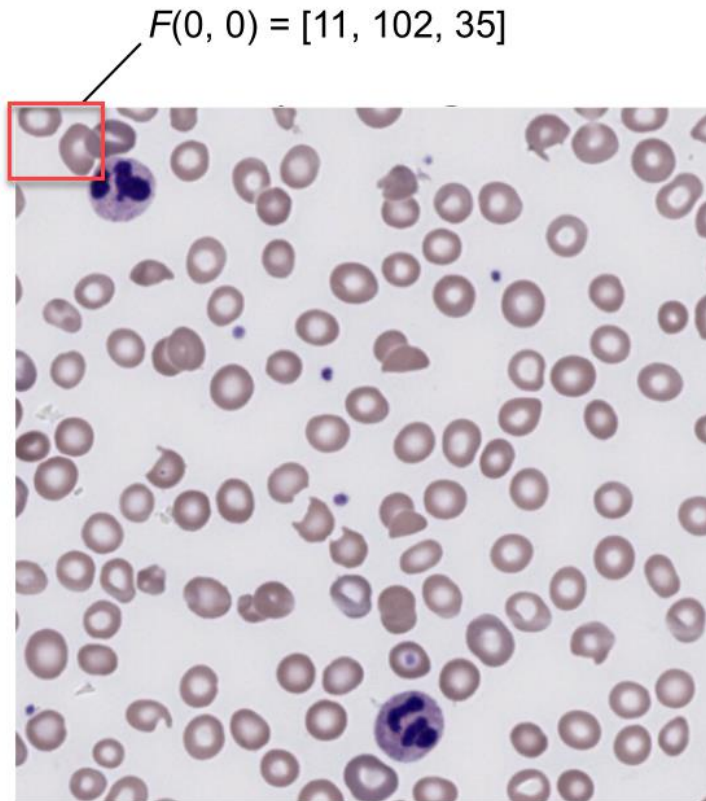


Convolutional neural networks CNN

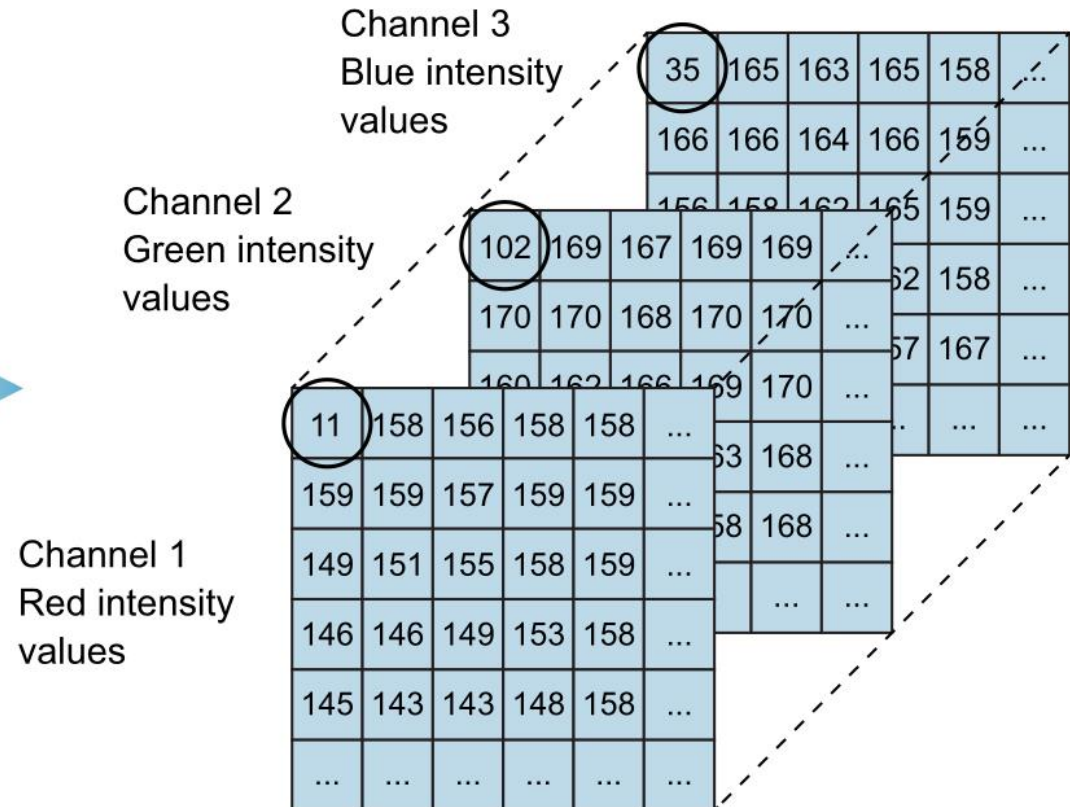
- ❑ Introduction
- ❑ Convolutional neural networks CNN
- ❑ Model Architectures
- ❑ Object Detection
- ❑ Segmentation
- ❑ Instance Segmentation

Convolution over color images (3D images)

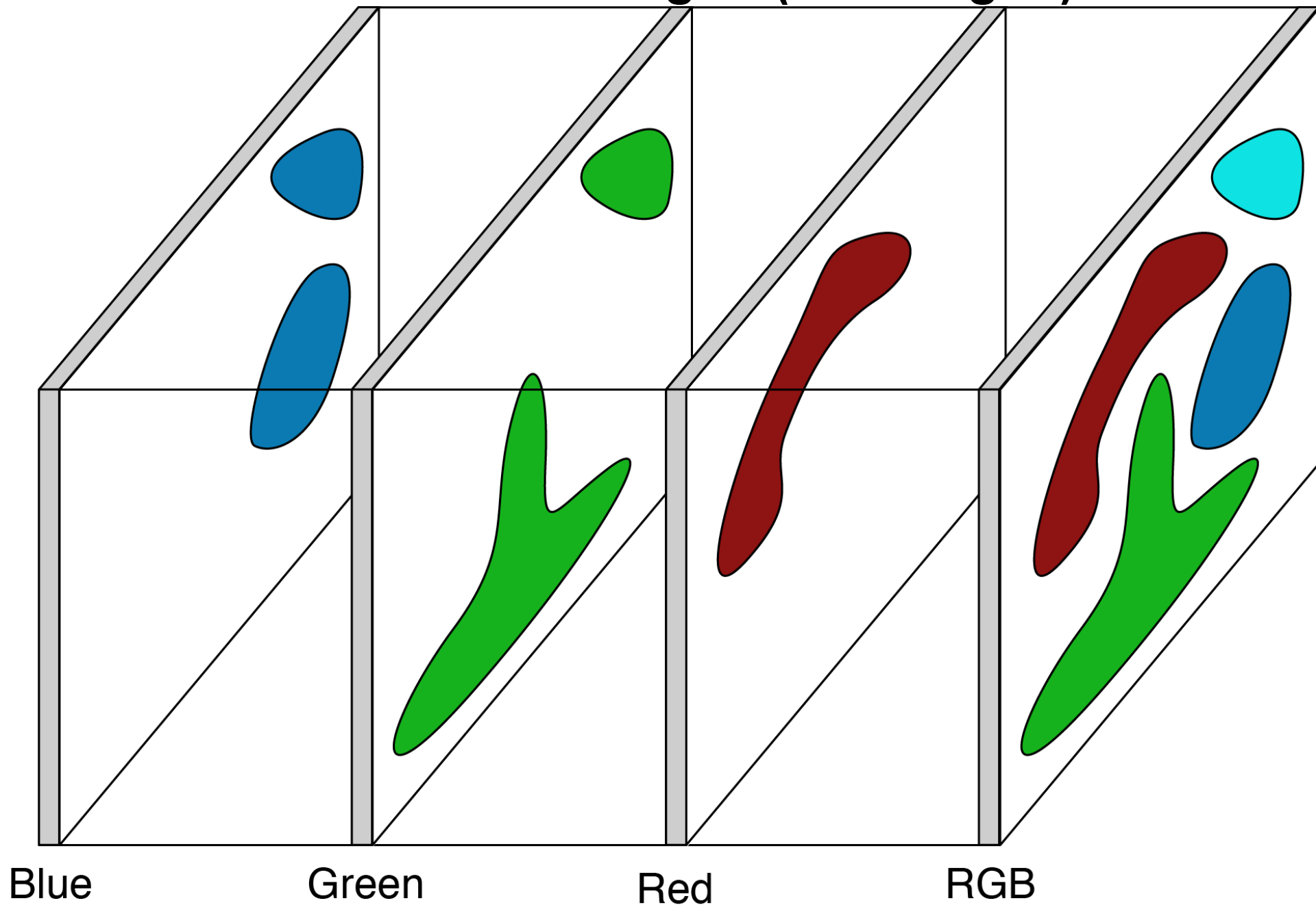
Color image



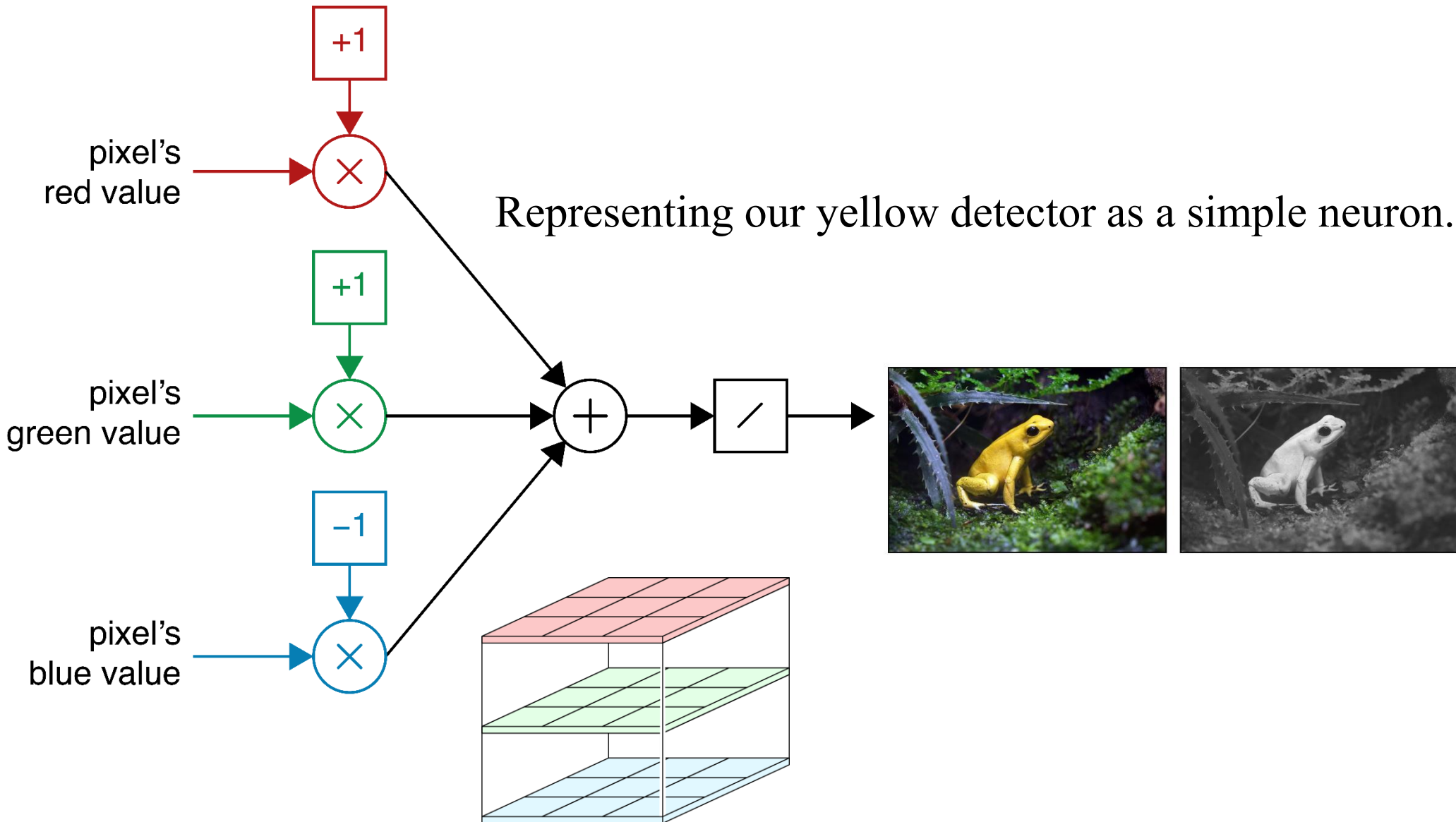
RGB channels



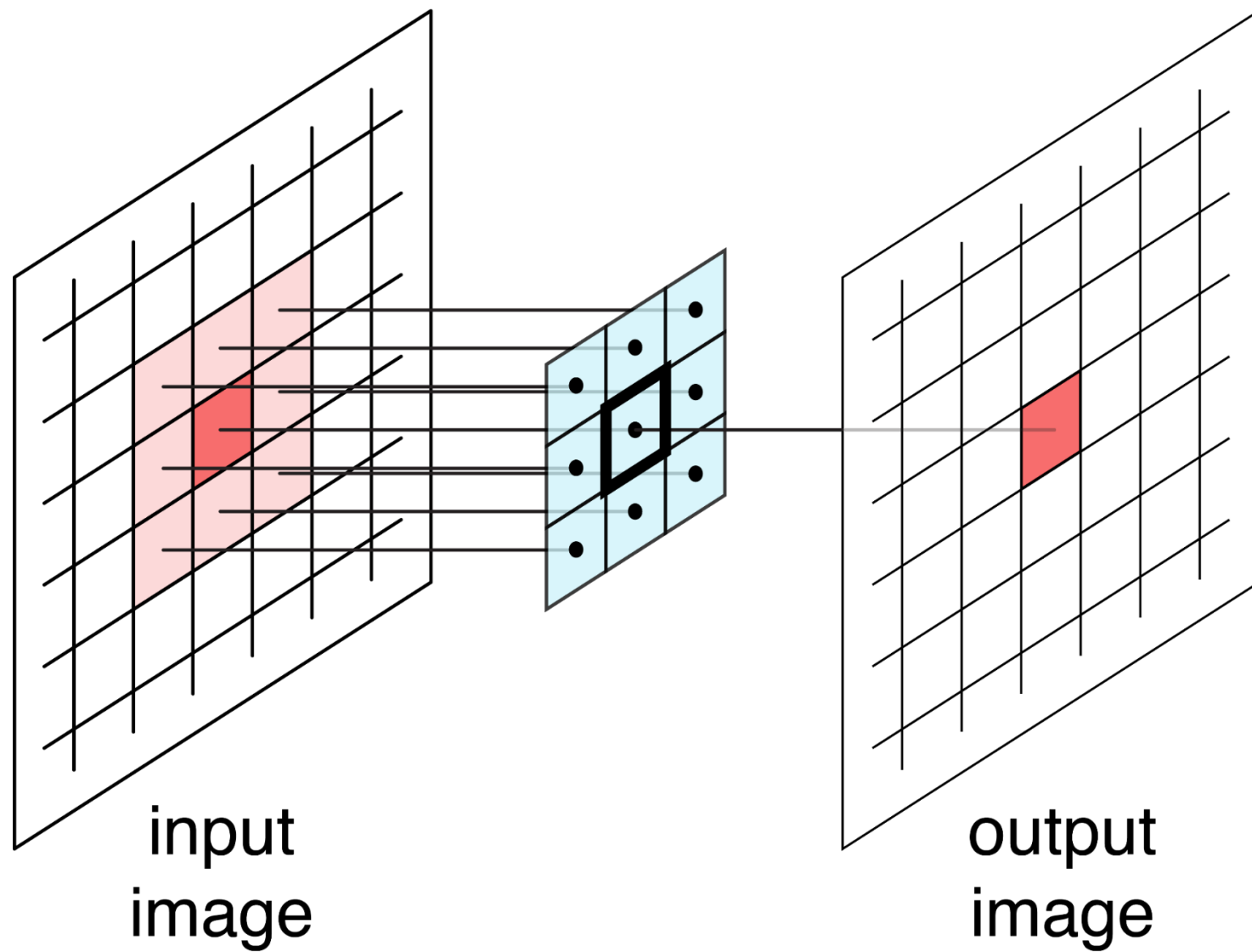
Convolution over color images (3D images)



Convolution over color images (3D images)

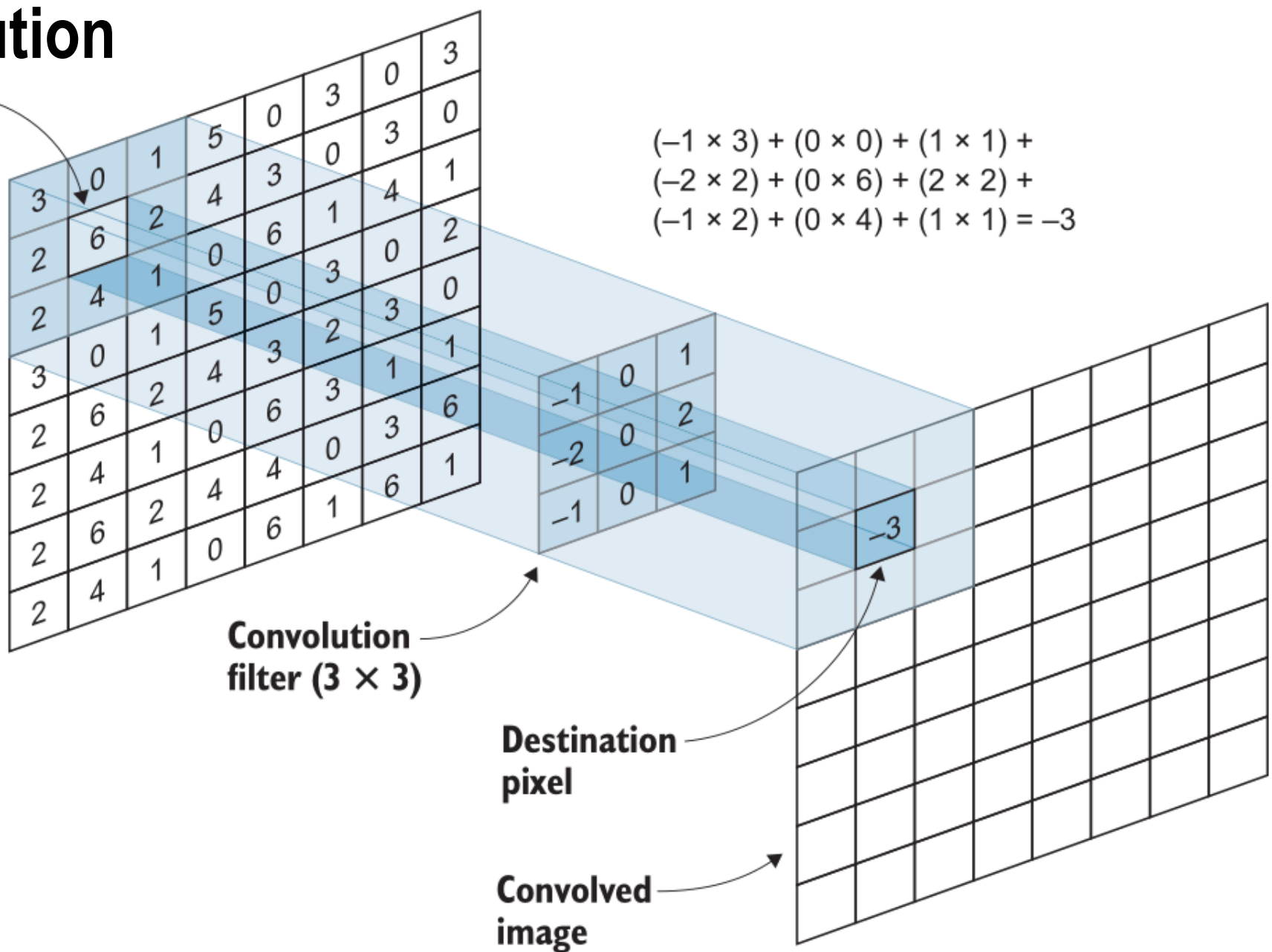


Convolution



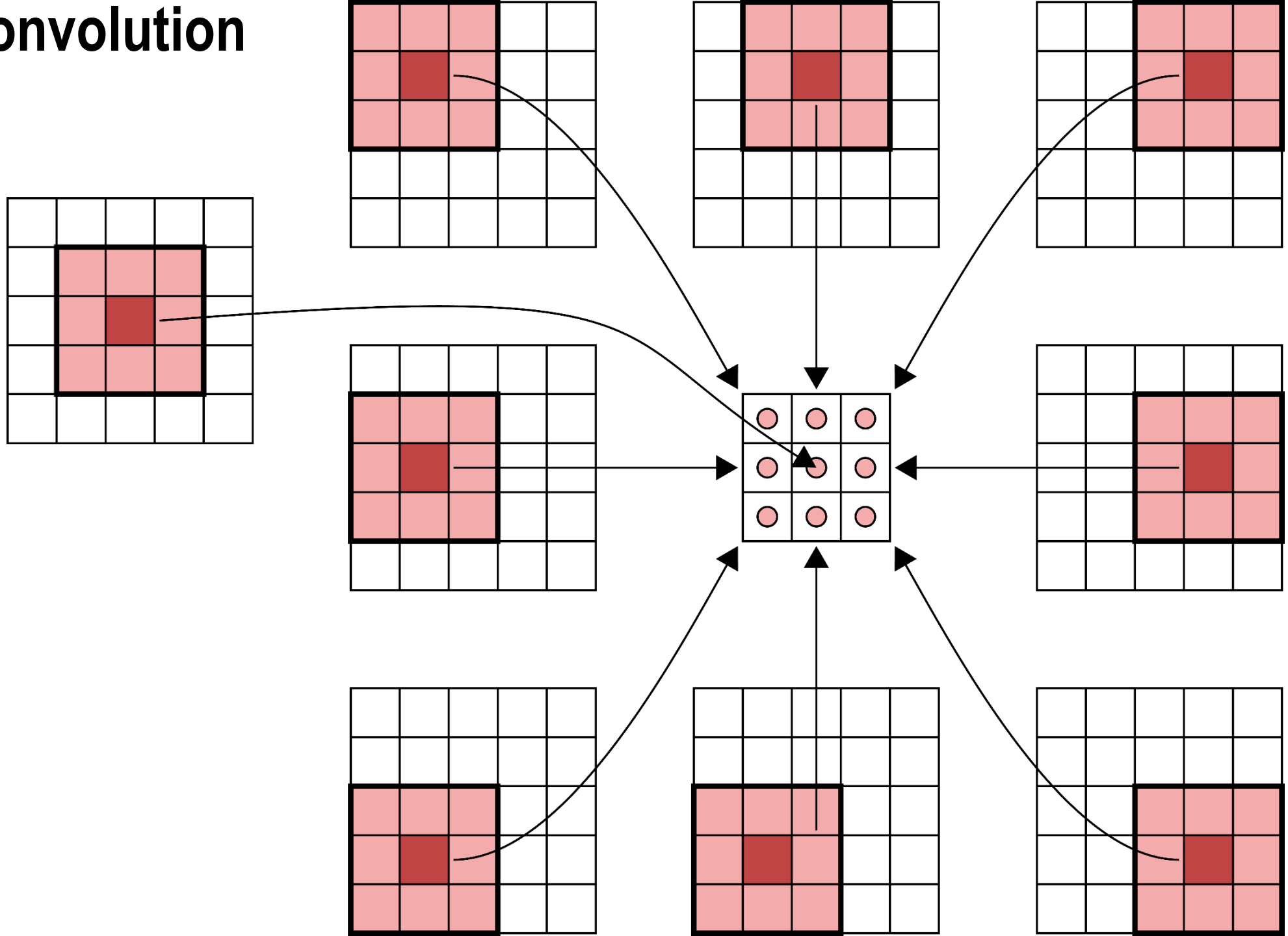
Convolution

Receptive
field

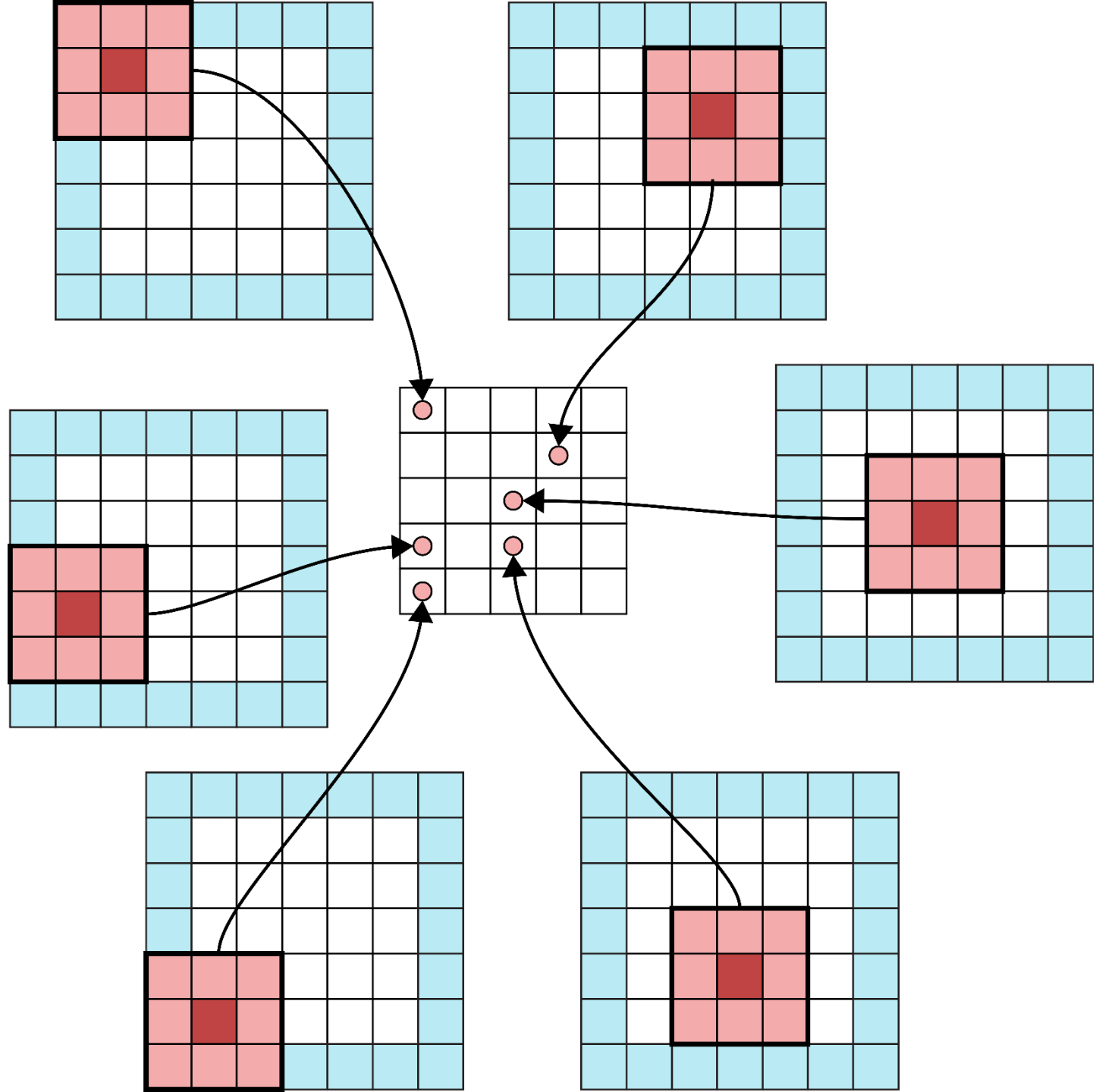


A 3×3 convolutional filter is sliding over the input image.

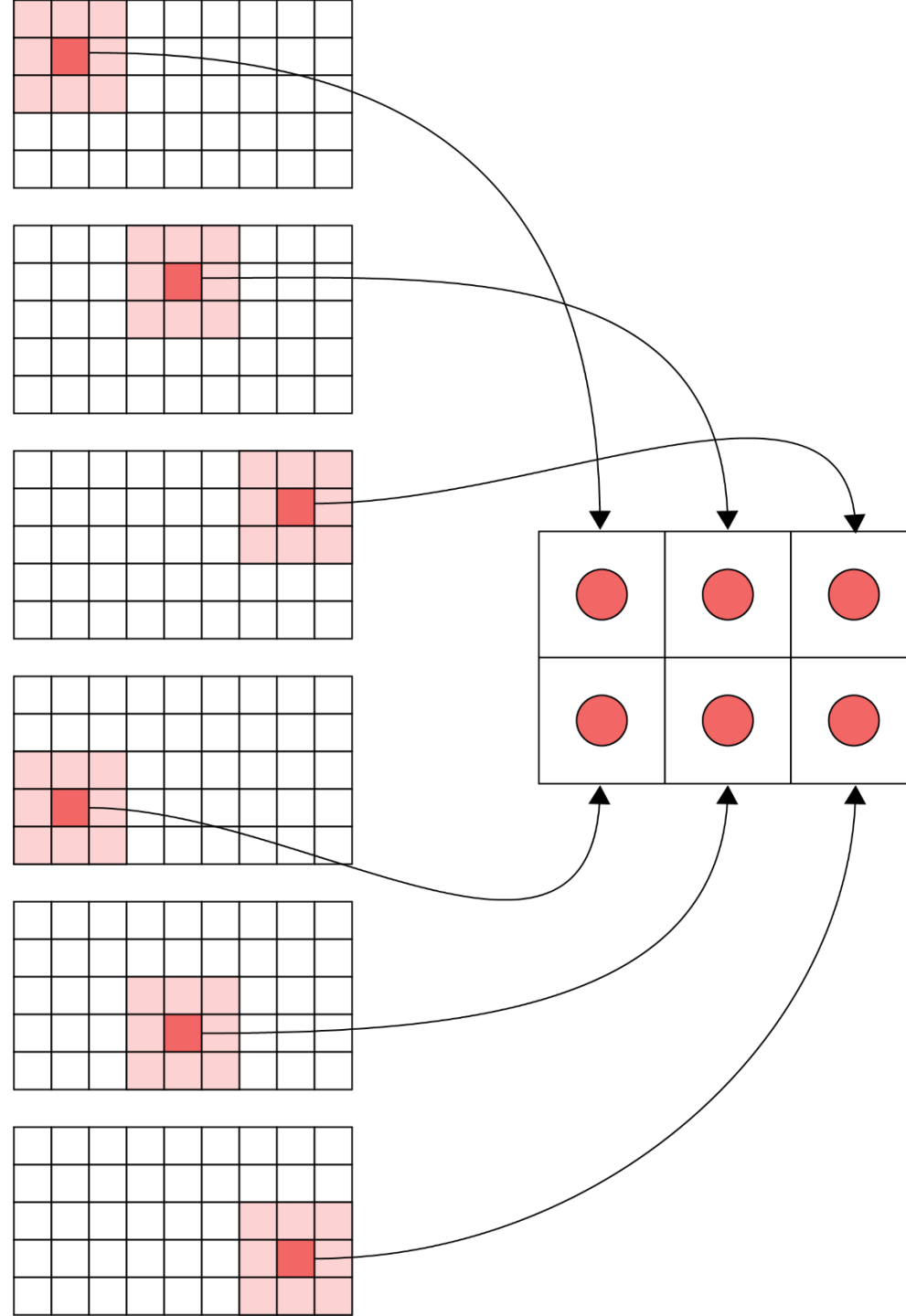
Convolution



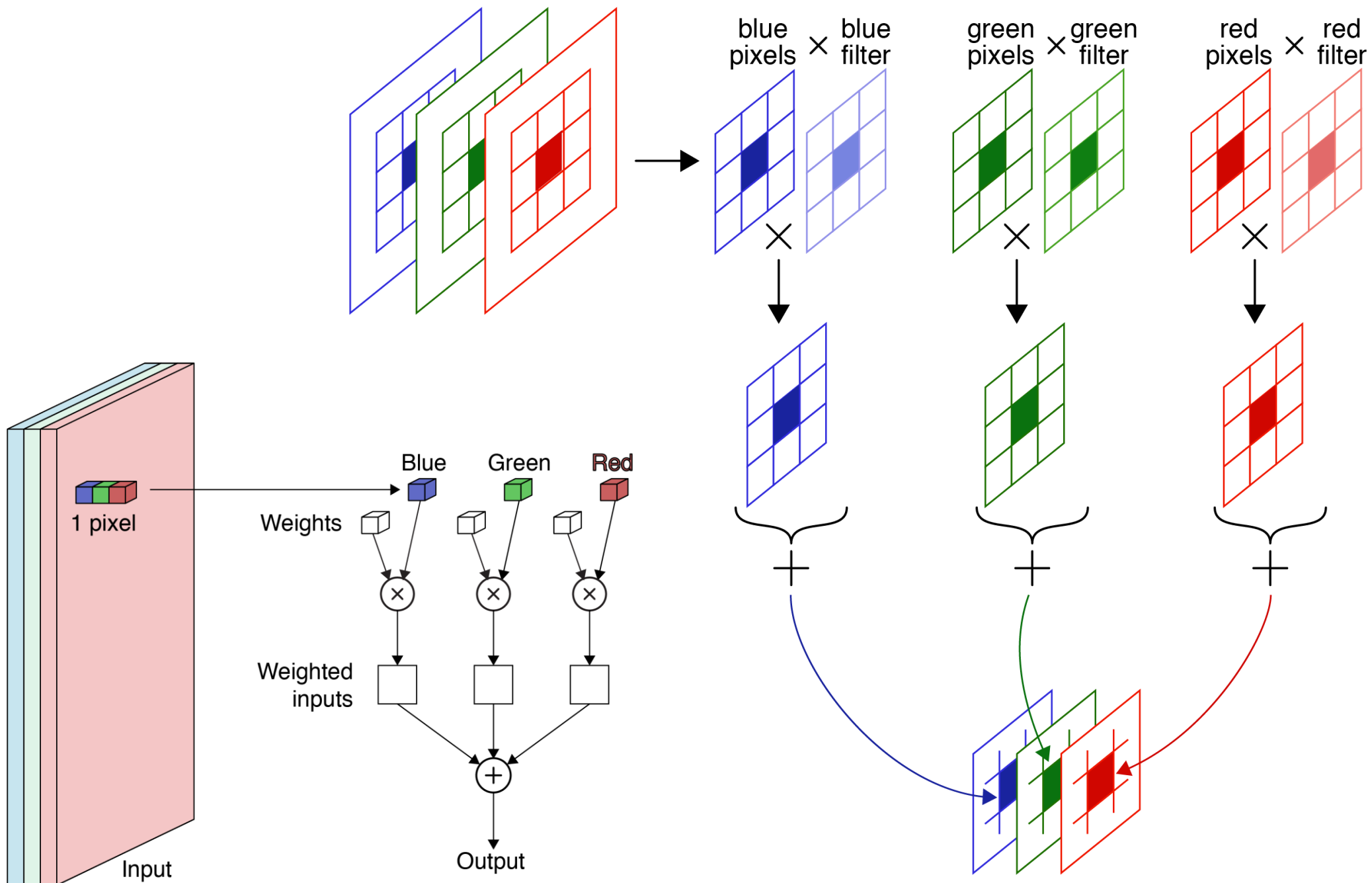
Convolution



Convolution



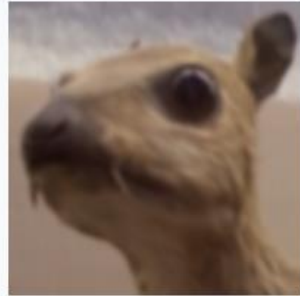
Convolution and filters



Filters

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

Filters

Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 × 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
Gaussian blur 5 × 5 (approximation)	$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	

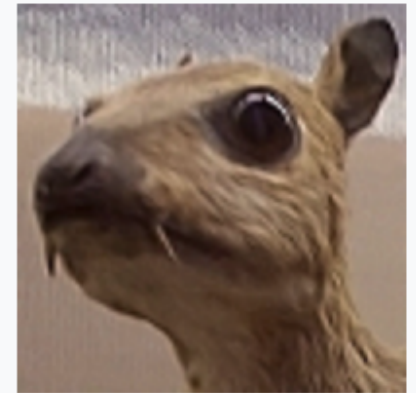
Filters

Unsharp masking 5×5

Based on Gaussian blur
with amount as 1 and
threshold as 0

(with no image mask)

$$\frac{-1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & -476 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$



[https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))

Max Pooling or Average Pooling

Max Pooling

29	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

100	184
12	45

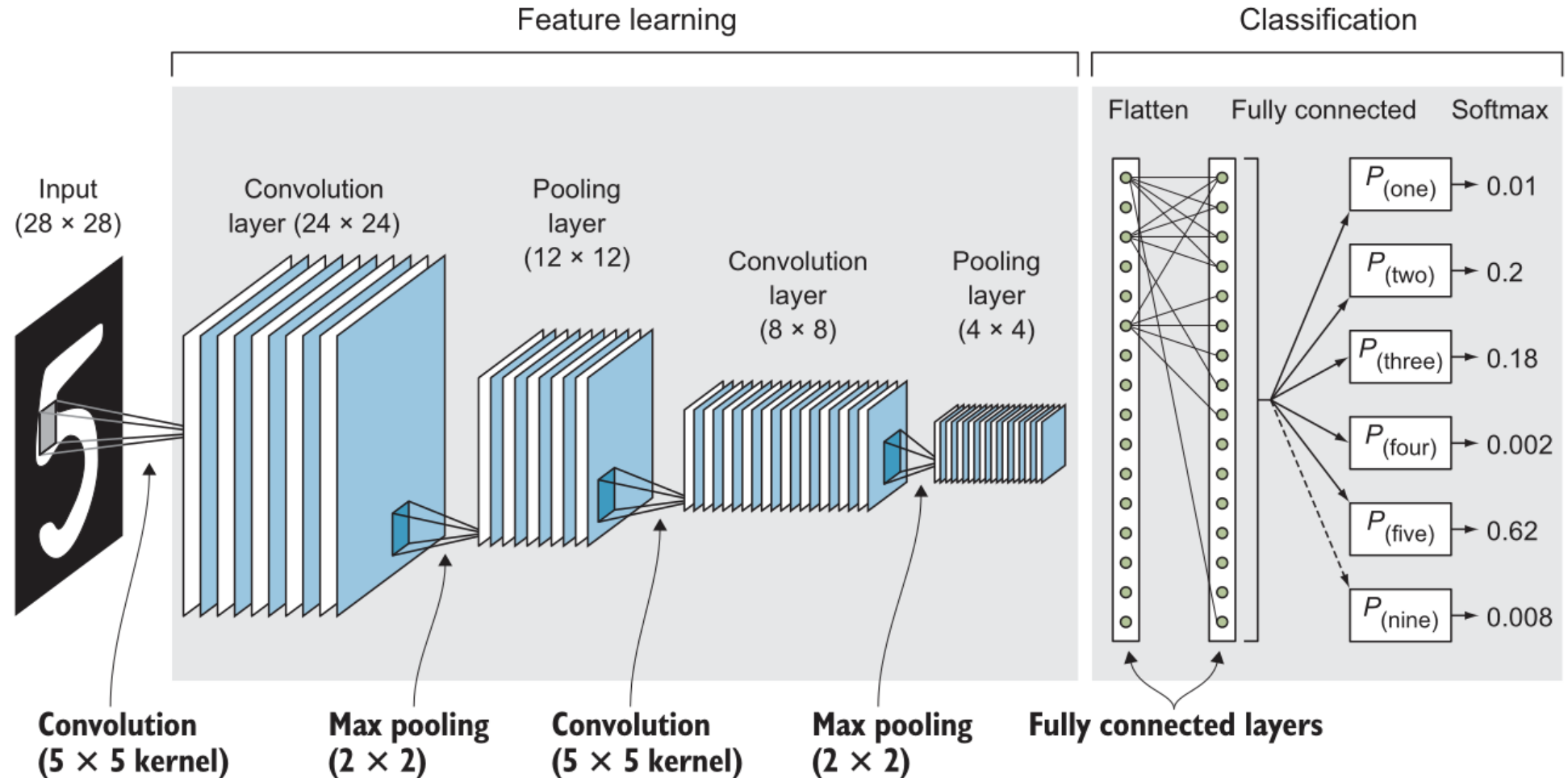
Average Pooling

31	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

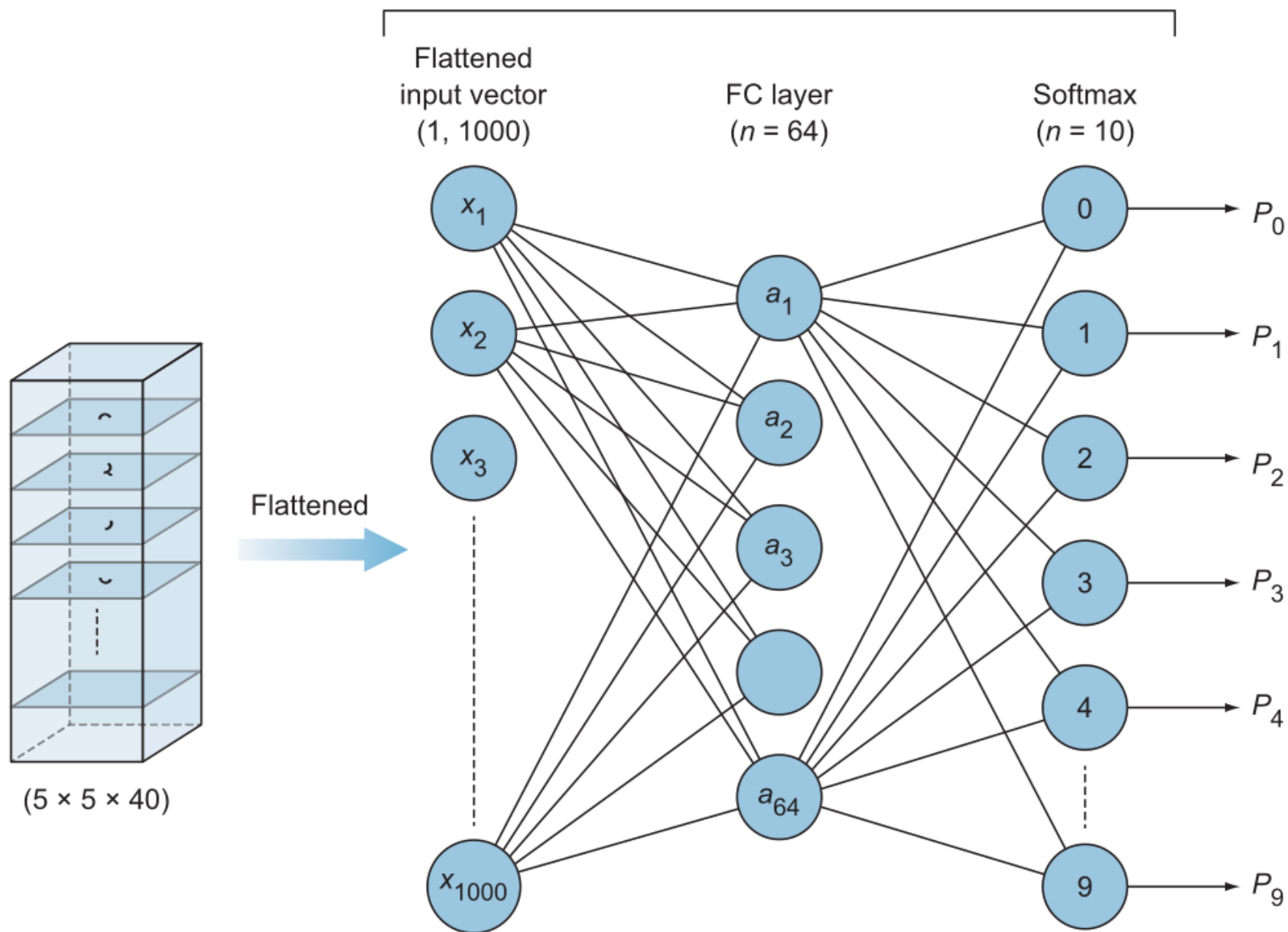
36	80
12	15

Basic components of a CNN



The basic components of convolutional networks are convolutional layers and pooling layers to perform feature extraction, and fully connected layers for classification.

MLP

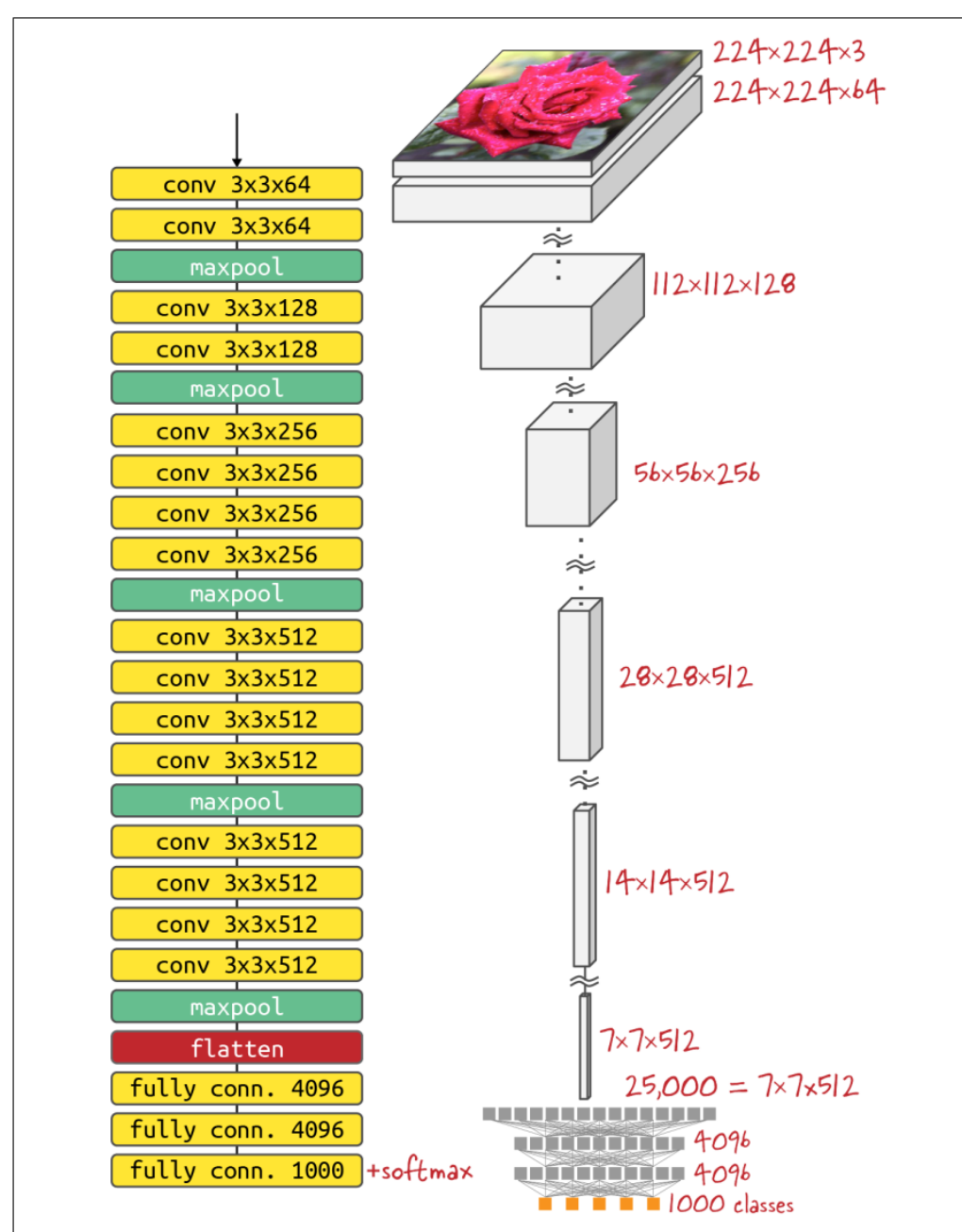


Fully connected layers for an MLP

Model Architectures

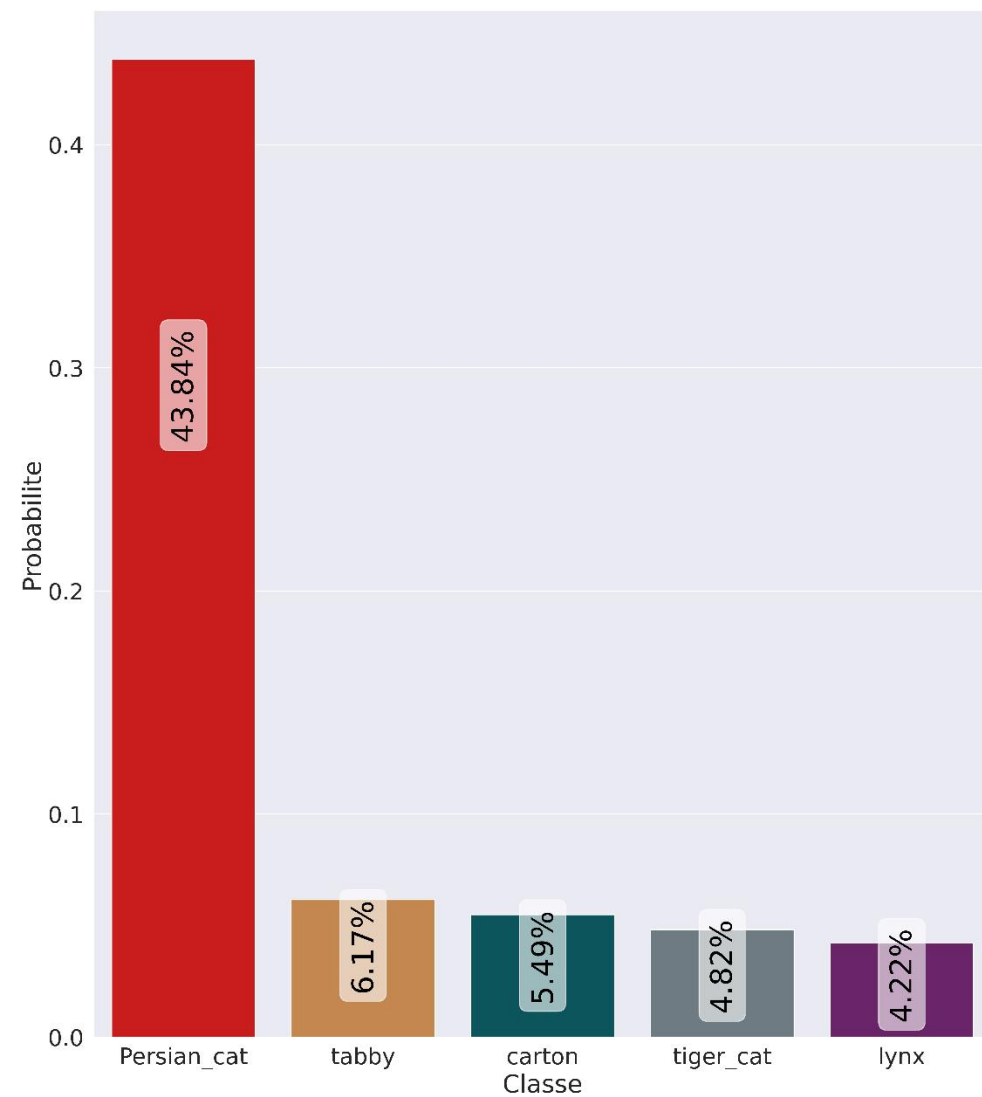
- ❑ Introduction
- ❑ Convolutional neural networks CNN
- ❑ Model Architectures
- ❑ Object Detection
- ❑ Segmentation
- ❑ Instance Segmentation
- ❑ Autoencoders

VGG19



The VGG19 architecture with 19 learnable layers (left). The data shapes are shown on the right (not all represented). Notice that all convolutional layers use 3x3 filters.

Filtres VGG19



block1_conv1

block1_conv1 (Conv2D)

(None, 224, 224, 64)

1,792

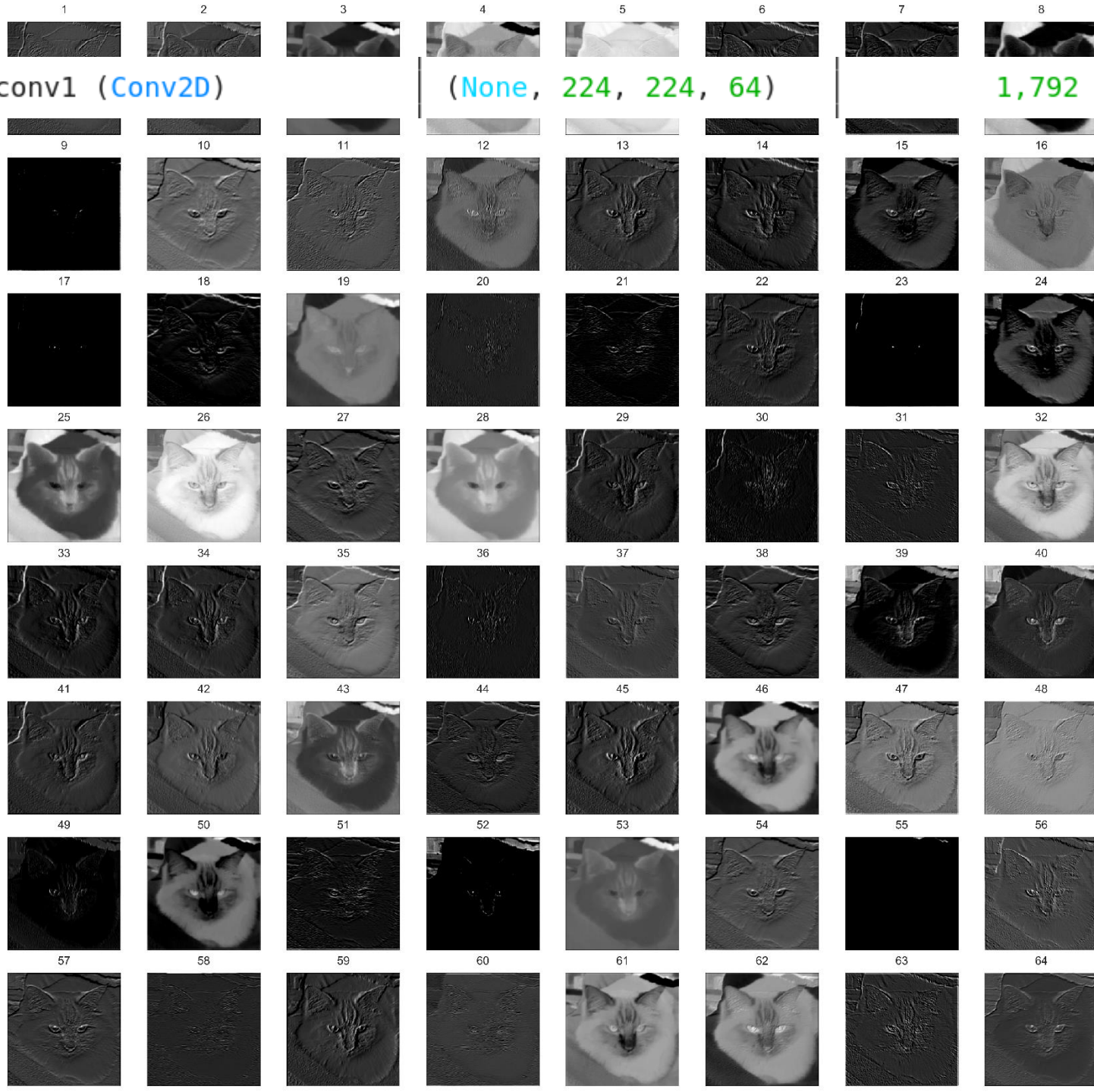
B1
conv 3x3x64
conv 3x3x64

B2
maxpool
conv 3x3x128
conv 3x3x128
maxpool

B3
conv 3x3x256
conv 3x3x256
conv 3x3x256
conv 3x3x256
maxpool

B4
conv 3x3x512
conv 3x3x512
conv 3x3x512
conv 3x3x512
maxpool

B5
conv 3x3x512
conv 3x3x512
conv 3x3x512
conv 3x3x512
maxpool



block2_conv1

block2_conv1 (Conv2D)

(None, 112, 112, 128)

73,856

B1

conv 3x3x64

conv 3x3x64

maxpool

B2

conv 3x3x128

conv 3x3x128

maxpool

B3

conv 3x3x256

conv 3x3x256

conv 3x3x256

conv 3x3x256

maxpool

B4

conv 3x3x512

conv 3x3x512

conv 3x3x512

maxpool

B5

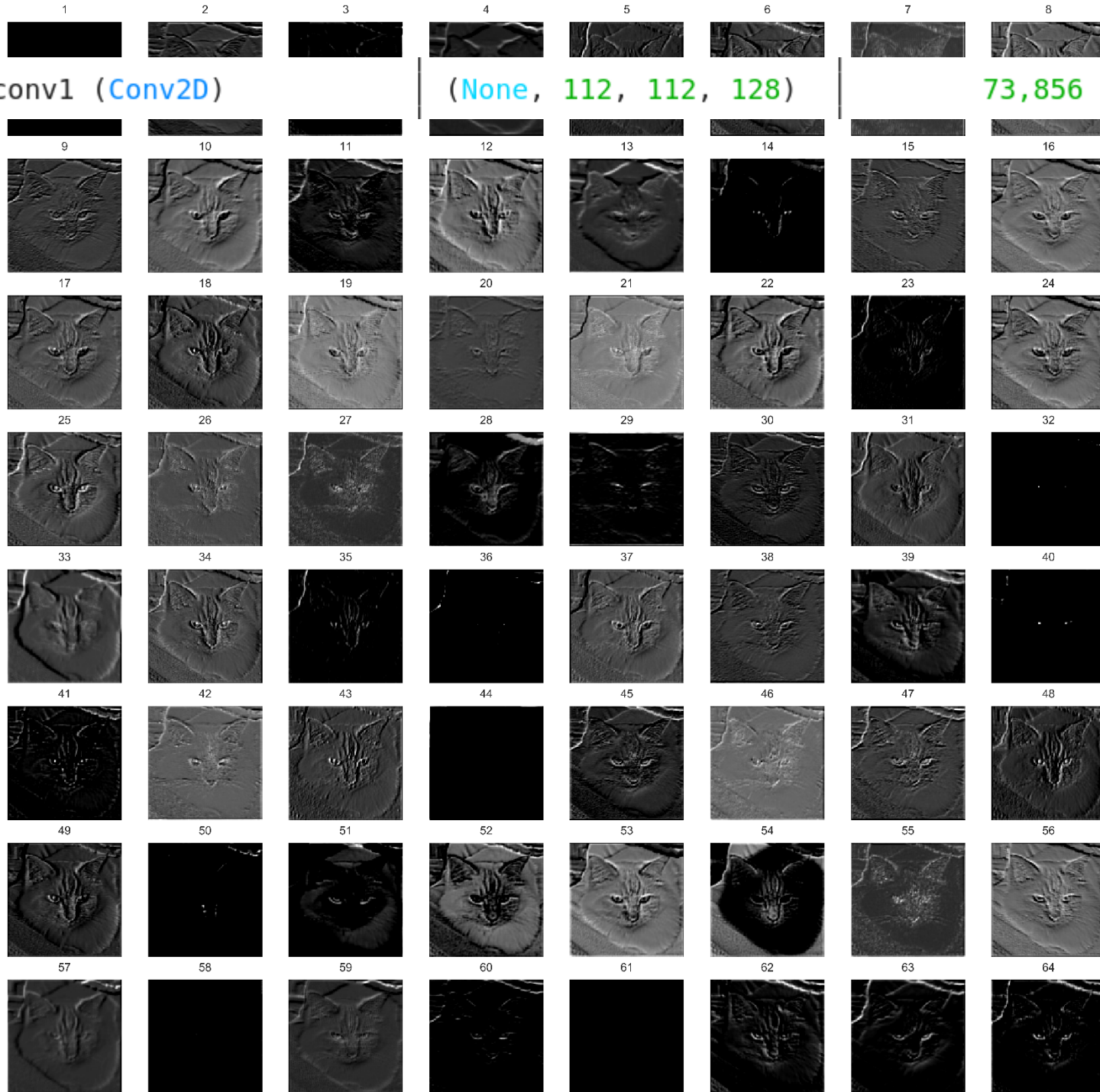
conv 3x3x512

conv 3x3x512

conv 3x3x512

conv 3x3x512

maxpool



block3_conv1

block3_conv1 (Conv2D)

(None, 56, 56, 256)

295,168

B1
conv 3x3x64
conv 3x3x64

maxpool

B2
conv 3x3x128
conv 3x3x128

maxpool

B3
conv 3x3x256
conv 3x3x256

conv 3x3x256

conv 3x3x256

maxpool

conv 3x3x512

B4
14x14x512

conv 3x3x512

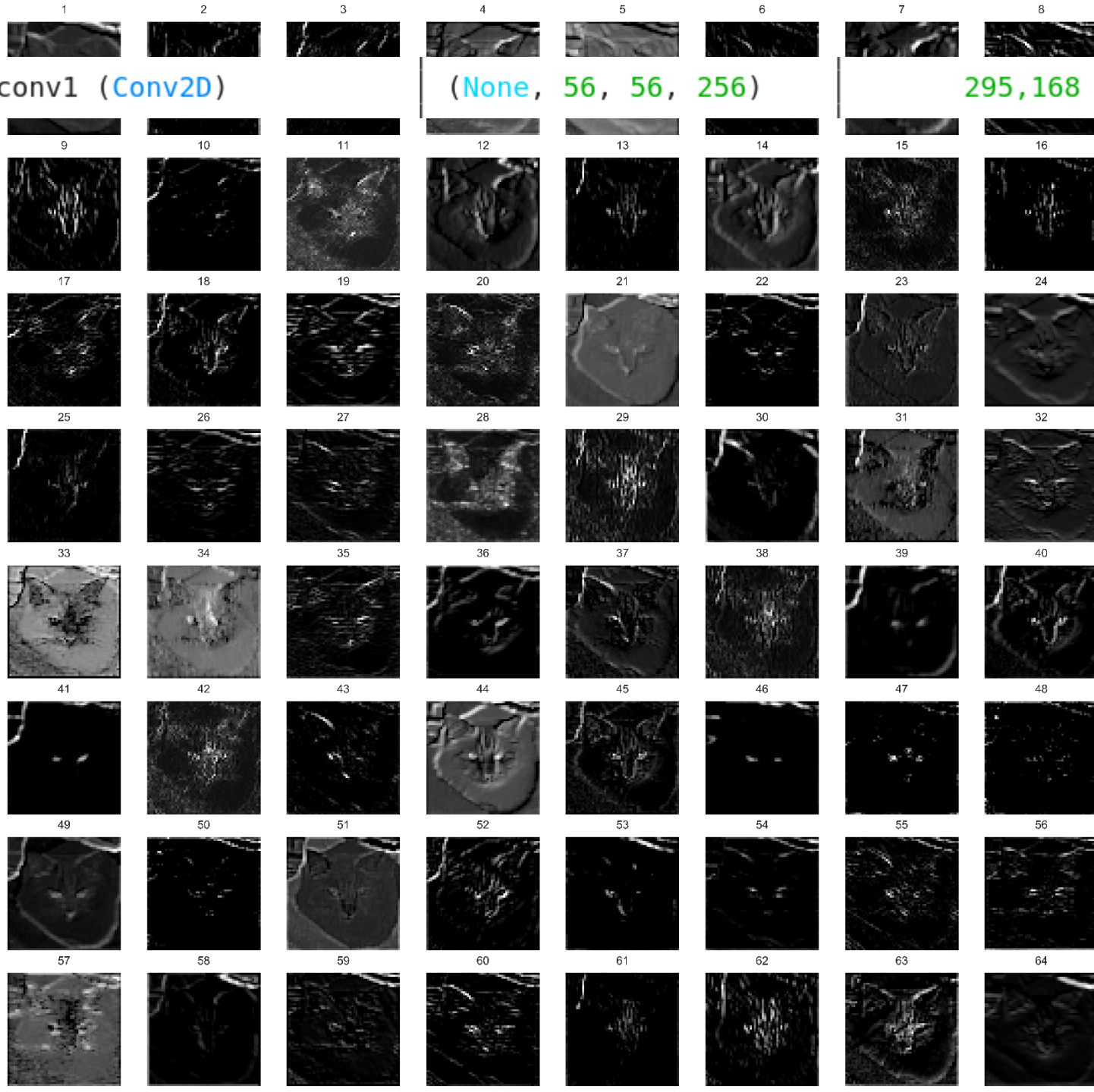
maxpool

conv 3x3x512

B5
conv 3x3x512
conv 3x3x512

conv 3x3x512

maxpool



block4_conv1

block4_conv1 (Conv2D)

(None, 28, 28, 512)

1,180,160

B1
conv 3x3x64
conv 3x3x64

maxpool

B2
conv 3x3x128
conv 3x3x128

maxpool

B3
conv 3x3x256
conv 3x3x256
conv 3x3x256

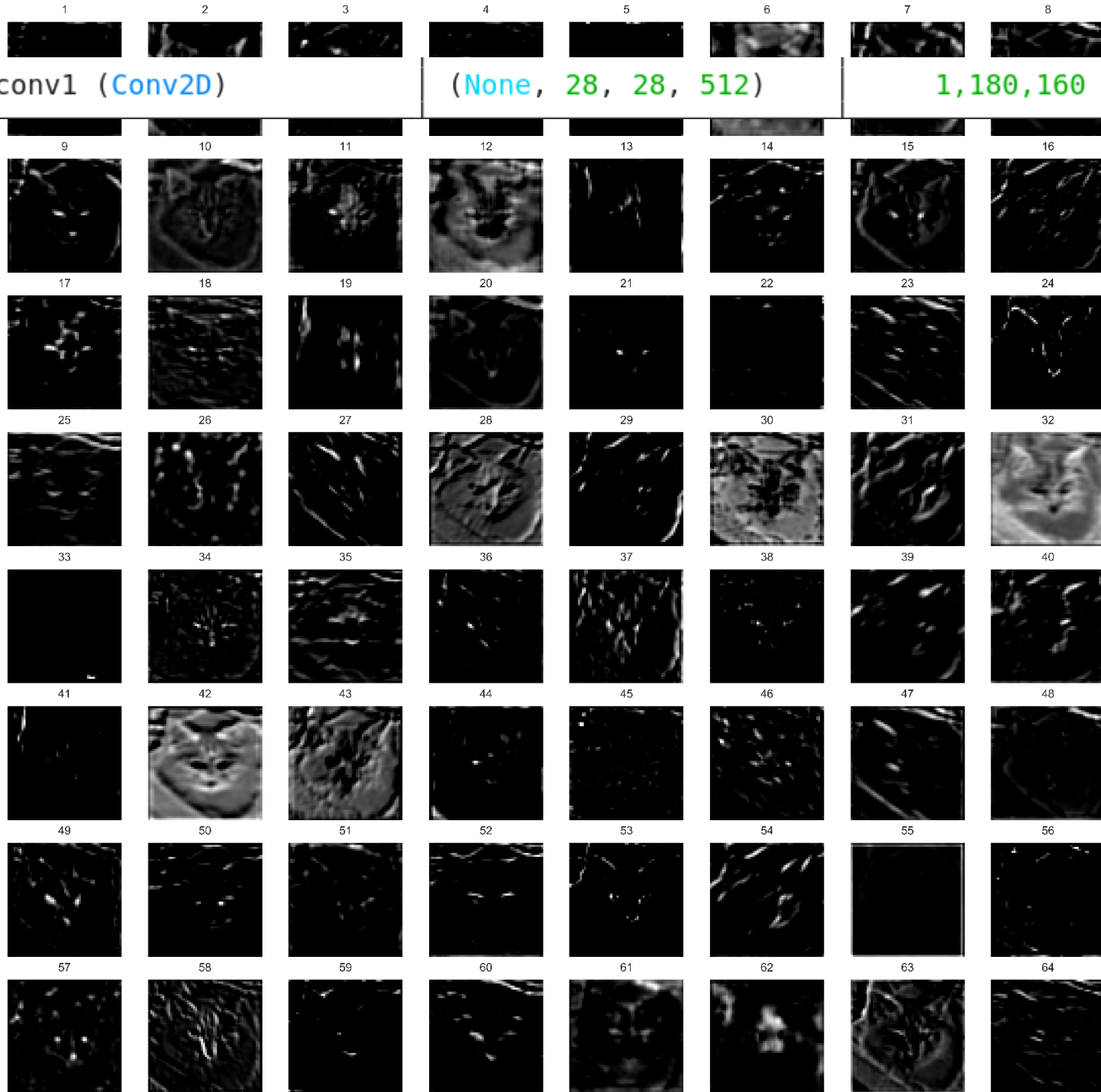
maxpool

B4
conv 3x3x512
conv 3x3x512
conv 3x3x512

maxpool

B5
conv 3x3x512
conv 3x3x512
conv 3x3x512

maxpool



block5_conv1

block5_conv1 (Conv2D)

(None, 14, 14, 512)

2,359,808

B1

conv 3x3x64

conv 3x3x64

maxpool

B2

conv 3x3x128

conv 3x3x128

maxpool

B3

conv 3x3x256

conv 3x3x256

conv 3x3x256

conv 3x3x256

maxpool

B4

conv 3x3x512

conv 3x3x512

conv 3x3x512

conv 3x3x512

maxpool

B5

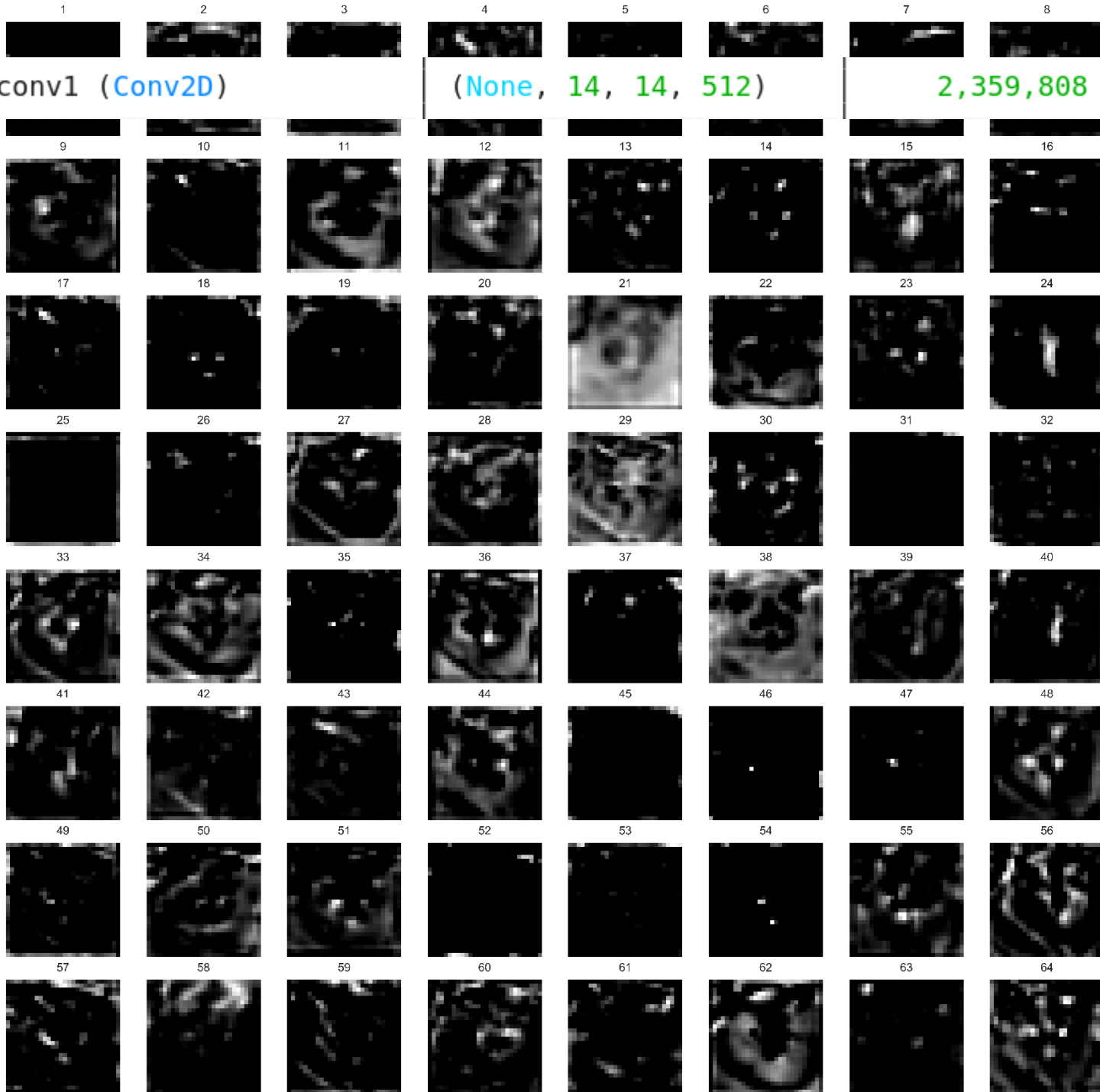
conv 3x3x512

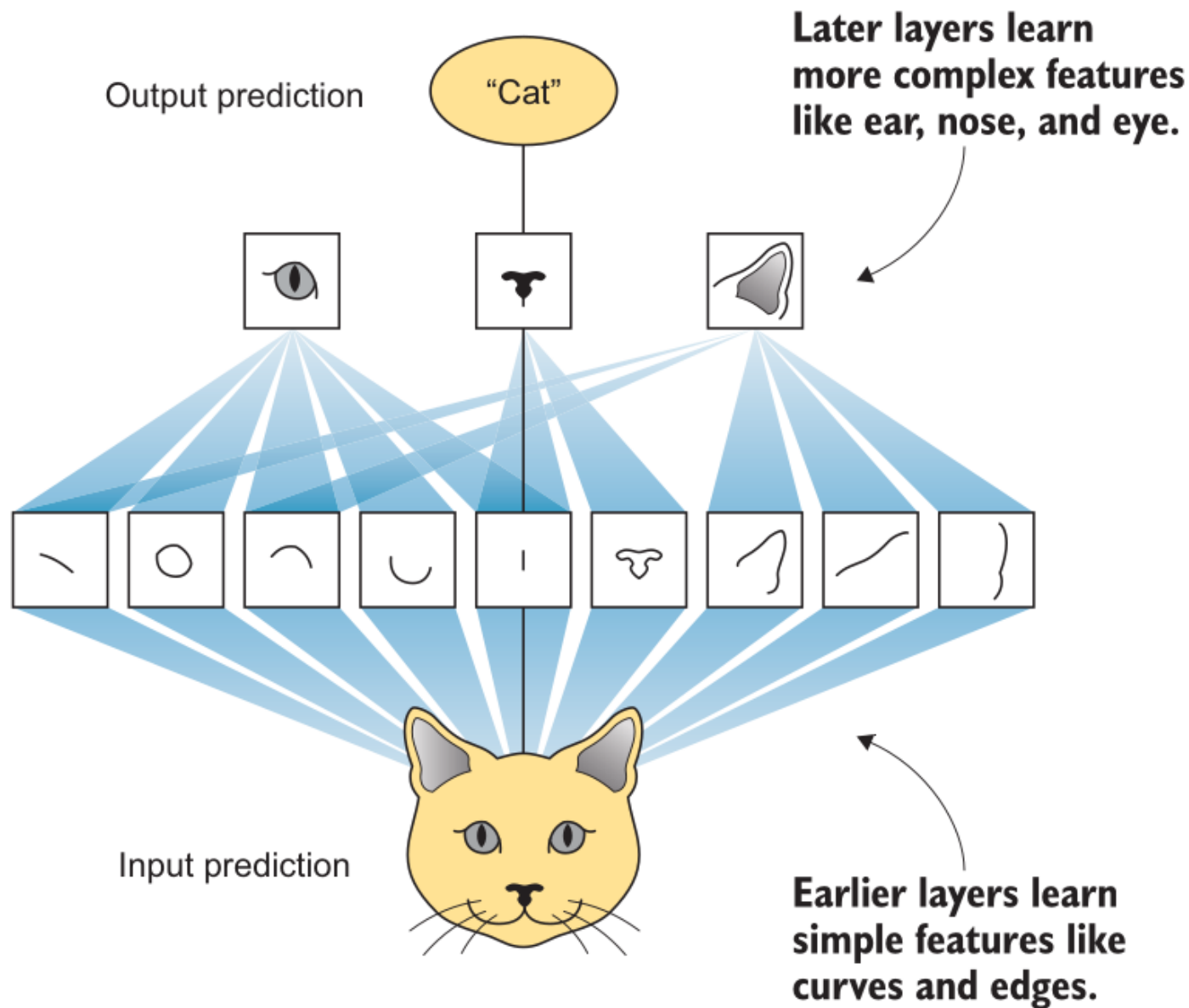
conv 3x3x512

conv 3x3x512

conv 3x3x512

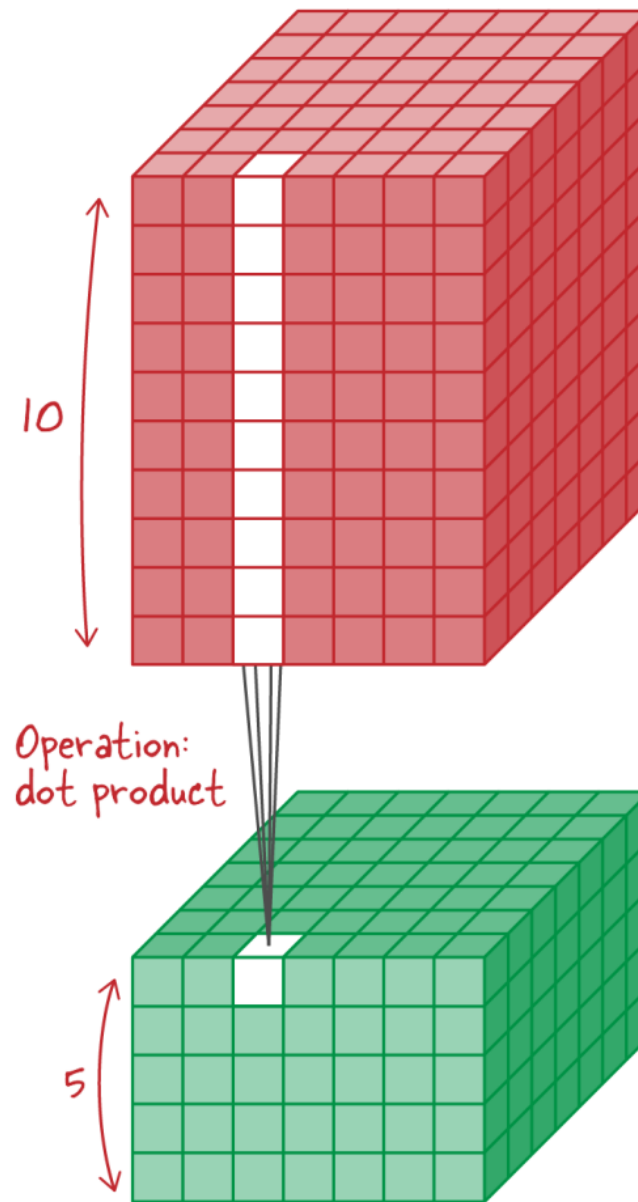
maxpool





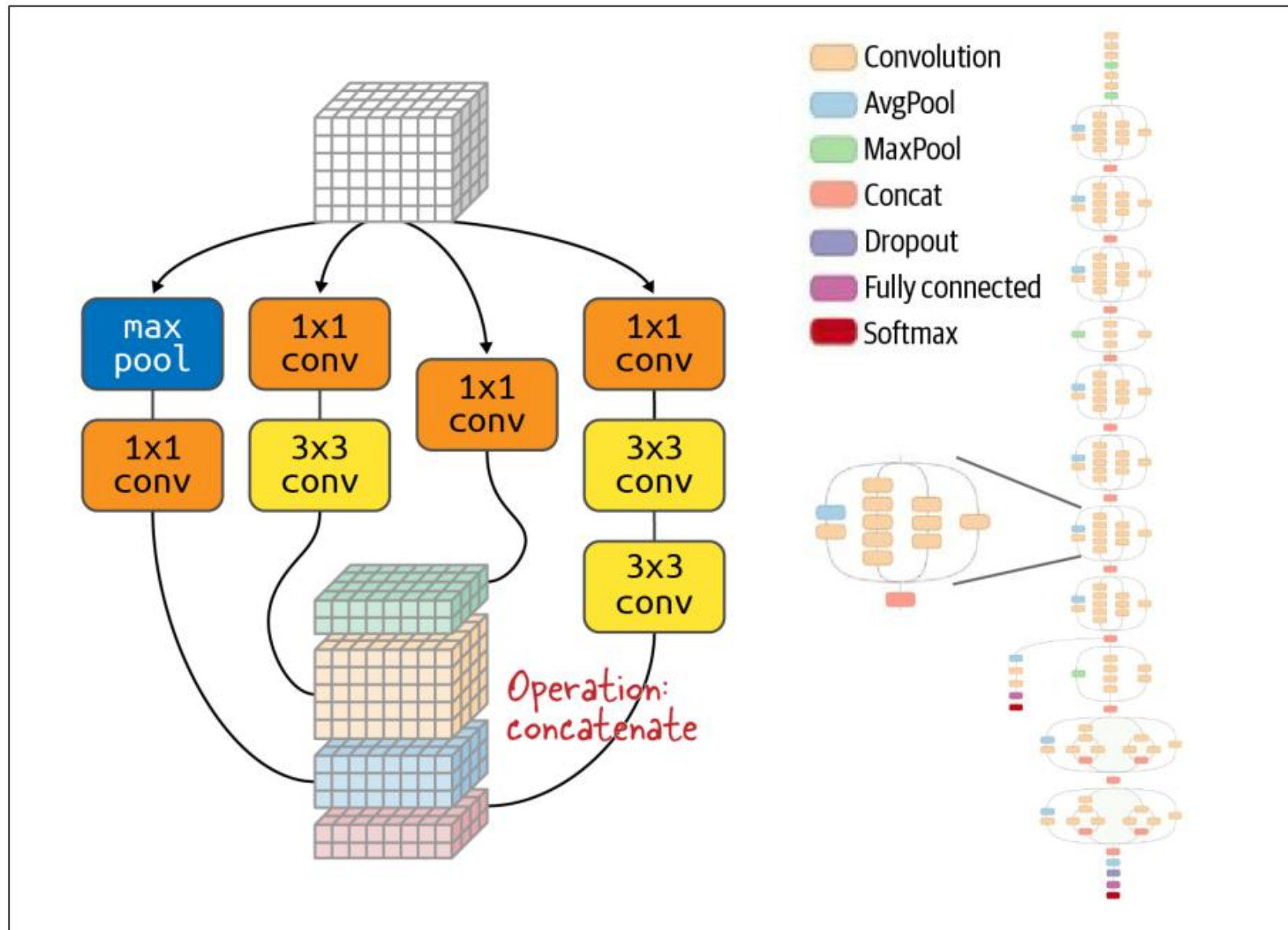
CNNs learn the image features through its layers.

Convolution 1x1



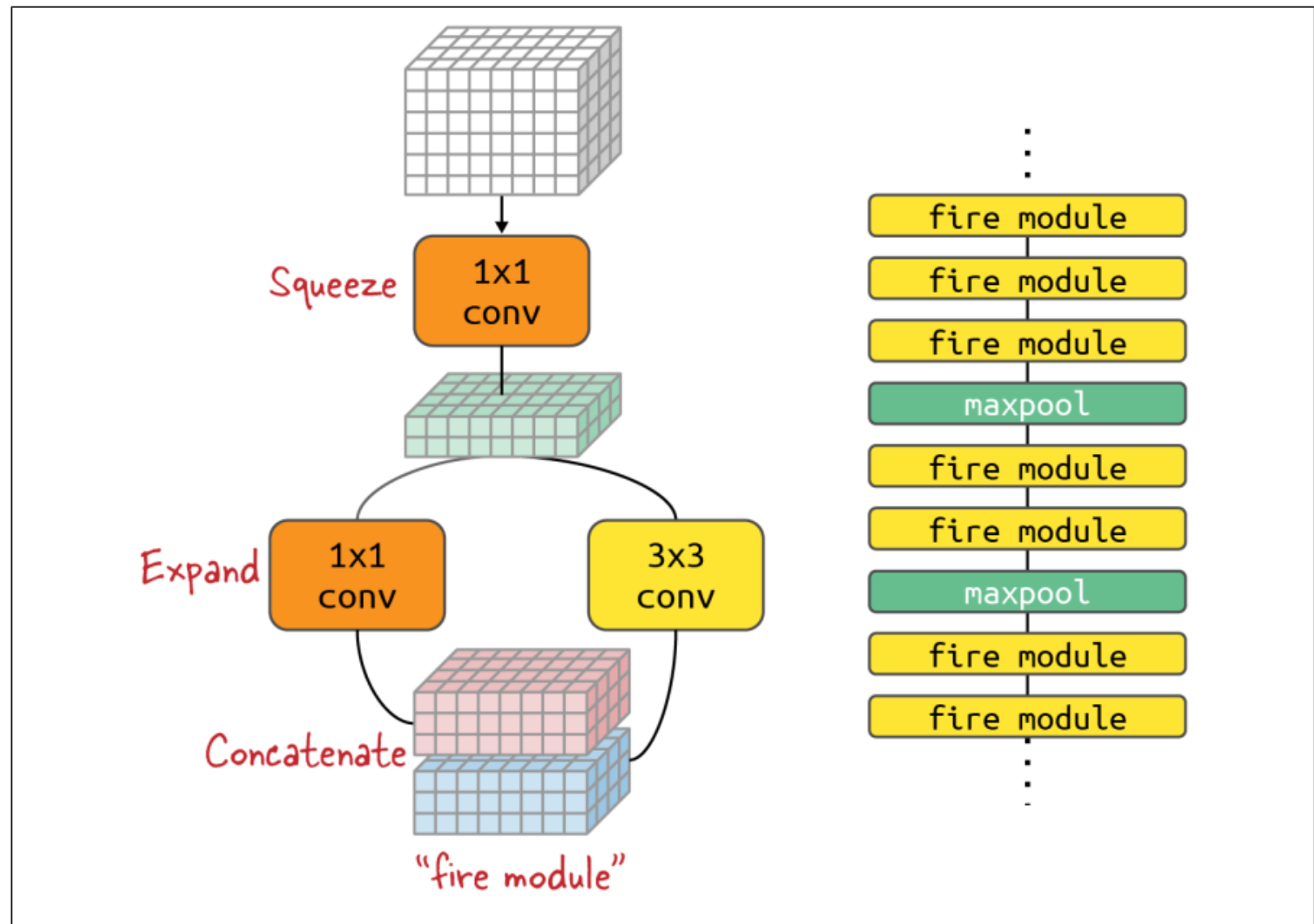
A 1x1 convolutional layer. Each filter has 10 parameters because it acts on a 10-channel input. 5 such filters are applied, each with its own learnable parameters (not shown in the figure), resulting in 5 channels of output data.

Inception



Example of an Inception module. The entire InceptionV3 architecture (on the right) is made up of many such modules.

SqueezeNet

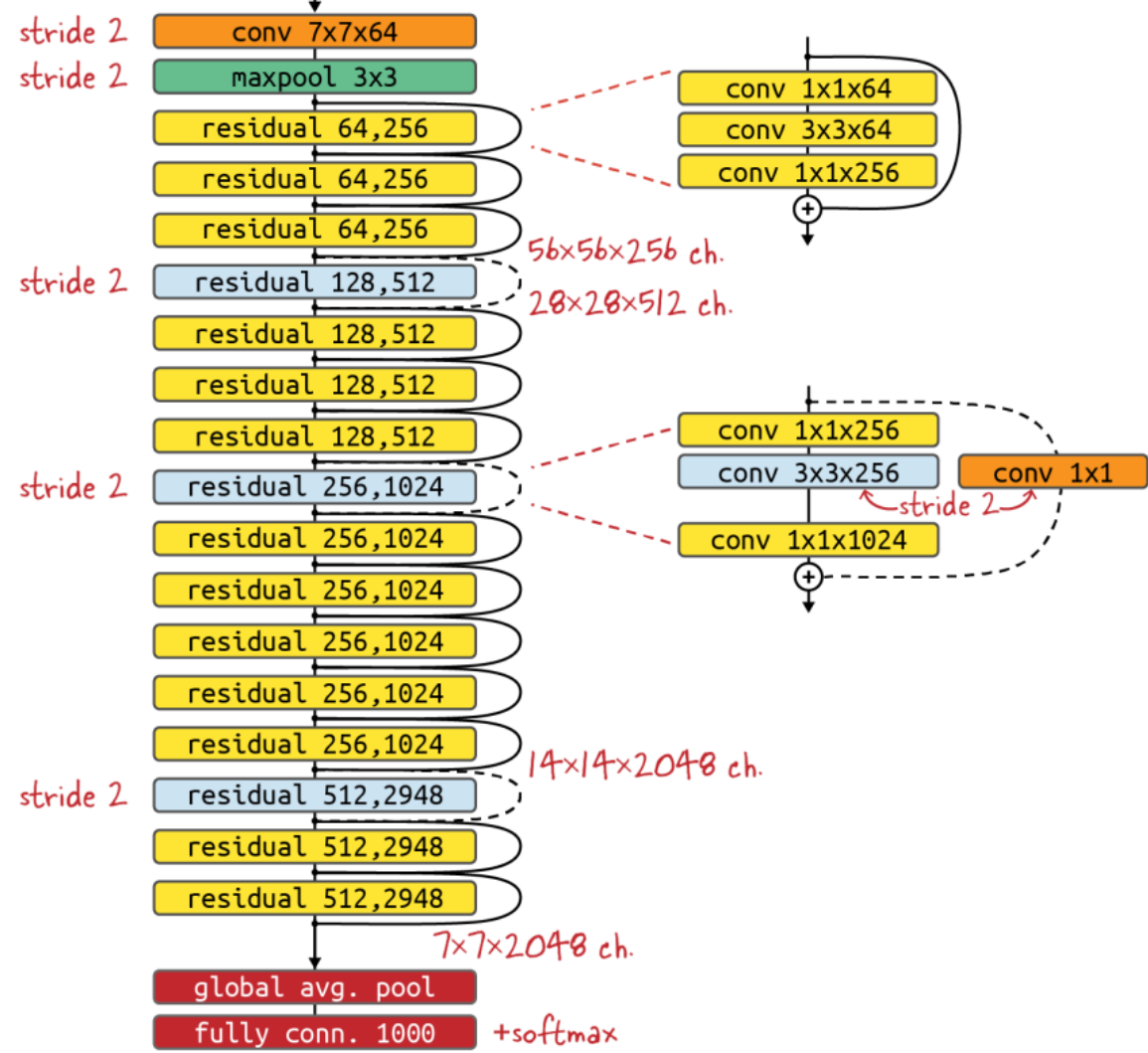


A simplified and standardized convolutional module from the SqueezeNet architecture. The architecture, shown on the right, alternates these "fire modules" with max-pooling layers.

ResNet and Skip Connections



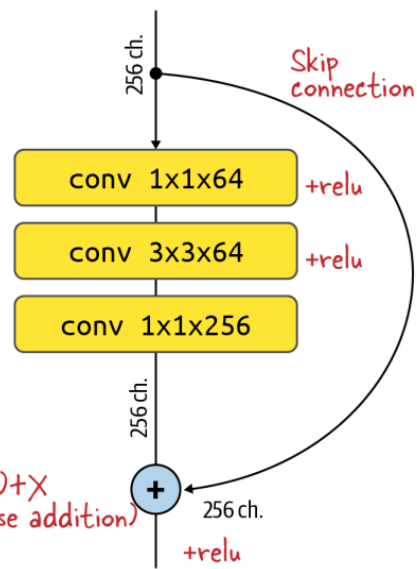
224x224x3 ch.



Input: x

$C(x)$

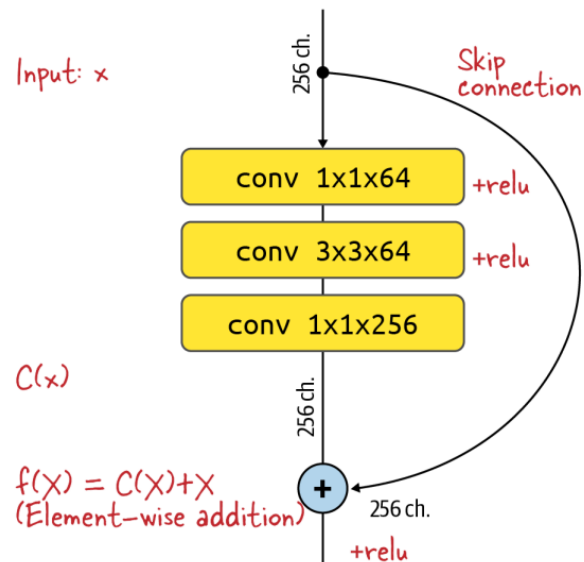
$f(x) = C(x) + x$
(Element-wise addition)



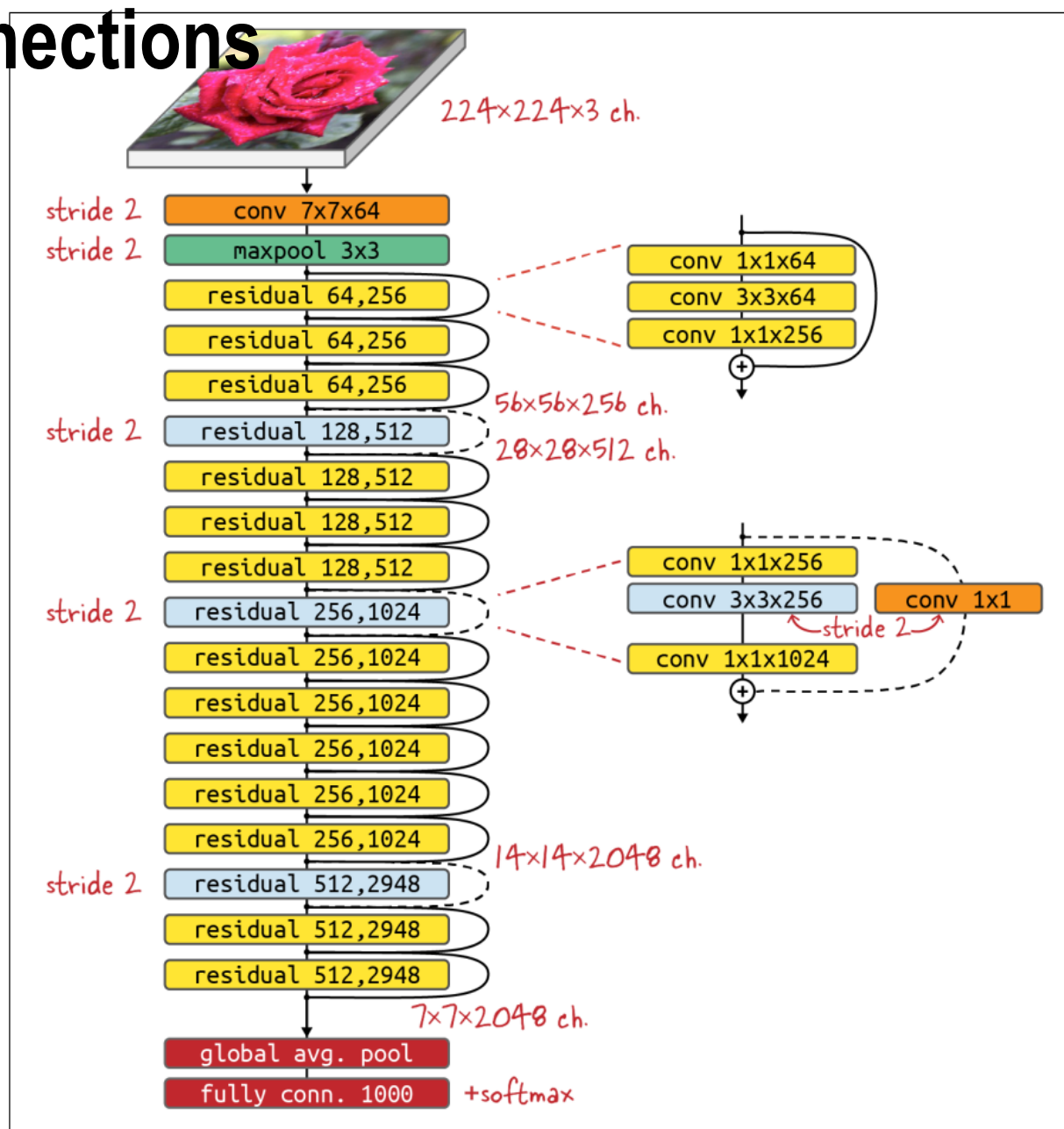
A residual block in ResNet.

The ResNet50 architecture. Residual blocks with a stride of 2 have a skip connection implemented using a 1x1 convolution (dotted line). The ResNet 101 architecture is similar, with the “residual 256, 1,024” block repeated 23 times instead of 6.

ResNet and Skip Connections

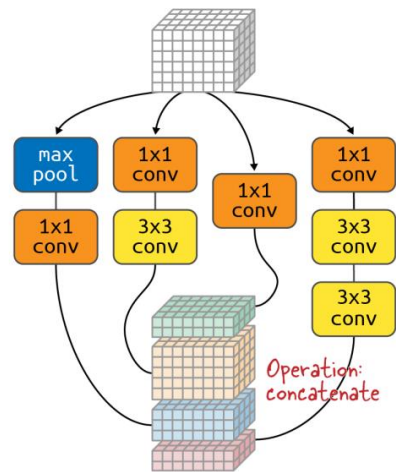


A residual block in ResNet.

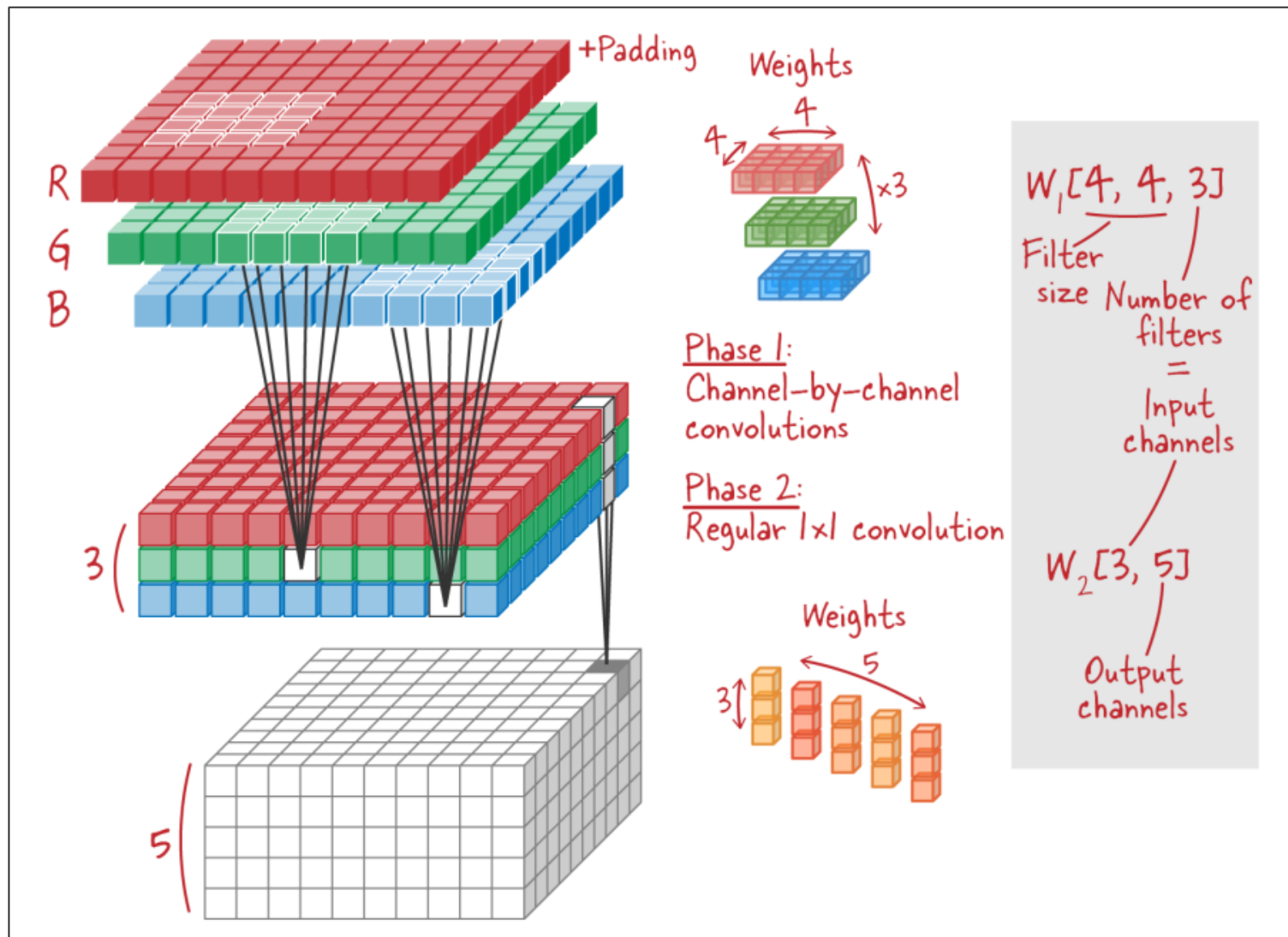


The ResNet50 architecture. Residual blocks with a stride of 2 have a skip connection implemented using a 1×1 convolution (dotted line). The ResNet 101 architecture is similar, with the “residual 256, 1,024” block repeated 23 times instead of 6.

Xception

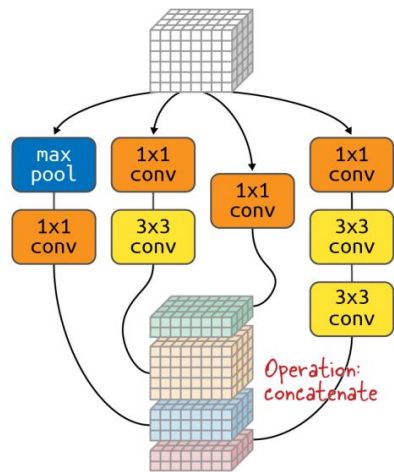


Example of an Inception module. The (the right) is made up of many such modules.

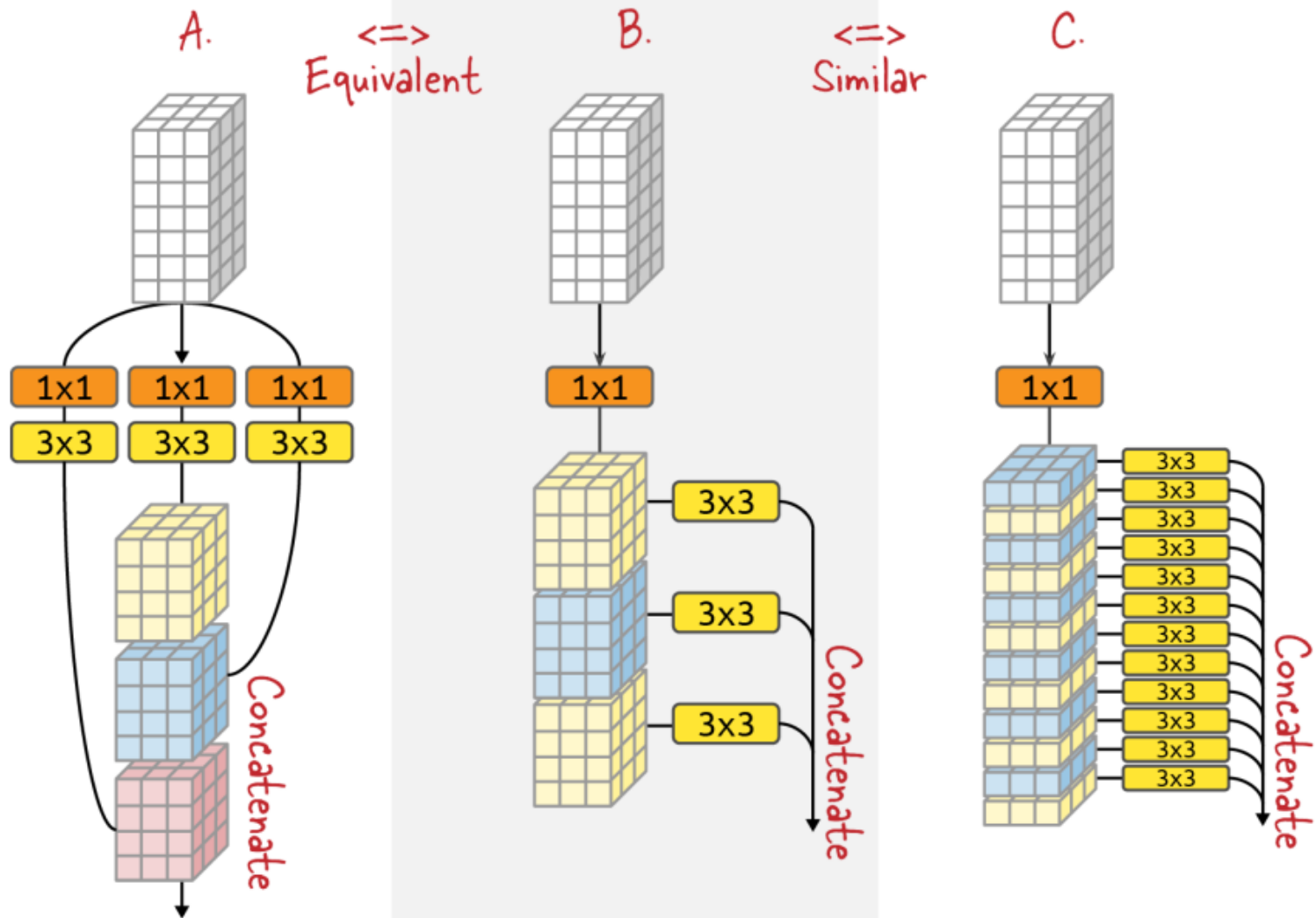


A 4x4 depth-separable convolutional layer. In phase 1, 4x4 filters are applied independently to each channel, producing an equal number of output channels. In phase 2, the output channels are then recombined using a 1x1 convolution (multiple weighted sums of the channels).

Xception

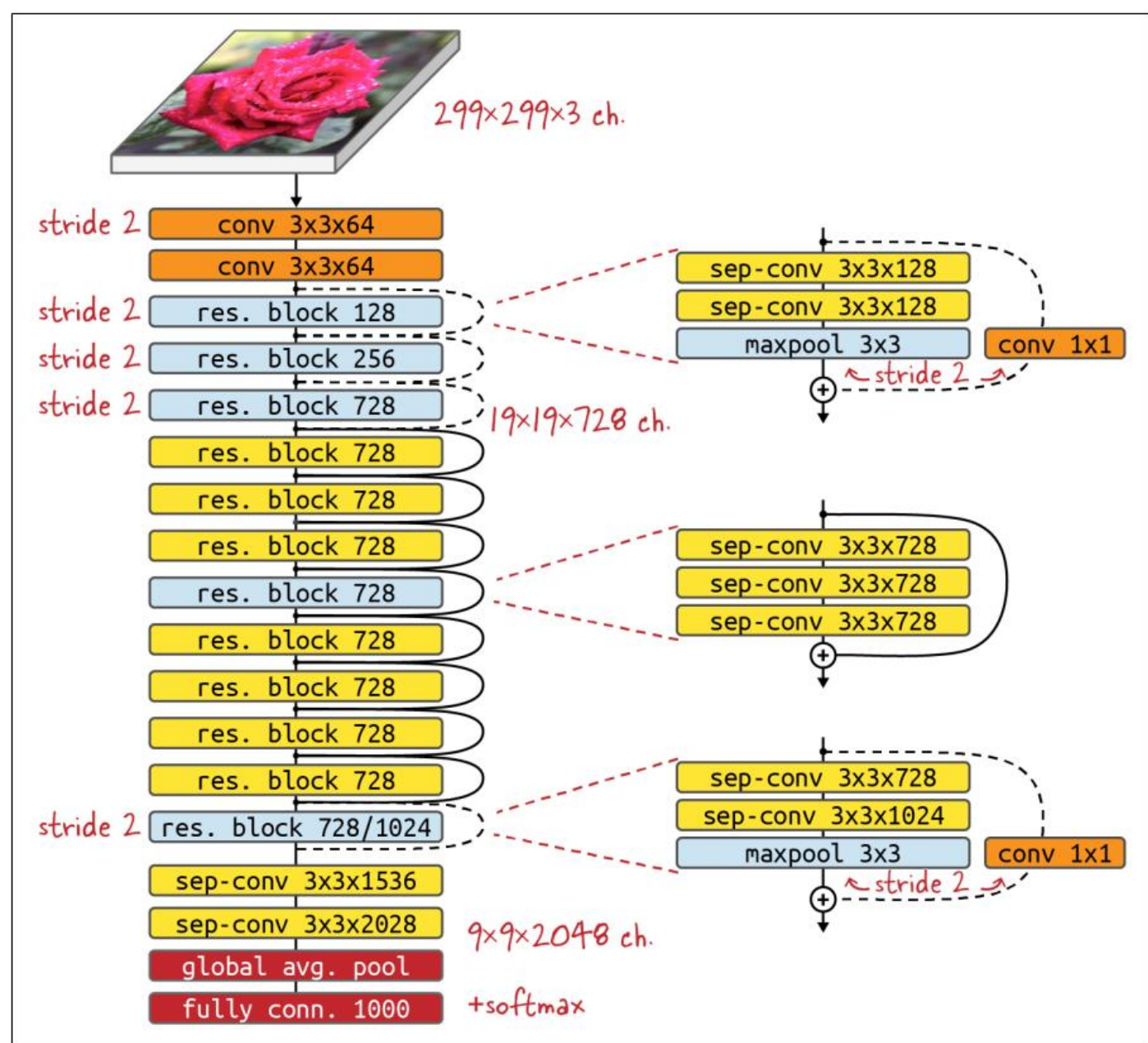


Example of an Inception module. The entire network (to the right) is made up of many such modules.



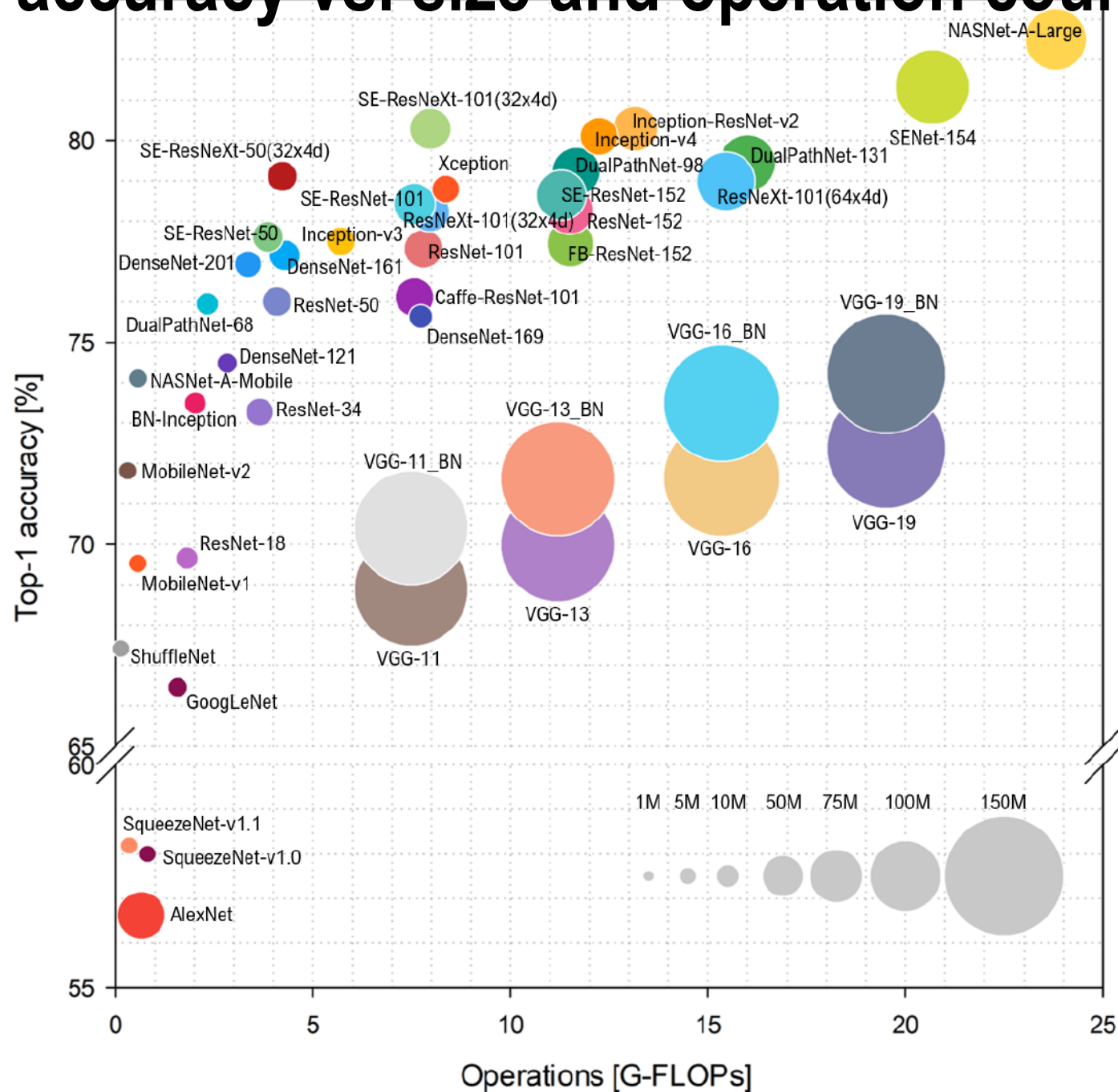
Architectural similarity between Inception modules and depth-separable convolutions: (A) a simplified Inception module with three parallel convolutional paths; (B) an exactly equivalent setup where there is a single 1x1 convolution but it outputs three times more channels; (C) a very similar setup with more 3x3 convolutions. This is exactly a separable convolution with the order of the 1x1 and 3x3 operations swapped.

Xception



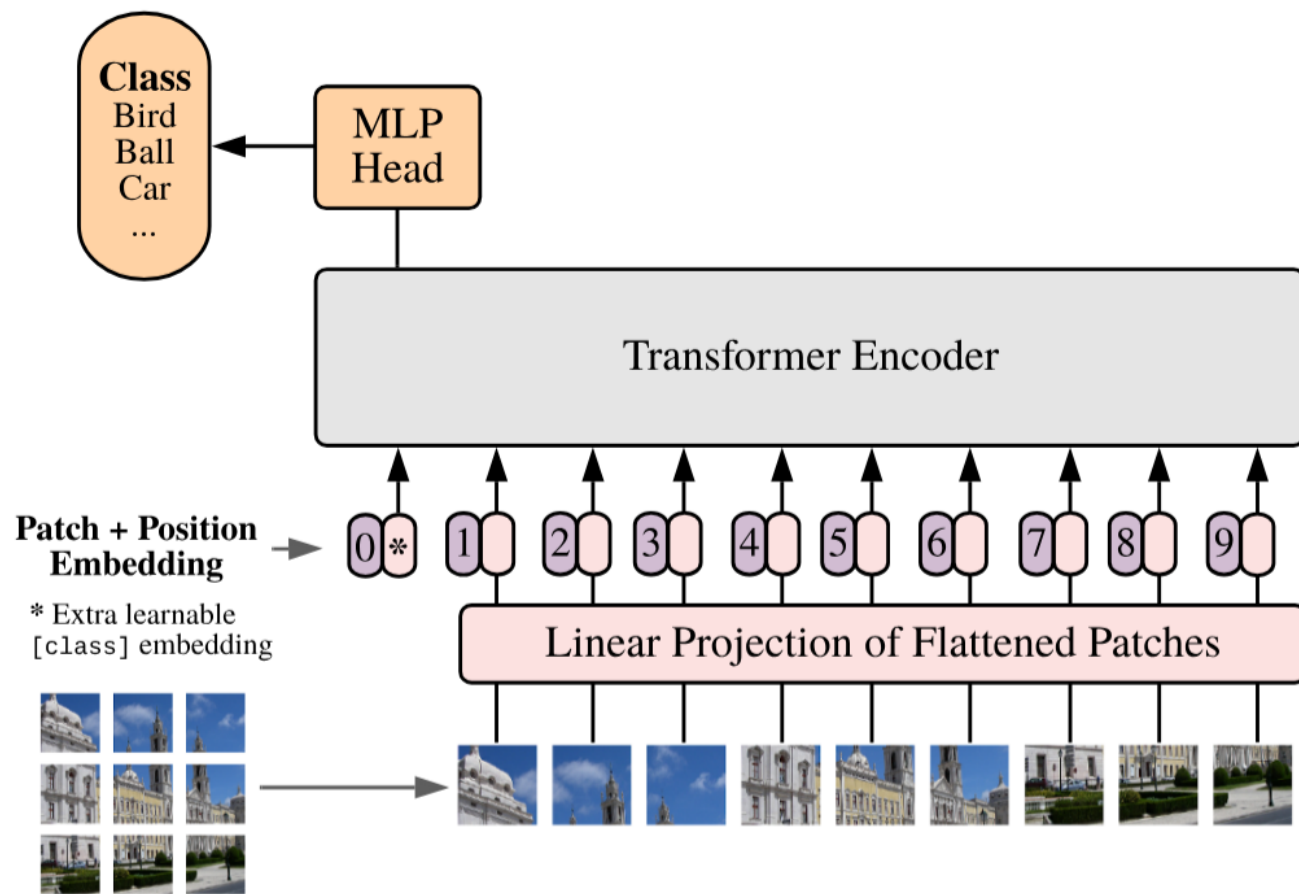
The Xception architecture with 36 convolutional layers. The architecture is inspired by ResNet but uses separable convolutions instead of traditional ones, except in the first two layers.

Network accuracy vs. size and operation counts

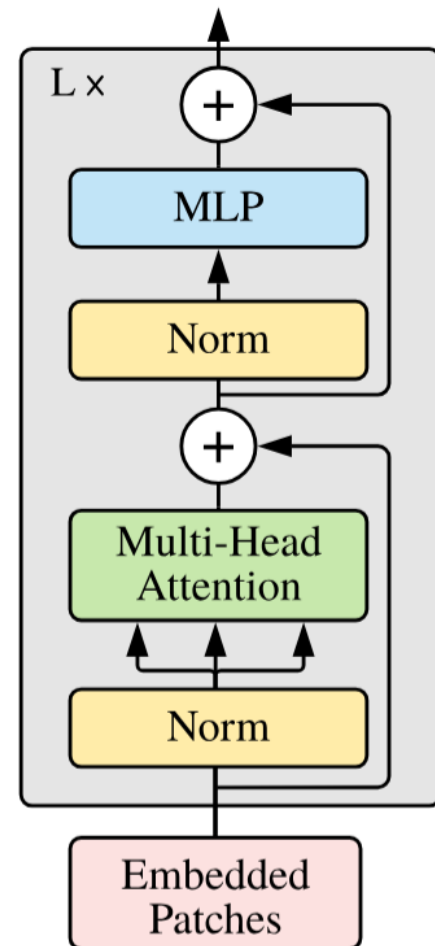


Vision Transformer

Vision Transformer (ViT)



Transformer Encoder



Global Context Vision Transformers

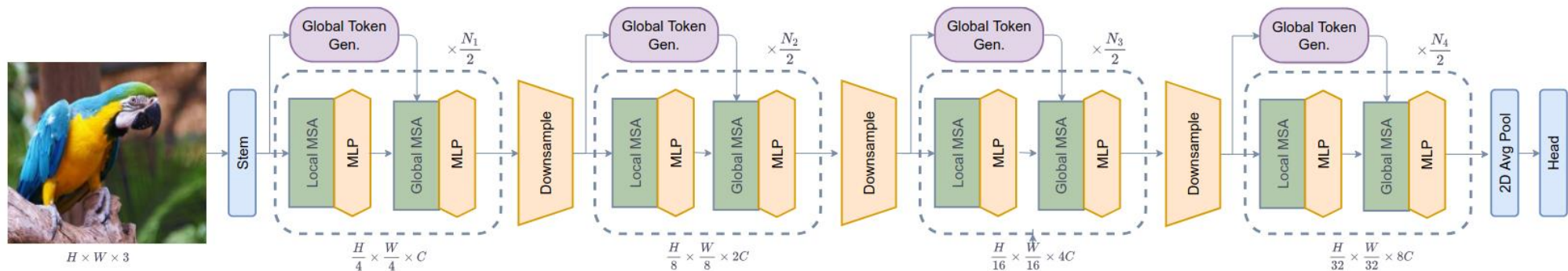
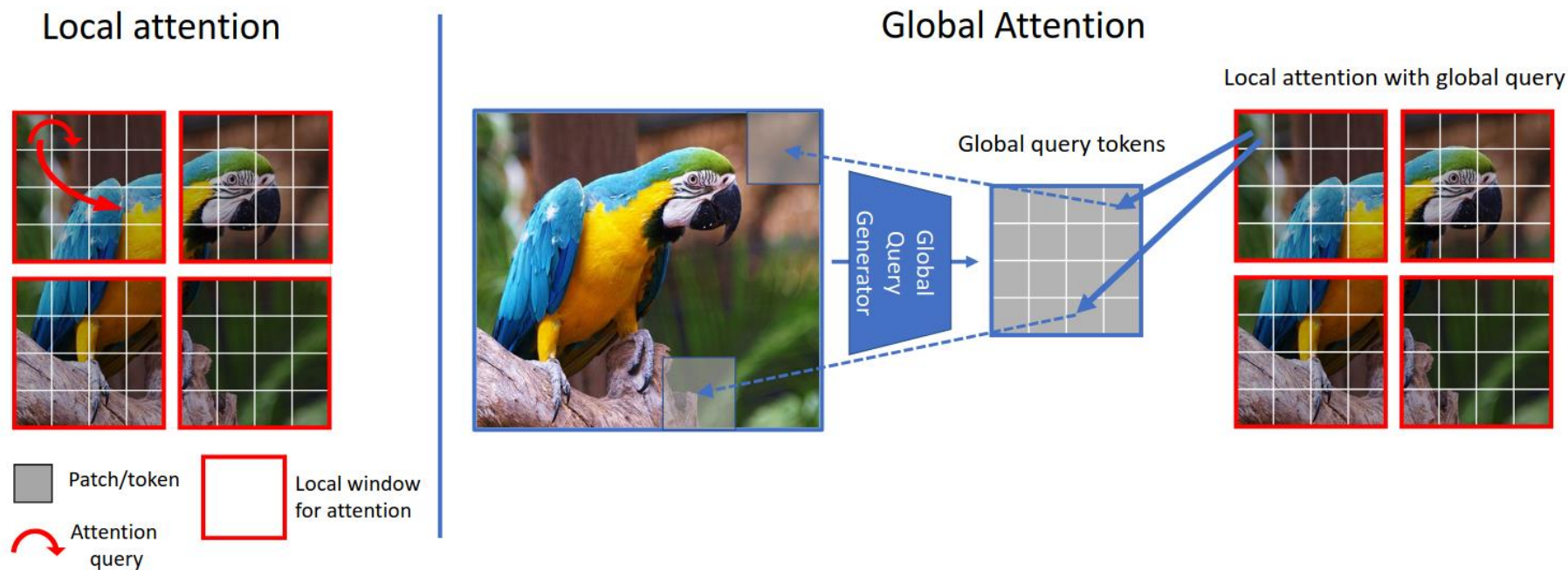
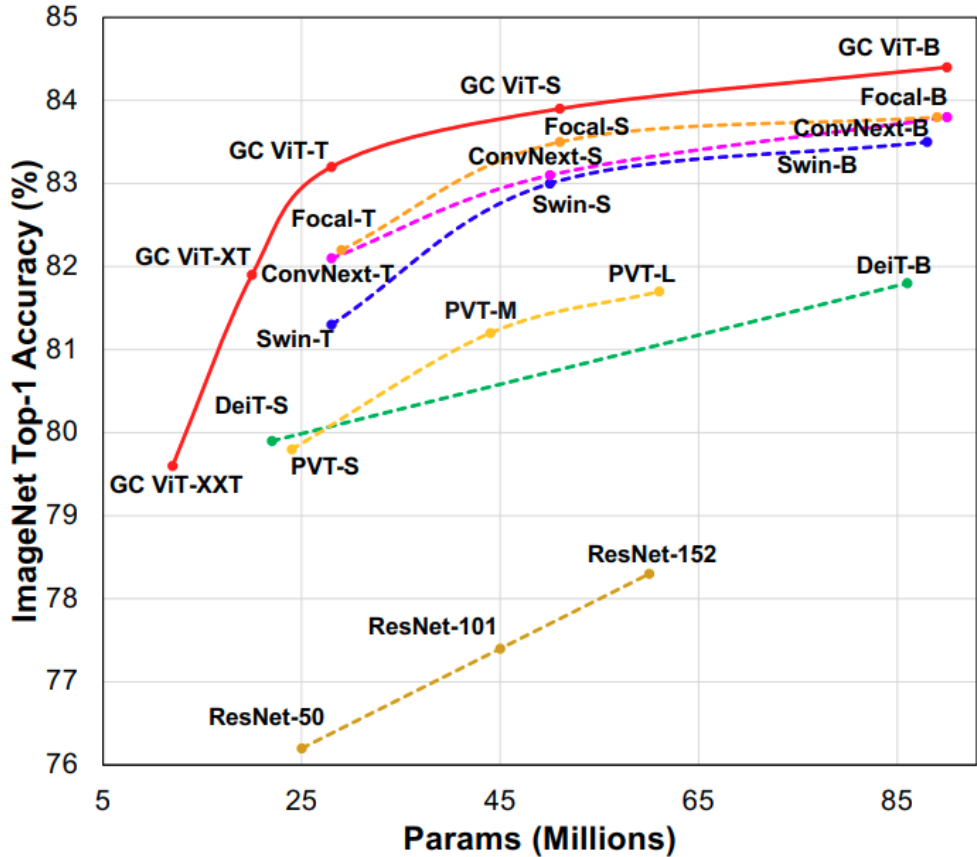
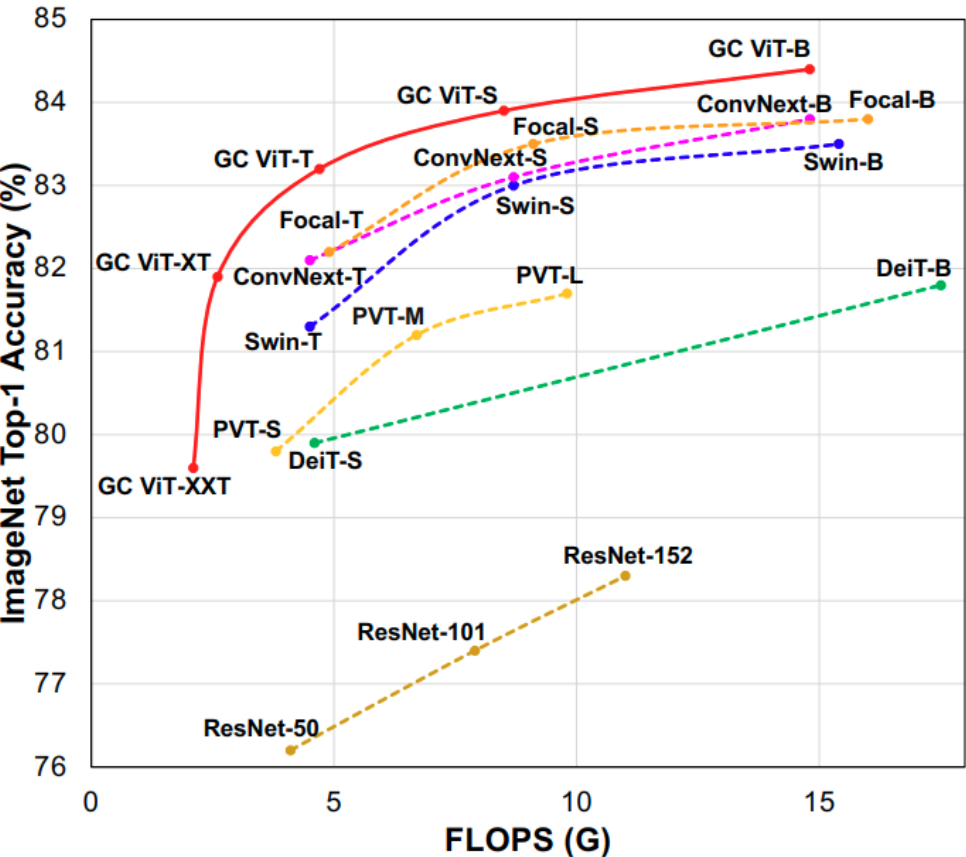


Figure 2 – Architecture of the proposed GC ViT. At each stage, a query generator extracts global query tokens which captures long-range information by interacting with local key and value representations. We use alternating blocks of local and global context self attention layers. Best viewed in color.



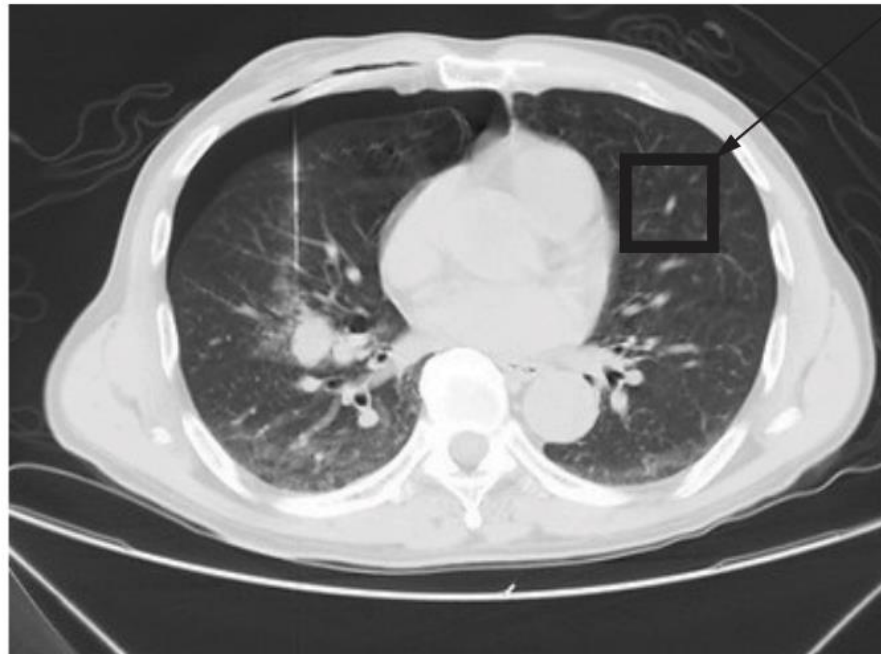
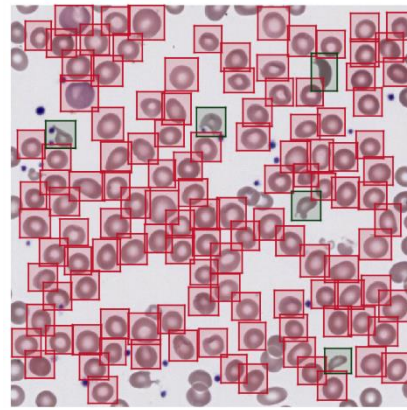
Global Context Vision Transformers



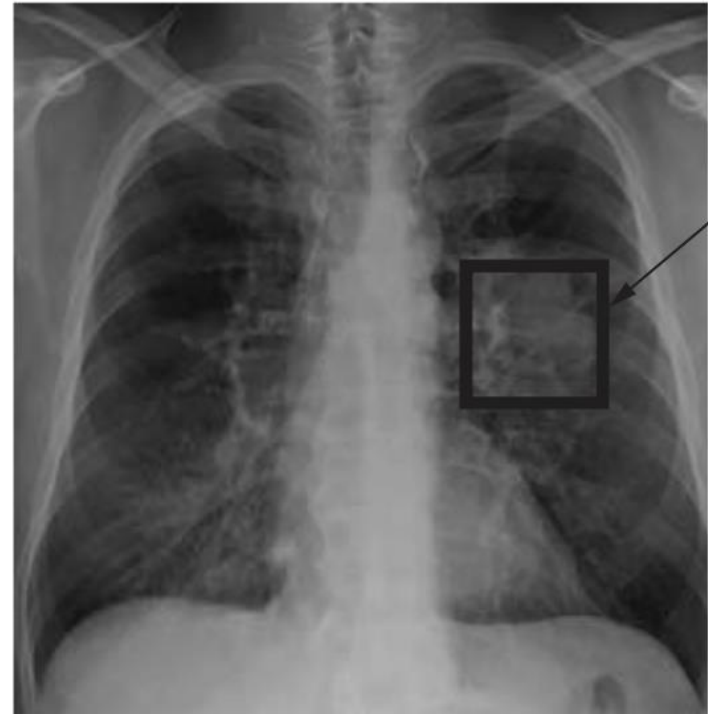
Object Detection

- ❑ Introduction
- ❑ Convolutional neural networks CNN
- ❑ Model Architectures
- ❑ Object Detection
- ❑ Segmentation
- ❑ Instance Segmentation
- ❑ Autoencoders

Object Detection



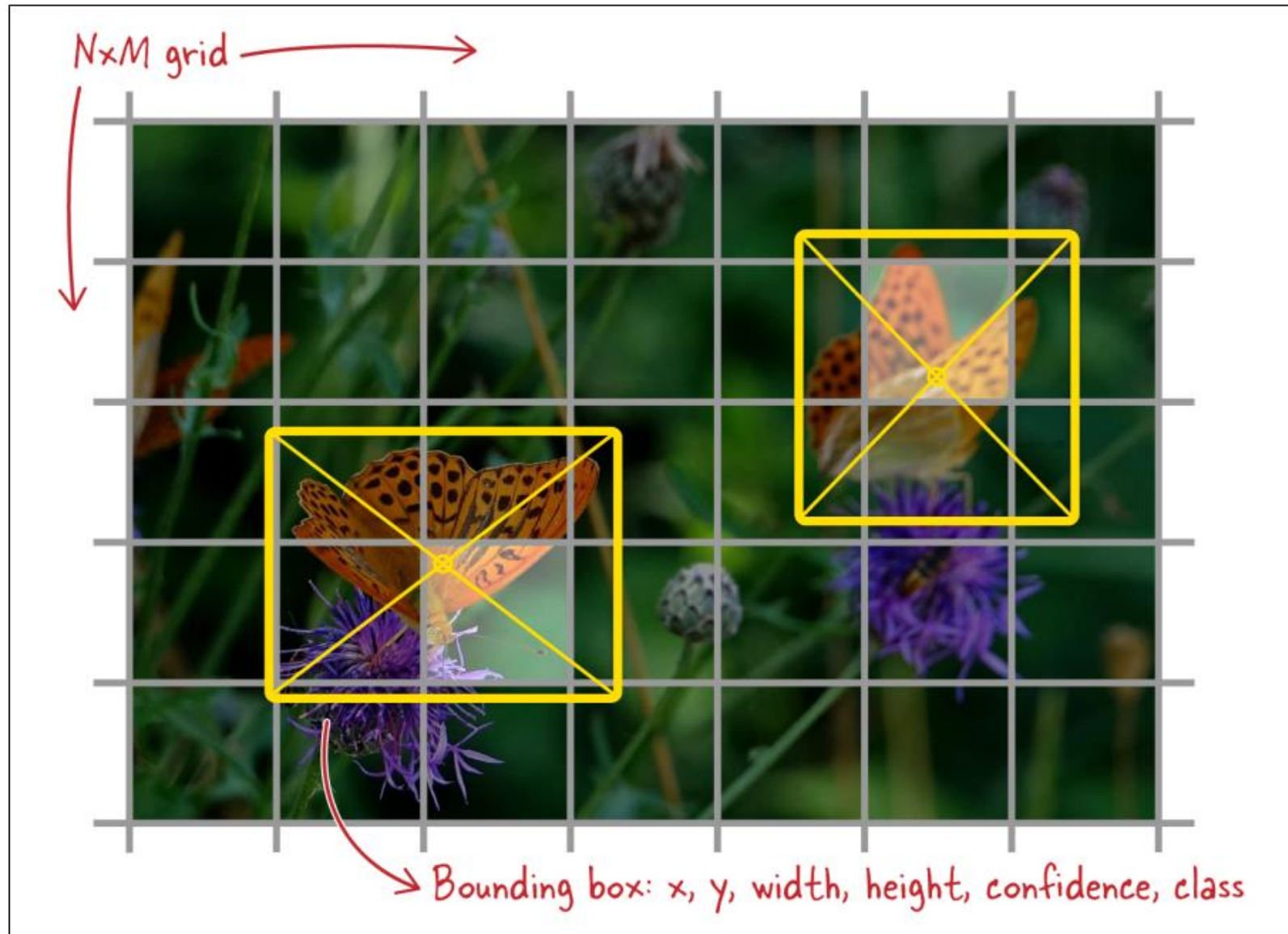
CT scan



X-ray

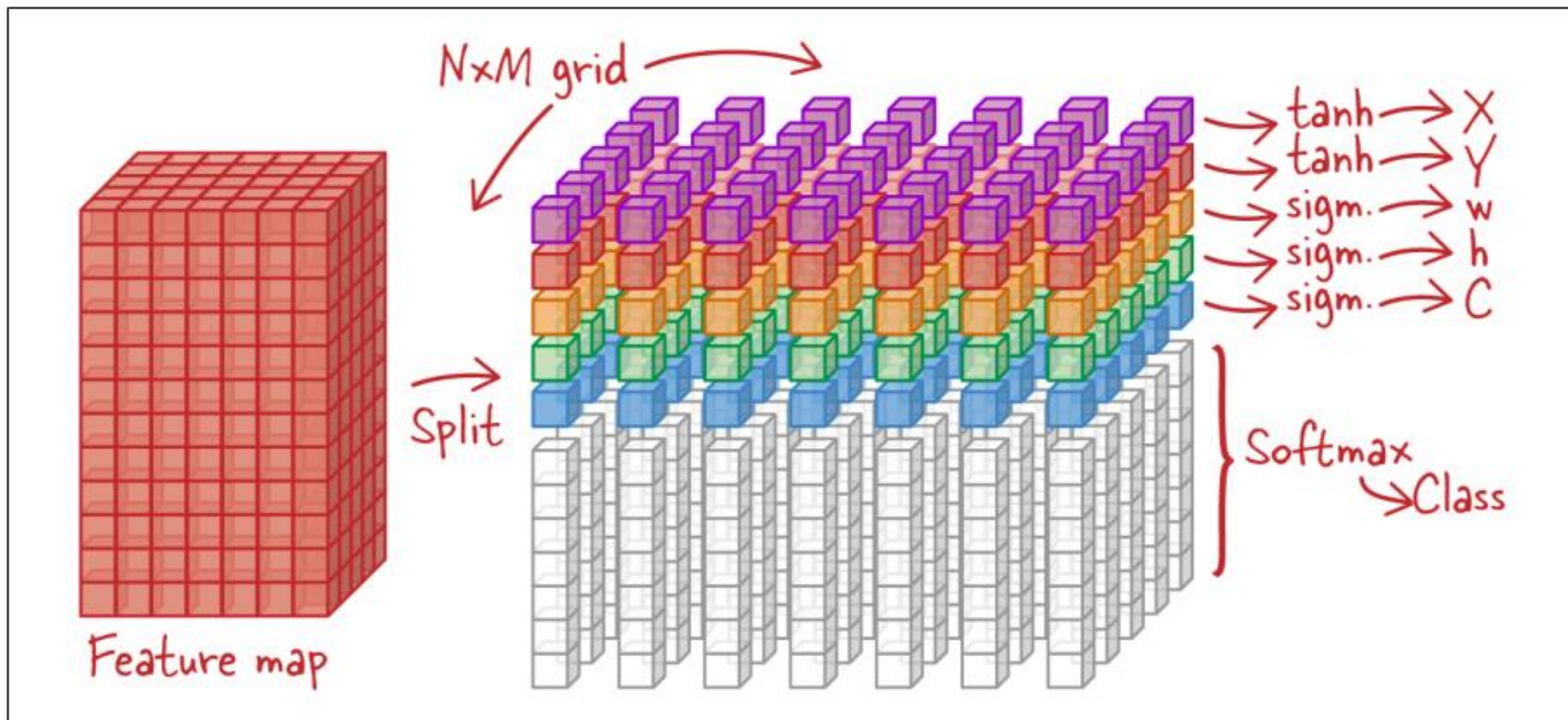
Vision systems are now able to learn patterns in X-ray images to identify tumors in earlier stages of development.

YOLO (you only look once)



*The YOLO grid. Each grid cell predicts a bounding box for an object whose center is somewhere in that cell. Image from *Arthropods dataset*.*

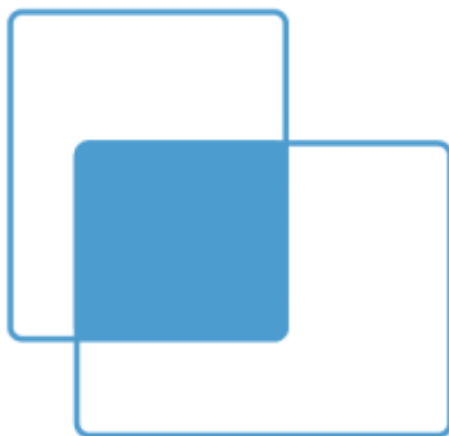
YOLO (you only look once)



A YOLO detection head predicts, for every grid cell, a bounding box (x, y, w, h), the confidence C of there being an object in this location, and the class of the object.

Metrics

$$\text{IOU} = \frac{\text{Area of intersection}}{\text{Area of union}}$$



Examples



IOU = 0.1



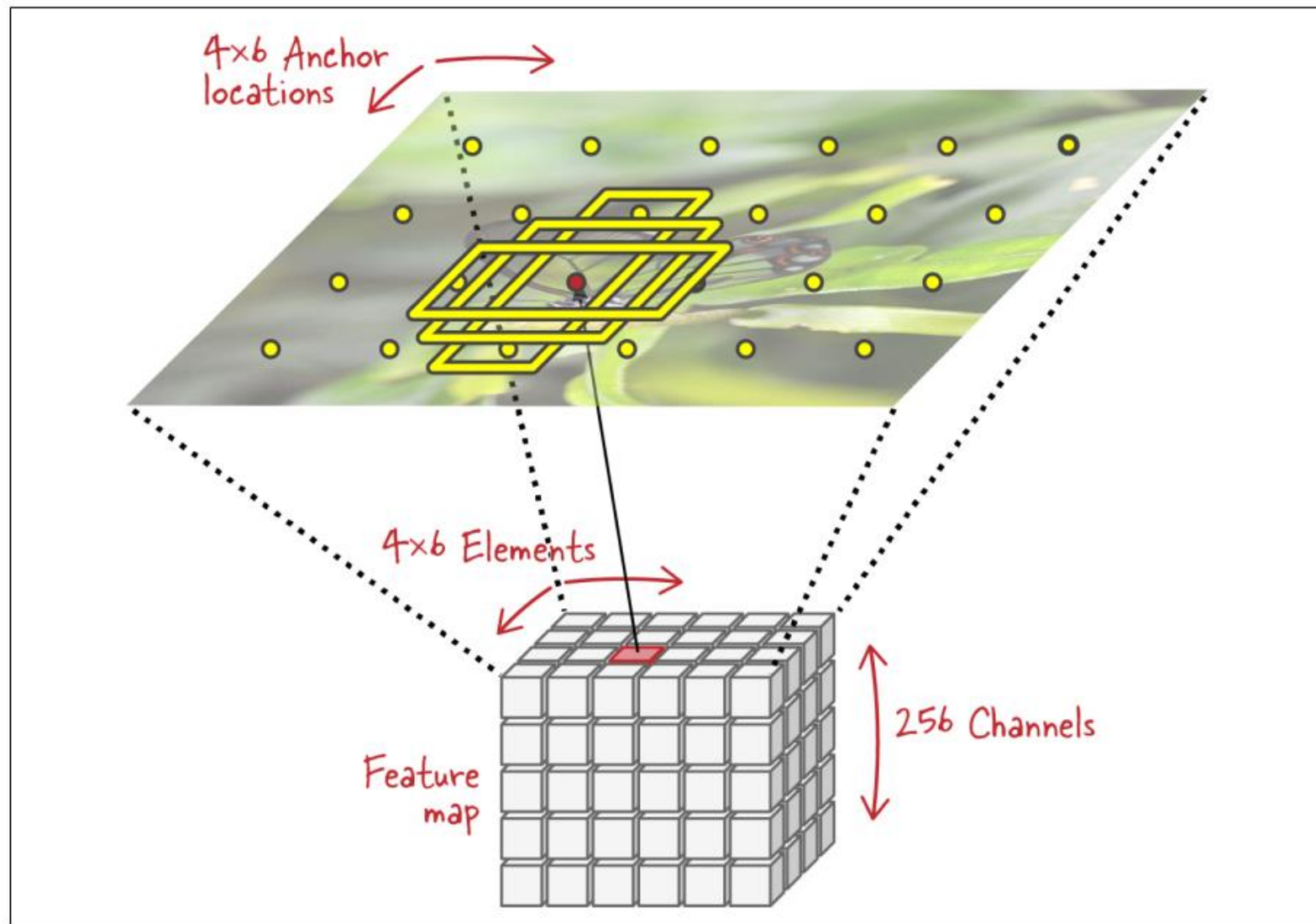
IOU = 0.3



IOU = 0.6

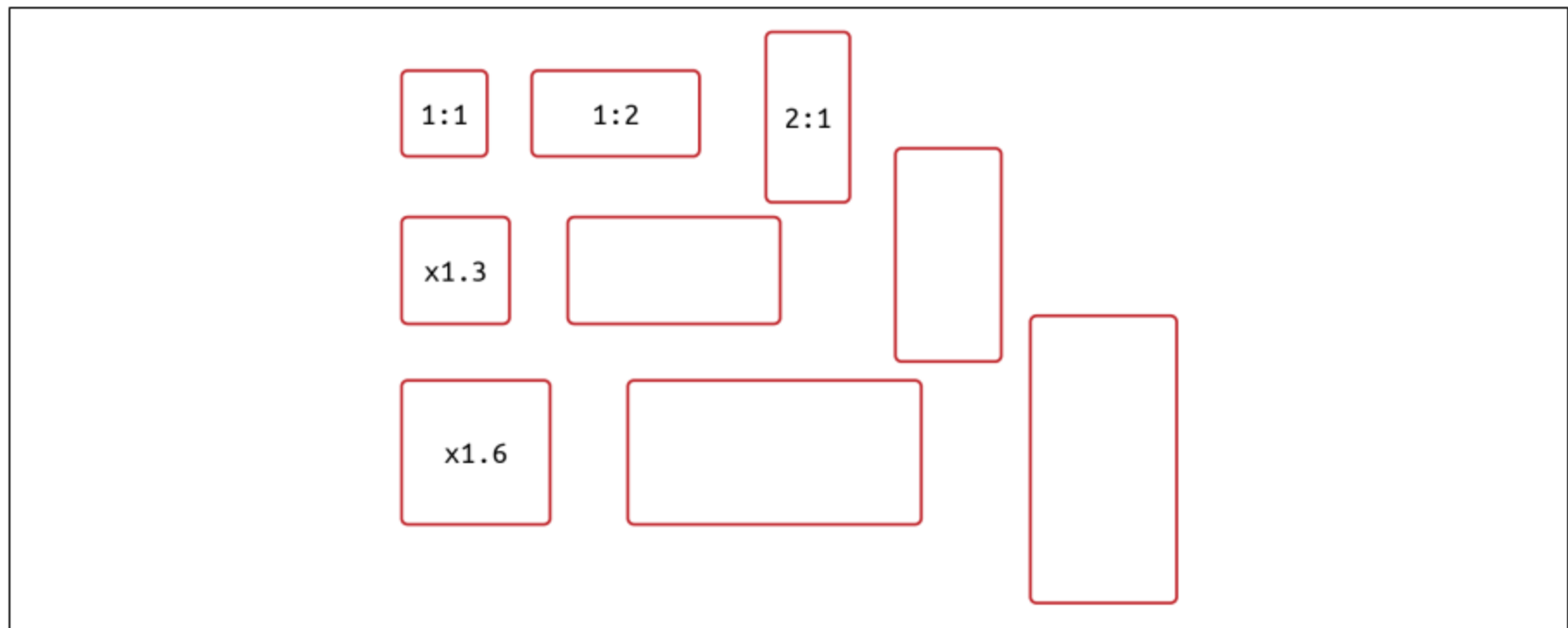
The IOU metric.

RetinaNet



Conceptual view of the RetinaNet detection head. Each spatial location in a feature map corresponds to a series of anchors in the image, all centered at the same point. For clarity, only three such anchors are shown in the illustration, but RetinaNet would have nine at every location.

RetinaNet



The nine different anchor types used in RetinaNet. Three aspect ratios and three different sizes.

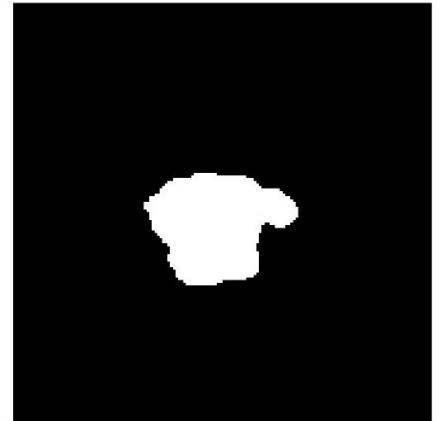
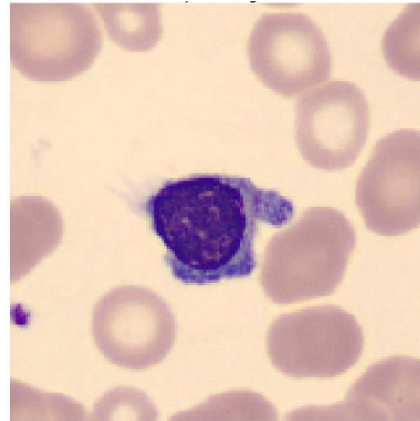
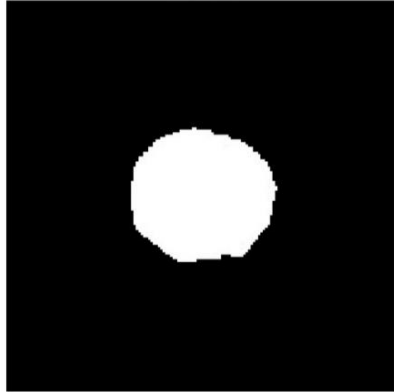
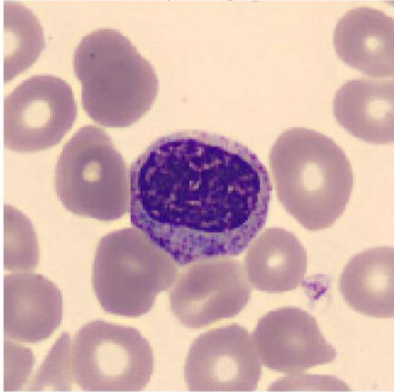
The diagram illustrates the FPN architecture for object detection. It starts with an input image of size $640 \times 960 \times 3$. This image is processed by a ResNet50 backbone, which produces feature maps P_3 , P_4 , and P_5 with dimensions $80 \times 120 \times 512$, $40 \times 60 \times 1024$, and $20 \times 30 \times 2048$ respectively. These feature maps are then processed by additional layers (P6, P7) to produce feature maps of size $10 \times 15 \times 256$ and $5 \times 7 \times 256$. The feature maps are then combined in a simplified FPN view to produce feature maps of size $80 \times 120 \times 256$, $40 \times 60 \times 256$, $20 \times 30 \times 256$, $10 \times 15 \times 256$, and $5 \times 7 \times 256$. The final output is a classification head (classes) and a detection head (boxes).

Complete view of the RetinaNet architecture. K is the number of target the number of anchor boxes at each position, which is nine in RetinaNet.

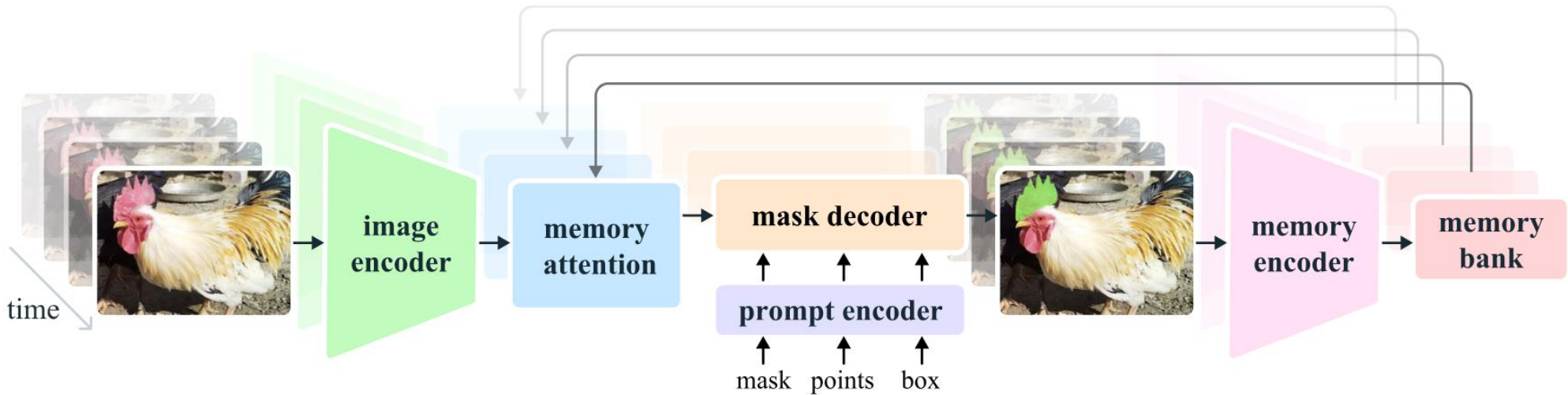
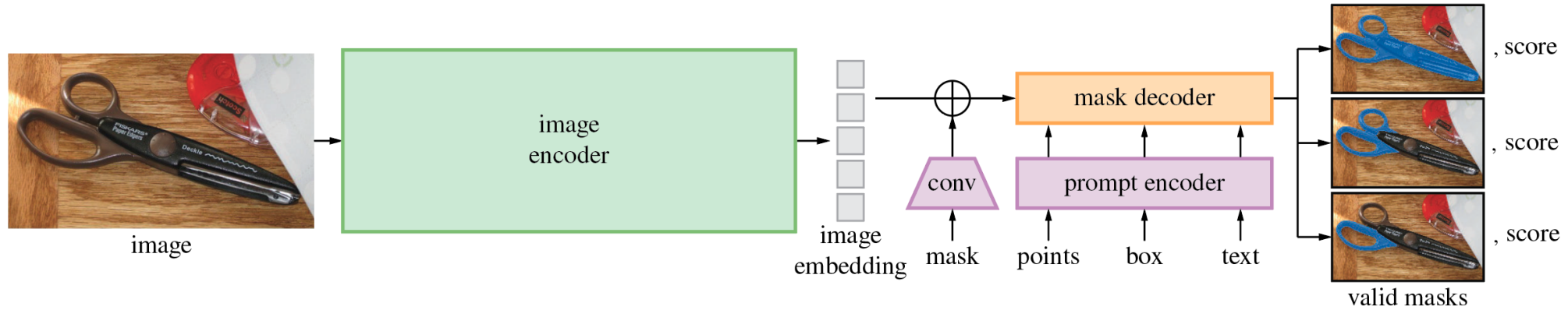
Segmentation

- ❑ Introduction
- ❑ Convolutional neural networks CNN
- ❑ Model Architectures
- ❑ Object Detection
- ❑ Segmentation
- ❑ Instance Segmentation
- ❑ Autoencoders

Segmentation



SAM (Segment Anything)

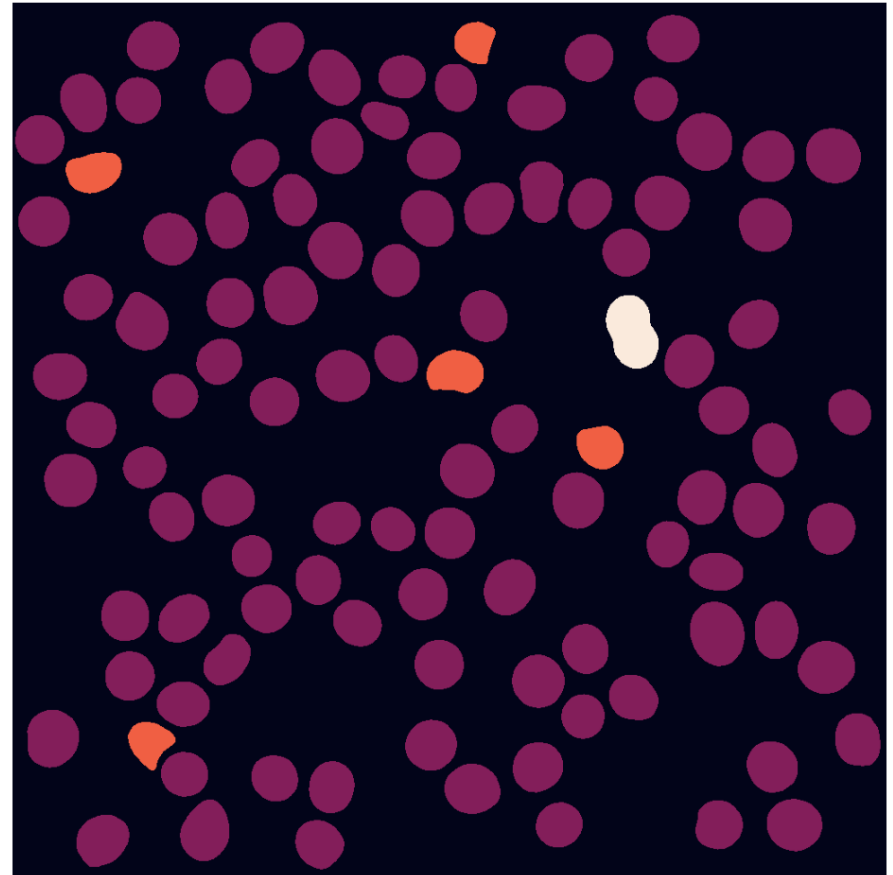
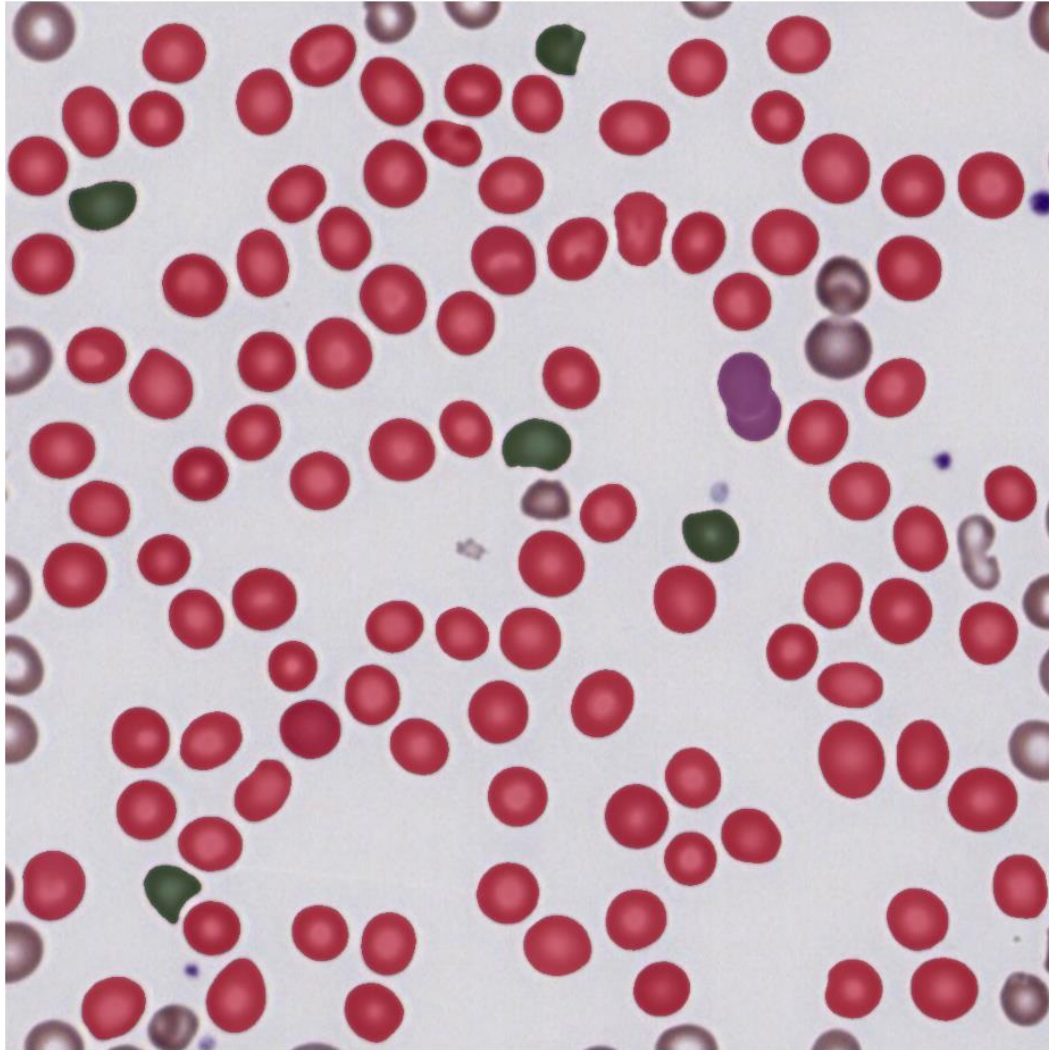


<https://segment-anything.com>

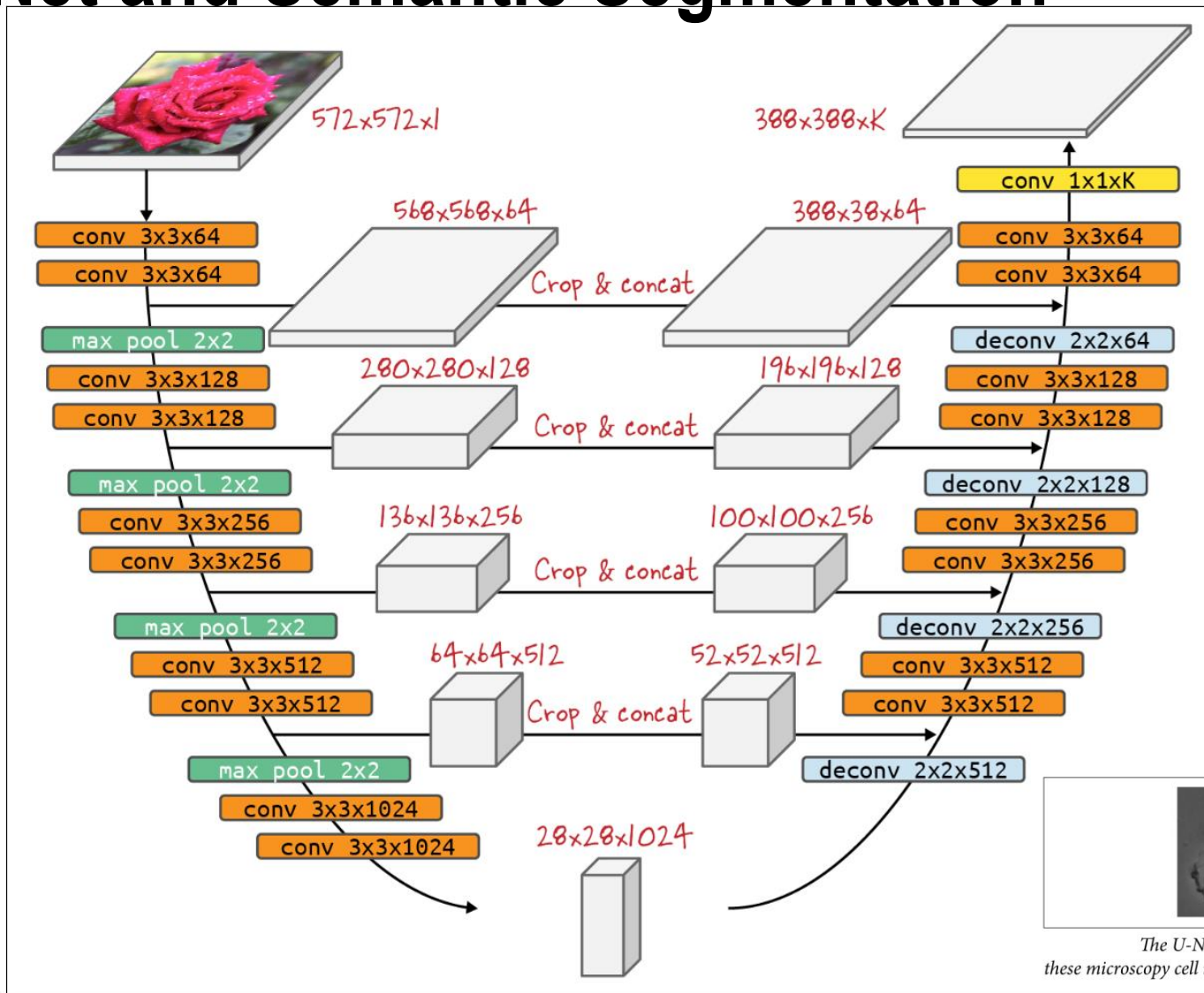
Instance Segmentation

- ❑ Introduction
- ❑ Convolutional neural networks CNN
- ❑ Model Architectures
- ❑ Object Detection
- ❑ Segmentation
- ❑ Instance Segmentation
- ❑ Autoencoders

Instance Segmentation

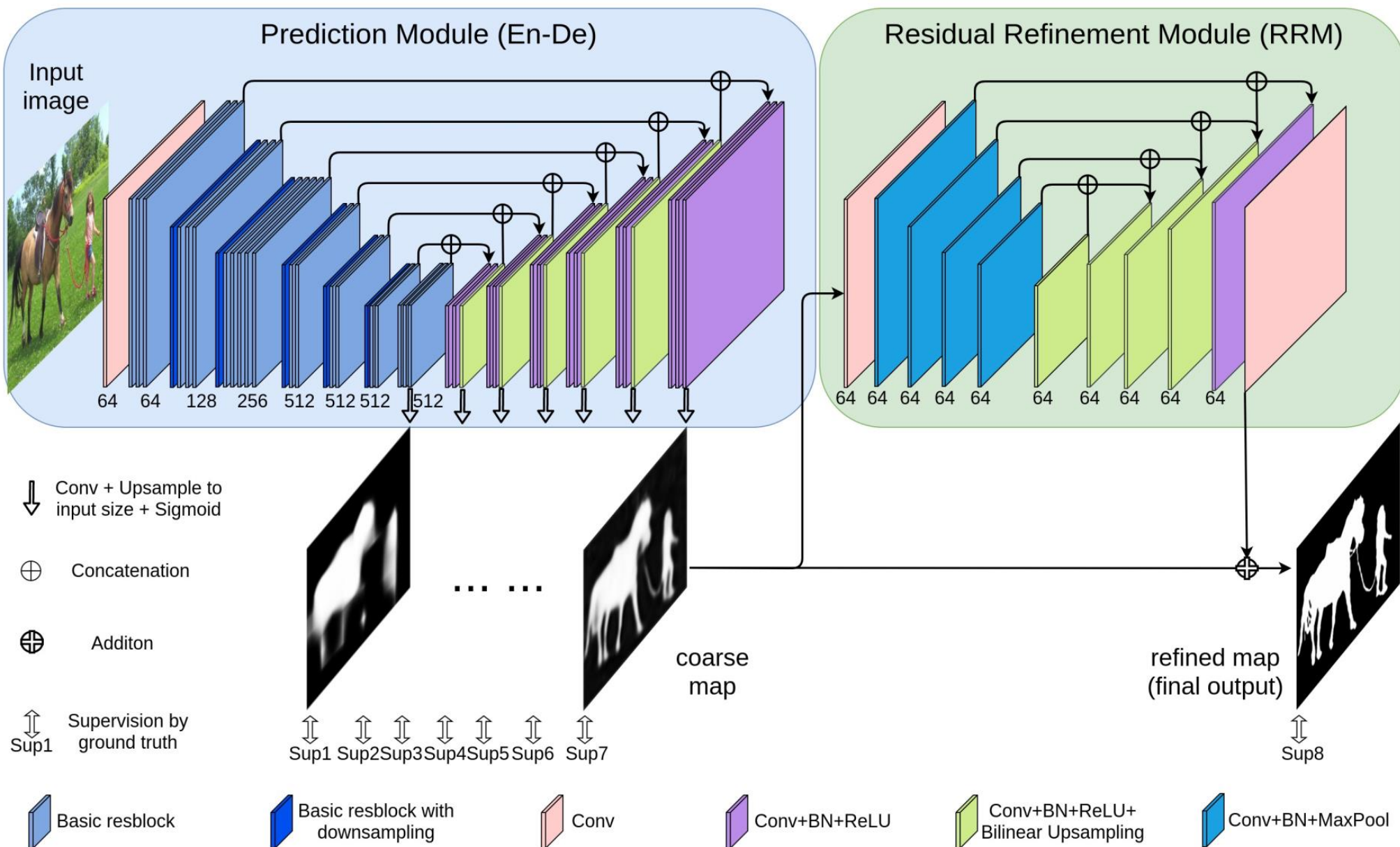


U-Net and Semantic Segmentation

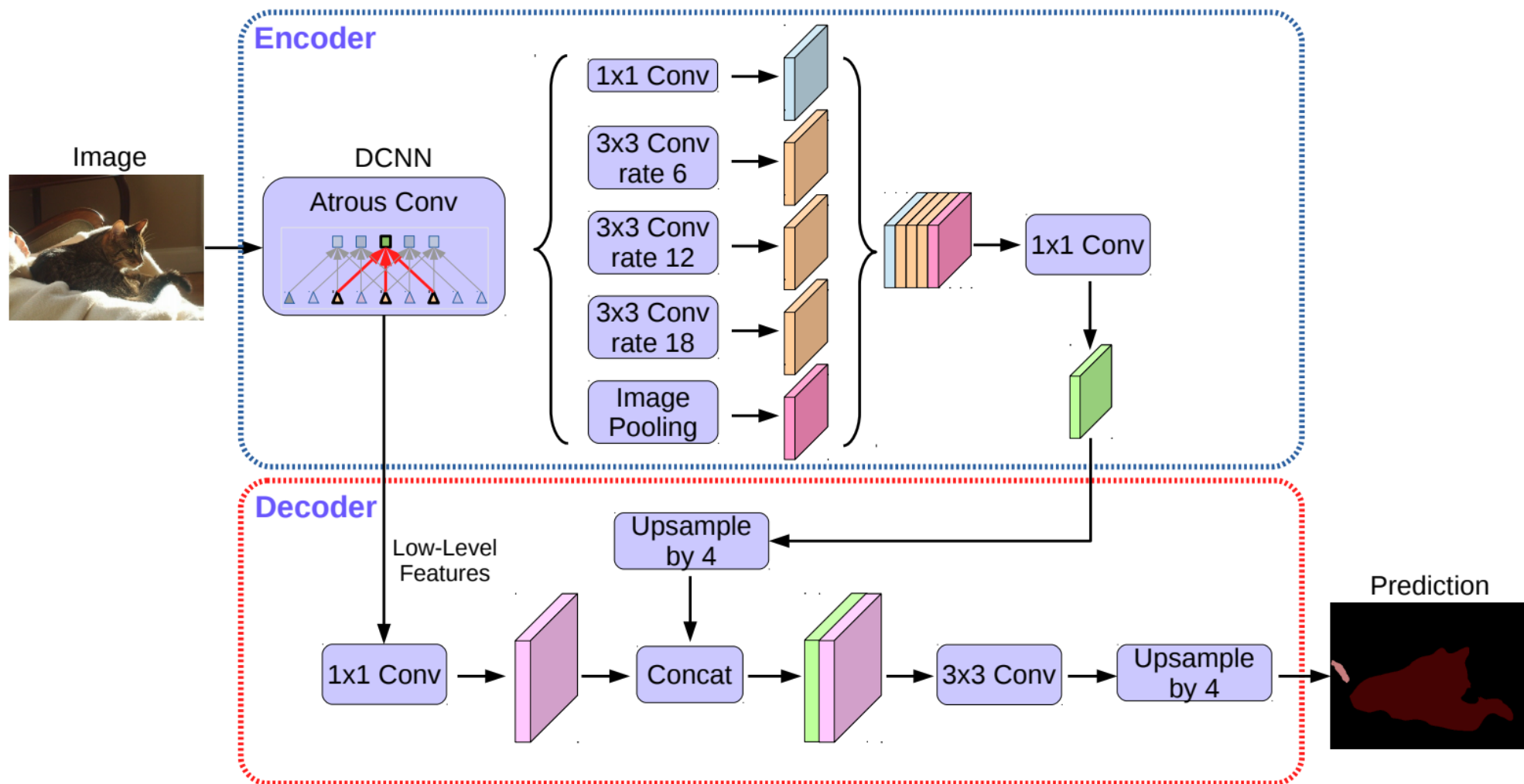


The U-Net architecture consists of mirrored encoder and decoder blocks that take on a U shape when depicted as shown here. Skip connections concatenate feature maps along the depth axis (channels). K is the target number of classes.

BASNet



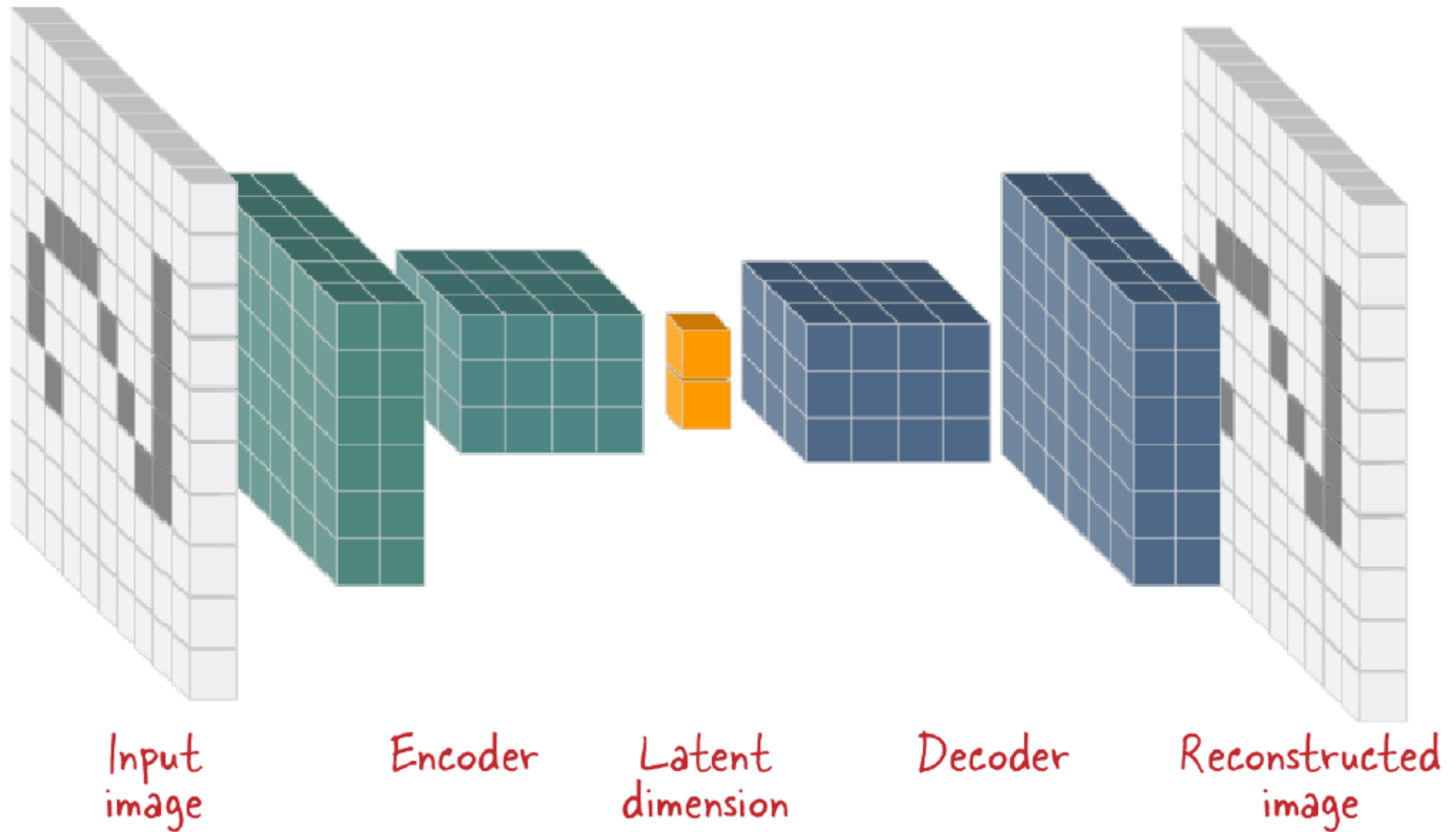
DeepLabV3



Autoencoders

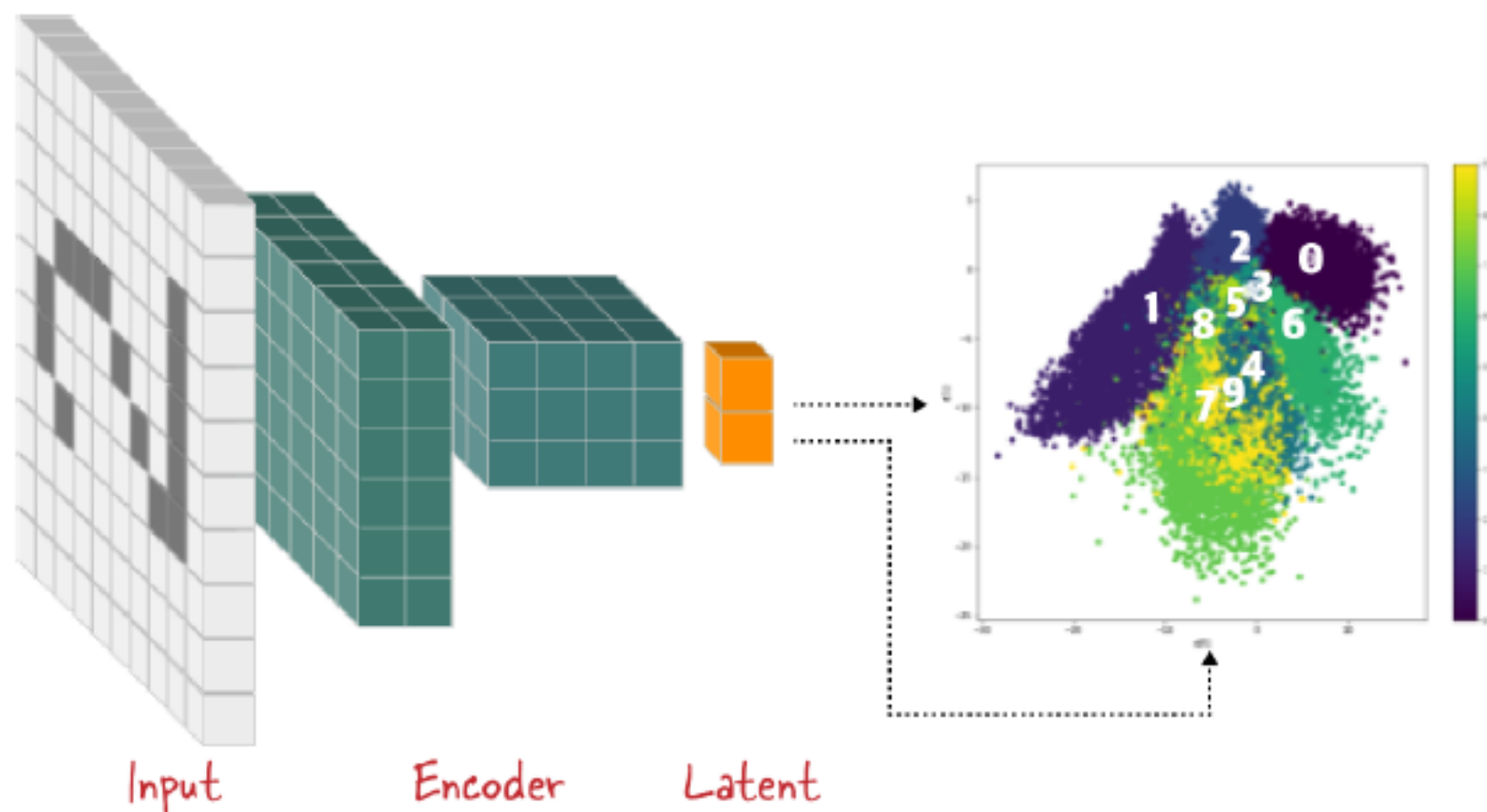
- ❑ Introduction
- ❑ Convolutional neural networks CNN
- ❑ Model Architectures
- ❑ Object Detection
- ❑ Segmentation
- ❑ Instance Segmentation
- ❑ Autoencoders

Autoencoders



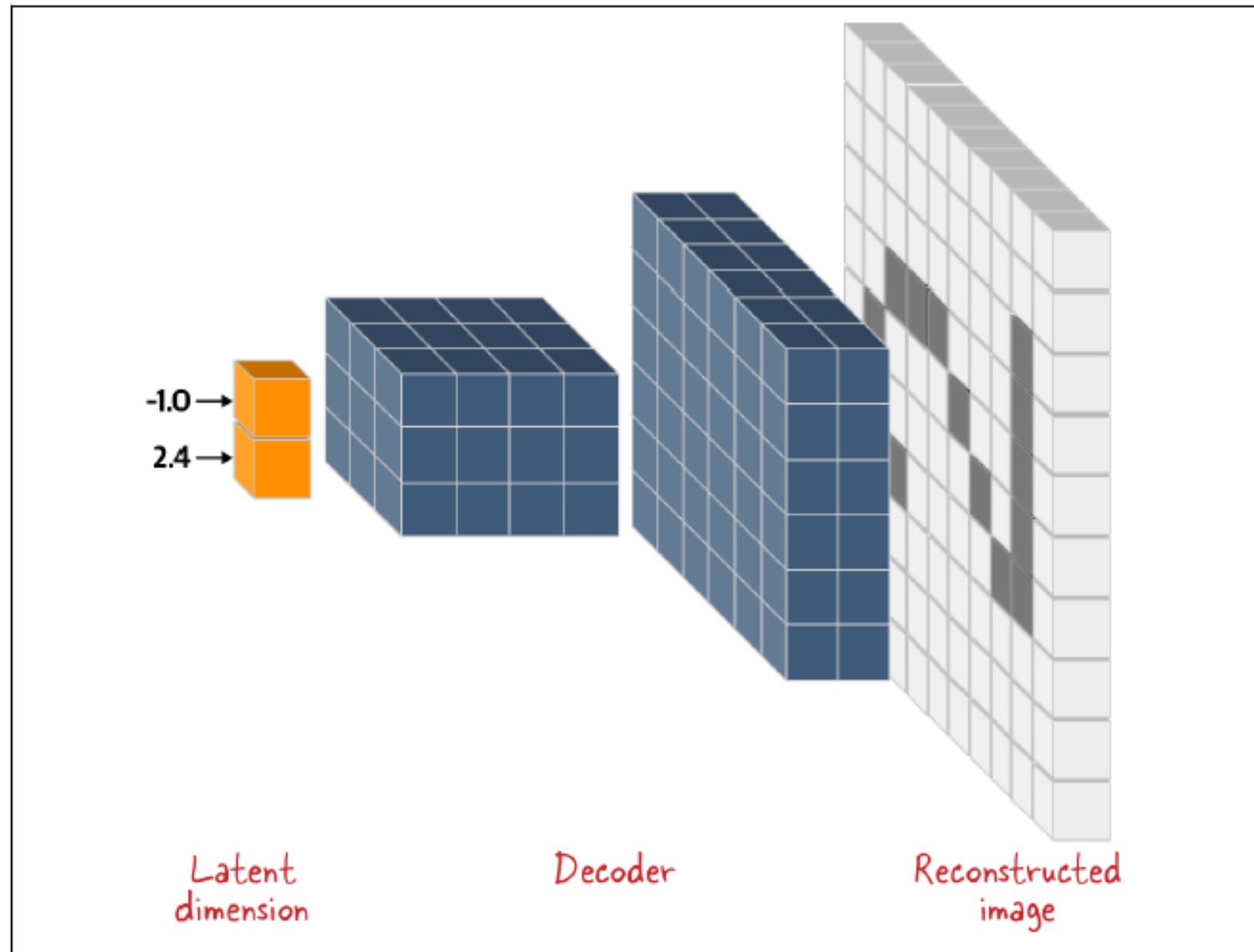
An autoencoder takes an image as input and produces the reconstructed image as output.

Autoencoders



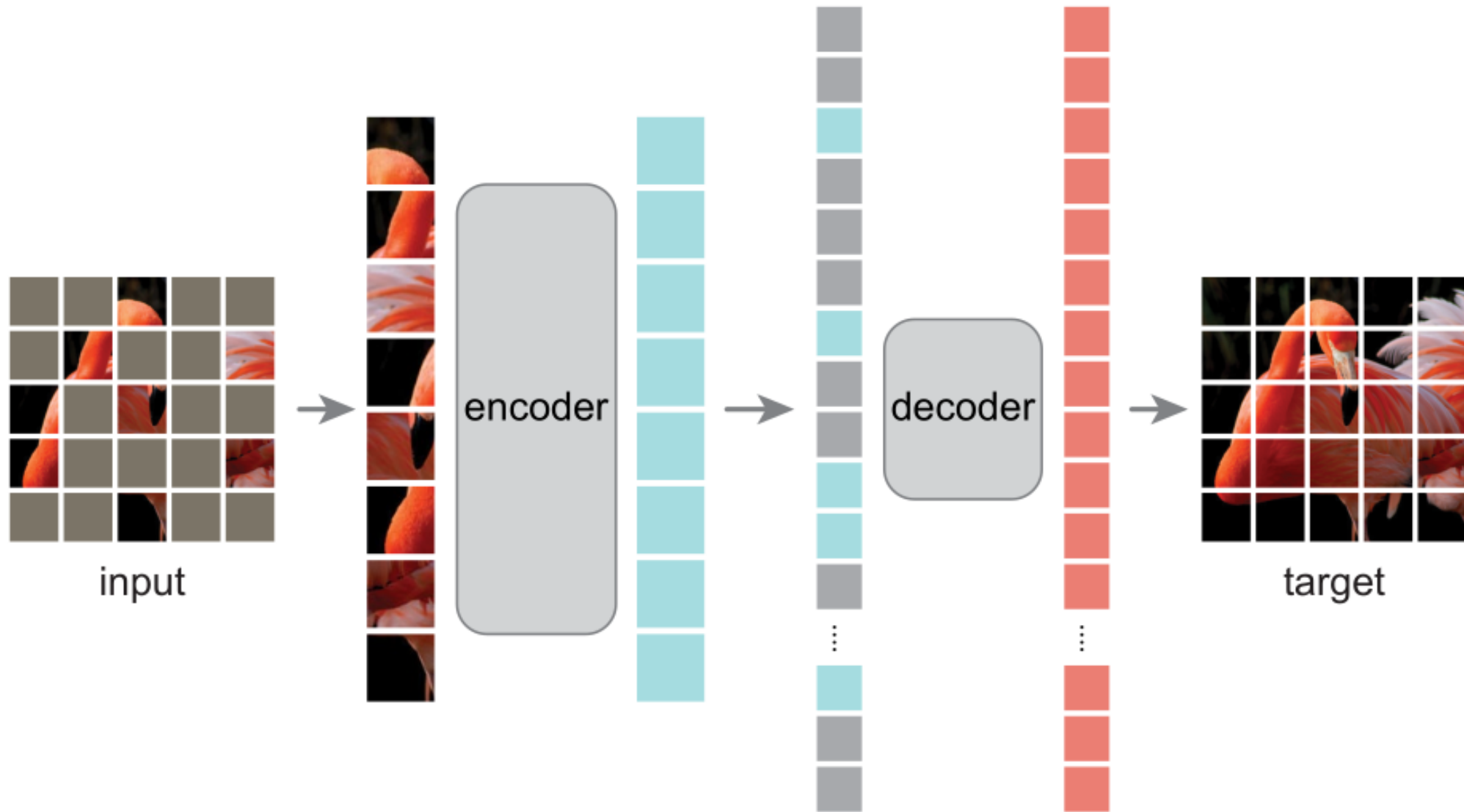
The encoder compresses input images into the latent representation. The latent representations for specific digits cluster together. We are able to represent the latent representations as points on a 2D graph because each latent representation is two numbers (x, y).

Autoencoders



The decoder decompressing a latent representation back into an image.

Masked Autoencoders Are Scalable Vision Learners



End-to-End Object Detection with Transformers

